



SAPIENZA
UNIVERSITÀ DI ROMA

Centralized Scheduling in Satellite Computing: An LP-based Approach

Faculty of Information Engineering, Computer Science and Statistics
Computer Science

Claudio Bagini

ID number 2045337

Advisor

Dr. Emiliano Casalicchio

Academic Year 2025/2026

Thesis not yet defended

Centralized Scheduling in Satellite Computing: An LP-based Approach

Bachelor's Thesis. Sapienza University of Rome

© 2026 Claudio Bagini. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Version: September 21, 2026

Author's email: claudio.bagini@gmail.com

*To countless cups of coffee, endless iterations,
and the version of me that refused to drop the task.*

Abstract

The evolution of Low Earth Orbit (LEO) satellite megaconstellations has opened new perspectives for Satellite Computing, enabling the extension of edge computing capabilities directly into space (Orbital Edge Computing). However, the highly dynamic network topology, stringent energy constraints of satellite nodes, and strict deadlines of real-time applications make task orchestration and offloading critical challenges.

This thesis presents the design, implementation, and experimental evaluation of a centralized orchestrator based on Integer Linear Programming (ILP). To mitigate solver computational costs and prevent starvation, the system integrates an event-driven dynamic batching mechanism controlled by Batch Size (BS) and Batch Timeout (BT) thresholds, along with a coordinated formulation of timing constraints that accounts for queuing delay prior to resolution.

Using discrete-event simulations developed within the SimPy framework, the proposed architecture is evaluated under stress conditions across varying request arrival rates (λ), comparing *Mapping Time* and *Mapping Energy* optimization strategies. The analysis investigates the systemic bottlenecks that emerge under elevated workloads, examining the interplay between physical compute capacity, network saturation, and temporal deadline constraints.

Finally, the centralized orchestrator is benchmarked against state-of-the-art distributed and heuristic solutions (*OrbitAware*, *DTS-base*, *DTS-APopt*, and distributed ILP) across heterogeneous workload profiles and operational energy budgets. Through this comprehensive evaluation, the thesis aims to characterize the operational boundaries and fundamental trade-offs between the theoretical global optimality of centralized mathematical scheduling and the responsiveness of decentralized models.

Contents

1	Introduction	1
1.1	Motivation and Context	1
1.2	Problem Statement	2
1.3	Objectives and Main Contributions	3
1.4	Thesis Outline	4
2	Background and State of the Art	5
2.1	Satellite Computing Paradigms	5
2.2	Task Offloading and Routing in LEO Networks	6
2.2.1	System Entities and Network Topology	6
2.2.2	Workload Profiles and Constraints	7
2.3	Centralized vs. Distributed Orchestration	8
2.3.1	Centralized Orchestration	8
2.3.2	Distributed Orchestration	8
2.4	State-of-the-Art Approaches	8
2.4.1	Distributed Heuristic Frameworks	9
2.4.2	Optimization-Based and Distributed ILP Formulations	9
2.5	Research Gap and Thesis Positioning	9
3	System Architecture, Mathematical Formulation, and Implementation	11
3.1	Regional Constellation Architecture and Offloading Model	11
3.2	Event-Driven Dynamic Batching Engine	14
3.3	Constellation Topology Mapping and Metric Normalization	16
3.4	Centralized Integer Linear Programming Formulation	19
3.4.1	Latency, Energy, and Orbital Eclipse Modeling	19
3.4.2	Multi-Objective Function and System Constraints	20
3.4.3	Solver Routine and Cascade Rejection Diagnostics	21
3.5	Task Execution and Resilient Hybrid Routing	27
3.6	Computational Complexity and Scalability Analysis	31
3.6.1	Topology Mapping and Pathfinding Complexity	31
3.6.2	ILP Formulation and Branch-and-Cut Solver Complexity	32
3.6.3	Hybrid Routing and Execution Forwarding Complexity	32
3.6.4	Constellation Scalability and Telemetry Overhead	33

4	Performance Analysis, Optimization, and Experimental Evaluation	34
4.1	Simulation Environment and Methodological Setup	35
4.1.1	Constellation Topology and Scenario Parameters	35
4.1.2	Two-Tier Optimization Methodology	35
4.2	Macro-Tuning: Exploration and Selection of Batching Policies	36
4.2.1	Success Rate Dynamics	36
4.2.2	Diagnostics of Failure Causes	39
4.2.3	Response Time Decomposition	42
4.2.4	Selection of Viable Batching Archetypes	45
4.3	Micro-Tuning: Multi-Objective Weight Calibration under Stress ($\gamma \times \alpha$)	45
4.3.1	Impact on Success Rate	46
4.3.2	Failure Diagnostics and Sunset Drop Elimination	47
4.3.3	Latency Inflation and Path-Bloat Dynamics	49
4.3.4	Selection of Calibrated Objective Weights	50
4.4	Consolidated Validation: Pre- vs. Post-Optimization Benchmark	51
4.4.1	Success Rate Recovery and Mapping Divergence	51
4.4.2	Rejection Causes Evolution	52
4.4.3	Response Time Containment and Execution Flattening	53
4.5	Summary of Findings and Architectural Readiness	54
5	Comparative Benchmark	56
5.1	Experimental Setup and Baseline Algorithms	56
5.1.1	Benchmark Dimensioning and Scenario Parameters	56
5.1.2	Evaluated Algorithms and State-of-the-Art Baselines	57
5.2	Comparative Performance Evaluation across Workload Profiles	59
5.2.1	Mixed Multi-Service Workload Profile ($\beta = (0.40, 0.35, 0.25)$)	59
5.2.2	Computation-Heavy Stress: CPU-Intensive Profile ($\beta = (0, 1, 0)$)	61
5.2.3	Joint Compute and Network Stress: CPU & Data-Intensive Profile ($\beta = (0, 0, 1)$)	64
5.3	Trade-Off Analysis and Operational Discussion	66
5.3.1	Theoretical Global Optimality vs. Ingress Queuing Latency	66
5.3.2	Energy Budget Sensitivity Across Computational Dimensions	67
5.3.3	Fast-Dispatch ($BT = 0.05$ s) vs. Controlled Small-Batching ($BS = 2$)	67
5.3.4	Architectural Synthesis and Deployment Guidelines	67
5.4	Summary of Chapter Findings	68
6	Conclusions and Future Work	69
6.1	Synthesis of Contributions and Research Question Resolution	69
6.1.1	Answering the Core Research Question	69
6.1.2	Algorithmic Archetypes and Workload Sensitivity	70
6.2	Critical Limitations of the Centralized Approach	71
6.2.1	Ingress Buffering and the Batching Dilemma	71
6.2.2	Single Point of Failure and Space Hazards	71
6.2.3	State Telemetry Overhead and Information Staleness	71
6.3	Future Research Directions	72
6.3.1	Two-Tier Hybrid Orchestration Architectures	72

6.3.2	Traffic-Aware Adaptive Batching via Reinforcement Learning	72
6.3.3	Multi-Agent Distributed Reinforcement Learning (MADRL)	72
6.3.4	Hardware-in-the-Loop (HIL) Prototyping on Space-Qualified Hardware	73
6.4	Concluding Remarks	73
Bibliography		74

Chapter 1

Introduction

In recent years, space communications and satellite networks have undergone a monumental paradigm shift. The rapid miniaturization of space components, combined with drastically reduced launch costs, has catalyzed the deployment of Mega-Constellations operating in Low Earth Orbit (LEO) [1]. Traditionally, satellite architectures functioned strictly under a "bent-pipe" model, where satellites served as passive transparent relays forwarding raw telemetry or communication signals back to Ground Stations (GS) for processing. However, the unprecedented volume of data produced by modern spaceborne applications—such as high-resolution Earth Observation (EO) [2], environmental monitoring, and ubiquitous Internet of Things (IoT) sensing [3]—has rendered this traditional pipeline highly inefficient.

To overcome the severe bandwidth limits of space-to-ground downlinks and the high propagation delays associated with remote terrestrial processing, the paradigm of **Orbital Edge Computing (OEC)**—also broadly termed Satellite Computing (SC)—has emerged [4, 5]. By equipping LEO satellites with onboard processing hardware, data can be computed directly in space close to the source, transmitting only actionable insights or highly compressed results to ground users. Despite its immense potential, executing compute-intensive tasks on resource-constrained satellite nodes introduces complex orchestration challenges that require advanced decision-making frameworks [1, 6].

1.1 Motivation and Context

The transition towards Orbital Edge Computing is driven by two main technological trends: the exponential increase in space-generated data and the strict temporal requirements of next-generation applications. Applications such as real-time disaster response, autonomous maritime navigation, and defense surveillance demand sub-second latency for task execution and reporting [2]. Processing these tasks on Earth introduces unavoidable latency bottlenecks due to intermittent Ground Station contact windows, dynamic weather interference, and severe downlink congestion.

LEO megaconstellations (such as Starlink, OneWeb, and Kuiper) consist of hundreds to thousands of satellites orbiting at altitudes between 300 km and 2000 km. These satellites move at high orbital velocities (approximately 7.5 km/s relative to the Earth's surface), resulting in a continuously changing network topology. Inter-

Satellite Links (ISLs)—utilizing high-throughput Free-Space Optics (FSO) or Radio Frequency (RF) channels—enable direct communication between adjacent satellites without routing signals down to the ground [7, 8].

Consequently, a LEO constellation effectively forms a distributed, high-velocity edge computing network in orbit. However, executing applications across this orbital fabric requires intelligent **Task Offloading and Routing Orchestration** [9, 10]. Ground users or source satellites generate computational tasks that must be dynamically assigned to suitable processing nodes across the constellation. Deciding *where* and *when* to execute a given task, while routing its associated payload across multiple dynamic ISL hops, constitutes a high-dimensional optimization problem that directly governs overall network performance.

1.2 Problem Statement

Orchestrating task offloading in Orbital Edge Computing environments presents a unique set of constraints and operational challenges that set it apart from conventional terrestrial cloud/edge systems [6]:

- **Severe Energy Budget Limits:** Satellite Edge Nodes (SENs) rely on solar panels and onboard battery storage. Computationally intensive task execution and RF/optical transmissions consume substantial power. Over-utilizing specific nodes leads to rapid battery depletion, thermal overhead, or total operational failure [6].
- **Heterogeneous and Limited Hardware Resources:** Onboard processing units (e.g., embedded CPUs, specialized AI accelerators) are constrained by strict Space, Weight, and Power (SWaP) requirements, as well as radiation-hardening constraints [5].
- **Dynamic Topology and Intermittent Connectivity:** High orbital mobility leads to frequent topology reconfigurations, changing link distances, and periodic loss of line-of-sight. Task routing paths must be continuously recalculated to avoid dropped links [7].
- **Strict Temporal Constraints and Queuing Delays:** Tasks possess strict deadlines (D_r). If the sum of transmission latency, queuing delay, and computation time exceeds D_r , the task fails, resulting in wasted system energy and degraded quality of service.

To achieve theoretical optimality in task placement and energy management, mathematical optimization tools such as **Integer Linear Programming (ILP)** are commonly employed [11, 12]. An ILP formulation can search the joint solution space of task routing, CPU allocation, and energy minimization to find globally optimal schedules.

However, centralized ILP solvers suffer from inherent computational complexity (NP-hard) [11]. In dynamic satellite networks with continuously arriving workloads, solving an ILP instance for every individual task request introduces unacceptable

computational overhead and processing latency at the central orchestrator. Conversely, accumulating task requests to solve them in batches introduces a trade-off: larger batches allow the ILP solver to achieve superior global resource allocation, but prolonged waiting times in orchestrator queues increase the risk of task deadline expiration (*Deadline Exceeded* failures).

1.3 Objectives and Main Contributions

The main objective of this thesis is to design, implement, and rigorously evaluate a **Centralized ILP Orchestration Framework** optimized for task routing and resource allocation in LEO satellite networks, incorporating a **Dynamic Batching Mechanism** to balance solver computational complexity with network queuing delay.

Specifically, this work addresses the key question: *To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?*

The main contributions of this thesis are summarized as follows:

1. **Formalization of the Centralized ILP Optimization Model:** We formulate a comprehensive ILP mathematical model that jointly optimizes task routing across Inter-Satellite Links and computational resource allocation on Satellite Edge Nodes. The model supports dynamic weighting between latency optimization (*Mapping Time*) and total energy preservation (*Mapping Energy*).
2. **Design of an Integrated Dynamic Batching & Timeout Mechanism:** To mitigate the computational overhead of solving the ILP model continuously, we integrate an adaptive batching policy governed by Batch Size (*BS*) and Batch Timeout (*BT*). Furthermore, we propose a mathematical deadline compensation formula to adjust task deadlines based on orchestrator queuing duration (T_{batch}).
3. **Design and Integration of the Centralized Orchestration Layer:** We design and implement the centralized ILP orchestration module and dynamic batching logic, integrating them into an existing Python/*SimPy* discrete-event simulation framework [13, 14]. The resulting environment accurately captures request arrival dynamics (λ), inter-satellite communication links, queue management, hardware processing, and fleet energy consumption under heterogeneous workload conditions.
4. **In-Depth Performance and Bottleneck Analysis:** We perform extensive stress testing under high workload arrival rates to identify system collapse thresholds, distinguishing between physical resource saturation (CPU/Network capacity) and temporal violations (*Deadline Exceeded*). We also conduct sensitivity analyses regarding deadline relaxation (ΔD).
5. **Comparative Benchmark Against State-of-the-Art Solutions:** We evaluate the centralized ILP orchestrator against established baseline heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP formulations

from the literature [6], establishing clear operational trade-offs regarding task completion rate, overall latency, and energy efficiency.

1.4 Thesis Outline

The remainder of this thesis is structured as follows:

- **Chapter 2 (Background and State of the Art):** Reviews the fundamental concepts of Satellite Computing, LEO network dynamics, and existing literature on task offloading. It provides a taxonomy of centralized versus distributed orchestration schemes and details key reference algorithms.
- **Chapter 3 (System Architecture and ILP Formulation):** Presents the formal mathematical system model, decision variables, objective functions (*Mapping Time* vs. *Mapping Energy*), constraints, and the dynamic batching/timeout mechanisms.
- **Chapter 4 (Performance Analysis and Evaluation):** Describes the SimPy discrete-event experimental setup and evaluates the proposed centralized orchestrator under varying batching configurations, heavy-load conditions, and deadline relaxation scenarios.
- **Chapter 5 (Comparative Benchmark):** Details the comparative analysis benchmarking our centralized orchestrator against state-of-the-art distributed heuristics and formulations, highlighting key trade-offs in latencies, energy consumption, and global optimality.
- **Chapter 6 (Conclusions and Future Work):** Summarizes the main findings of this research, outlines limitations of centralized orbital orchestration, and proposes future research directions toward hybrid adaptive architectures.

Chapter 2

Background and State of the Art

The rapid proliferation of Low Earth Orbit (LEO) satellite constellations has fundamentally redefined the landscape of satellite communications and space-borne computing [1]. Transitioning from traditional passive data relays to active processing nodes requires a comprehensive understanding of orbital dynamics, task offloading models, and orchestration strategies [4, 5]. This chapter provides the theoretical foundation and contextual literature for this thesis. It details the paradigms of Orbital Edge Computing (OEC), analyzes task offloading and routing dynamics in LEO networks, compares centralized and distributed orchestration models, reviews state-of-the-art baseline solutions, and articulates the research gap addressed by our work.

2.1 Satellite Computing Paradigms

Space-based networking architectures have historically been categorized by their orbital altitudes: Geostationary Earth Orbit (GEO), Medium Earth Orbit (MEO), and Low Earth Orbit (LEO) [1]. GEO satellites, positioned at approximately 35,786 km, offer stationary coverage over vast geographic areas but suffer from high round-trip propagation delays (exceeding 250 ms) and high free-space path loss. MEO satellites (2,000 km – 35,786 km) reduce latency slightly but remain unsuitable for ultra-low-latency processing.

In contrast, LEO satellites operate at altitudes ranging from 300 km to 2,000 km. Their proximity to Earth drastically lowers propagation latency (10 – 40 ms round-trip) and reduces power loss, making them ideal for high-throughput, real-time communications [7, 2]. However, LEO satellites travel at extreme velocities relative to the surface (~ 7.5 km/s), completing an orbit in approximately 90 – 120 minutes. This rapid mobility results in dynamic topologies where individual satellites remain in line-of-sight with a fixed ground position for only a few minutes [3].

To process data efficiently within this dynamic environment, satellite network architectures have evolved through three primary paradigms [4, 1]:

1. **Transparent Bent-Pipe Relay:** Satellites act merely as transparent repeaters. Raw data captured by Earth Observation (EO) sensors or transmitted by Ground Users (GUs) is amplified and forwarded directly to Ground Stations (GSs). Computation occurs entirely in terrestrial cloud centers. This

model suffers from severe downlink bottlenecks and complete connectivity loss when flying over remote regions (e.g., oceans or polar zones) lacking GS infrastructure [2].

2. **Terrestrial Cloud Offloading with Inter-Satellite Routing:** Satellites utilize Inter-Satellite Links (ISLs) to route raw data across orbital planes until a satellite with an active GS connection is reached. While this mitigates local GS coverage gaps, the volume of raw data still strains optical and RF link bandwidth [7, 8].
3. **Orbital Edge Computing (OEC) / Satellite Edge Computing (SEC):** Satellites are equipped with onboard processing units—termed Satellite Edge Nodes (SENs)—capable of executing compute-intensive algorithm payloads (e.g., AI inference, image compression, signal filtering) directly in orbit [5, 10]. Raw data is processed locally at the source node or offloaded across ISLs to adjacent SENs. Only refined, actionable insights or small computation results are downlinked to Earth, drastically reducing bandwidth consumption and end-to-end response times [3, 15].

2.2 Task Offloading and Routing in LEO Networks

Task offloading in Orbital Edge Computing involves receiving task execution requests generated by Ground Users (GUs) or space-borne sensors and strategically allocating their computational execution across available SENs while routing payloads through Inter-Satellite Links (ISLs) [9, 6].

2.2.1 System Entities and Network Topology

An OEC network architecture consists of three core structural entities [6, 16]:

- **Ground Users (GUs) / Remote Sensors:** Distributed terrestrial entities that issue computational task requests r_i . Each request is characterized by a data payload size D_r^{data} (in bits), required CPU compute cycles W_r , and a strict operational deadline D_r (in seconds) [6].
- **Satellite Edge Nodes (SENs):** LEO satellites equipped with CPU cores, memory modules, and onboard energy reserves (solar panels and battery banks). Each SEN s_j possesses a processing capacity f_j (cycles/second) and an operational energy budget E_j [6, 17].
- **Inter-Satellite Links (ISLs):** High-speed communication channels connecting adjacent satellites. Intra-plane ISLs connect satellites within the same orbital plane (stable distance), whereas inter-plane ISLs connect satellites in adjacent planes (dynamically varying distance and pointing angles) [7, 8]. Each ISL link (u, v) is constrained by bandwidth B_{uv} and propagation delay τ_{uv} .

2.2.2 Workload Profiles and Constraints

Unlike conventional terrestrial cloud datacenters operating on uniform, high-throughput fabrics, Orbital Edge Computing must accommodate highly heterogeneous task demands under severe physical boundaries [6]. In spaceborne environments, computational workloads are fundamentally characterized by their temporal tolerance and their resource footprint [4, 2]:

- **Real-Time, Mission-Critical Workloads:** Time-sensitive applications—such as real-time disaster detection, defense alerts, and active maritime tracking—demanding sub-second or deterministic response latencies ($D_r \leq 1.0\text{--}2.0$ s) [2]. For these services, queuing delays or multi-hop routing stalls directly jeopardize operational utility.
- **Batch and Delay-Tolerant Workloads:** Routine background processes, including scientific telemetry compression, orbital ephemeris synchronization, and non-urgent Earth Observation indexing [4]. These tasks tolerate deferred execution or multi-hop store-and-forward routing, running during operational lulls when satellite resources are underutilized.

Orthogonally to their temporal deadline, workloads impose asymmetric resource stress across the orbital infrastructure [6]: *computation-heavy* tasks demand extensive CPU execution cycles on the processing unit, whereas *data-intensive* transfers require transmitting multi-megabyte payloads across Inter-Satellite Links, driving network interface and transceiver saturation [6].

Consequently, task offloading and scheduling decisions must jointly satisfy four coupled systemic constraints [9, 6]:

1. **Computational Capacity:** The aggregate processing demand scheduled on satellite node j cannot exceed its physical onboard compute frequency f_j and memory limits, avoiding processor starvation and excessive queue backlog.
2. **Network Link Bandwidth:** Payload traffic routed across directed Inter-Satellite Link (u, v) is strictly bounded by the available transceiver bandwidth B_{uv} , preventing packet buffer overflows and queue-induced latency inflation.
3. **Fleet Energy Budgets:** Because satellites rely on cyclic photovoltaic recharging and finite onboard battery storage (E_j), the cumulative energy expended for RF/optical transmission (P_{tx}) and dynamic CPU switching (P_{comp}) must remain within available battery thresholds per orbital pass.
4. **Strict End-to-End Service Deadlines:** The end-to-end task completion latency—spanning ingress queuing, multi-hop uplink ISL forwarding (T_{up}), onboard queuing and execution (T_{cpu}), and downstream result return (T_{down})—must strictly comply with the application deadline:

$$T_{total}(r) = T_{up}(r) + T_{cpu}(r) + T_{down}(r) \leq D_r \quad (2.1)$$

Exceeding D_r triggers immediate task drops (*Deadline Exceeded*), wasting both scarce constellation energy and orbital link capacity.

2.3 Centralized vs. Distributed Orchestration

Managing resource allocation across hundreds of moving SENSs requires an orchestration paradigm [1]. Literature categorizes these paradigms into two primary architectures: Centralized and Distributed.

2.3.1 Centralized Orchestration

In a centralized orchestration architecture, a dedicated master entity—such as a designated lead satellite (Orbital Orchestrator) or a ground control station—collects global network state information (CPU load, battery state, link traffic) from all SENSs [1, 6].

- **Advantages:** The central orchestrator maintains complete global visibility over all network resources, enabling mathematically optimal decision-making. Global optimization models (e.g., Integer Linear Programming) can eliminate load imbalances, prevent local hotspots, and minimize network-wide energy consumption [11, 12].
- **Disadvantages:** Centralized schemes incur significant control signaling overhead to aggregate state information across the constellation. Furthermore, routing all requests to a central orchestrator creates queuing delays and bottlenecks under high request arrival rates (λ) [6]. Most critically, relying on a centralized controller introduces a Single Point of Failure (SPOF): if the orchestrator experiences hardware faults, orbital visibility loss, or link outages, the entire scheduling and task-handling pipeline is compromised, severely undermining system fault tolerance [5].

2.3.2 Distributed Orchestration

In distributed orchestration, individual SENSs or localized clusters make autonomous offloading and routing decisions based solely on locally observed state information or partial neighborhood telemetry [18, 6].

- **Advantages:** Distributed schemes eliminate centralized control bottlenecks, offer sub-millisecond decision latency, and demonstrate high fault tolerance against dynamic topology changes and satellite failures [6].
- **Disadvantages:** Lacking global visibility, distributed algorithms often converge to sub-optimal solutions. They are prone to localized network congestion, unbalanced CPU utilization, and premature battery depletion on popular forwarding paths [19, 9].

2.4 State-of-the-Art Approaches

To place our research in context, we review key baseline algorithms and mathematical models established in recent IEEE literature on Orbital Edge Computing and Task Offloading.

2.4.1 Distributed Heuristic Frameworks

Heuristic approaches sacrifice theoretical optimality in favor of computational speed and low decision latency [6]:

- **OrbitAware Architecture:** A dynamic heuristic framework designed specifically for highly dynamic LEO topologies. *OrbitAware* leverages periodic orbital trajectory updates to predict link availability and select shortest processing/routing paths locally, reducing control overhead at the cost of potential link queue saturation [6].
- **DTS-base (Dynamic Task Scheduling - Base):** A greedy distributed scheduling baseline that assigns incoming tasks to the nearest available SEN with sufficient idle CPU capacity. While fast, it ignores global energy budgets and multi-hop link traffic [6].
- **DTS-APopt (Dynamic Task Scheduling - Access Point Optimized):** An enhanced version of *DTS-base* that incorporates multi-criteria decision metrics, optimizing both the local access point selection and initial ISL hop bandwidth before dispatching tasks [6].

2.4.2 Optimization-Based and Distributed ILP Formulations

To improve allocation quality, recent studies have formulated mathematical optimization models using Integer Linear Programming (ILP) and mixed-integer techniques [11, 12]:

- **Distributed / Hierarchical ILP:** Frameworks that decompose a global ILP problem into smaller sub-problems solved locally across satellite clusters [6]. While more scalable than pure centralized ILP, distributed ILP approaches suffer from convergence delays and coordination overhead during inter-cluster task transfers [17].
- **Weighted Objective Functions:** Optimization models in the literature frequently balance competing goals—such as latency minimization versus energy preservation—by assigning scalar weights (W_{lat}, W_{en}) to objective terms [9, 6].

2.5 Research Gap and Thesis Positioning

While state-of-the-art distributed heuristics offer rapid processing times and distributed ILP frameworks attempt local decomposition, a significant research gap remains regarding the practical limits of **Centralized ILP Orchestration** in LEO satellite networks [6, 1].

Existing literature frequently assumes either ideal zero-latency centralized orchestrators or dismisses centralized optimization entirely due to its computational complexity. Few works rigorously examine the trade-offs when a centralized ILP orchestrator is equipped with an integrated **Dynamic Batching** and **Timeout Mechanism** (BS/BT) to manage request queues under heavy workloads ($\lambda > 6$ req/s) [6].

This thesis addresses this exact gap by:

1. Formalizing a centralized ILP model that dynamically toggles between latency minimization (*Mapping Time*) and energy conservation (*Mapping Energy*) [6].
2. Quantifying the systemic performance bounds of centralized optimization when coupled with dynamic request batching and queuing delay compensation.
3. Conducting a rigorous comparative benchmark against established state-of-the-art heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP variants under realistic discrete-event simulation conditions (*SimPy*) [13, 6].

Chapter 3

System Architecture, Mathematical Formulation, and Implementation

This chapter provides a comprehensive dissertation on the theoretical framework, mathematical optimization formulation, and concrete software architecture of the proposed Centralized Integer Linear Programming (ILP) Orchestrator. To avoid conceptual disconnection between the theoretical modeling and its practical realization, each section progressively couples the formal system dynamics with its executable implementation in Python, utilizing the `SimPy` discrete-event simulation engine [13] and the `PuLP` mathematical programming library [14] backed by the open-source `HiGHS` solver [20].

We formalize the regional Low Earth Orbit (LEO) constellation environment, derive the multi-objective optimization problem governing task assignment and multi-hop Inter-Satellite Link (ISL) routing, detail the dynamic batching and greedy recovery mechanisms, and conclude with a rigorous computational complexity analysis.

3.1 Regional Constellation Architecture and Offloading Model

We consider a regional orbital edge computing network deployed within a Low Earth Orbit (LEO) satellite constellation [1]. The orbital network is modeled as a regional Walker-Delta LEO geometry [16]. The system comprises a set of candidate Satellite Edge Nodes (SENs), denoted by $S = \{1, 2, \dots, n\}$, and a set of regional Access Points (APs) or satellite gateways AP_{global} [6]. Within the operational region, a designated *Master* functions as the centralized orchestrator, maintaining global visibility over constellation kinematics, dynamic processor queue occupancies, residual battery energy budgets, and active inter-satellite link topologies.

Ground Users (GUs) generate computational requests that arrive dynamically at the regional gateways. Each offloading request $r \in R$ is characterized by a

five-element tuple:

$$r = (d_{cpu}, D_r, s_r, t_{arr}, AP_r) \quad (3.1)$$

where d_{cpu} is the computational CPU workload demand, D_r represents the maximum allowable service processing deadline, s_r denotes the input/output image size (in MB), t_{arr} is the generation timestamp, and $AP_r \in AP_{global}$ identifies the originating entry point [6].

To achieve robust real-time execution in highly dynamic space environments, the operational lifecycle of task r is modeled as a decoupled, three-phase circuit:

1. **Uplink Control and Forwarding Phase (T_{up}):** The originating gateway AP_r injects the payload into the orbital plane, which is routed across $h_{up}(r, i)$ ISL hops to candidate satellite $i \in S$ [6].
2. **On-Board Computational Execution Phase (T_{cpu}):** Satellite i processes d_{cpu} units of work, incurring both the deterministic waiting time in the on-board processor queue $W_{cpu,i}$ and the physical execution delay [4, 6].
3. **Post-Execution Geographic Result Delivery (T_{down}):** Upon computation completion, the generated output payload is routed back from the executing satellite i to the originating Access Point AP_r across the dynamic ISL topology via an autonomous geographic greedy routing protocol.

Because result return is governed dynamically in real time by the autonomous greedy protocol, the centralized ILP orchestrator focuses its optimization capacity on the joint satellite selection, uplink multi-hop routing, and processing deadline compliance.

To formalize the optimization problem and network dynamics unambiguously, Table 3.1 summarizes the foundational notations, sets, and operational parameters adopted throughout the system model and ILP formulation.

Table 3.1. Summary of mathematical notations and system parameters used in the Centralized ILP Orchestrator model.

Category	Symbol	Description
Sets & Topology	R	Set of task requests accumulated in the active batch ($ R \leq BS$).
	S	Set of candidate Satellite Edge Nodes (SENs) in the regional constellation.
	AP_{global}	Set of active regional Access Points / satellite gateways.
	AP_r	Originating Access Point for computational task request r .
	$\vec{P}_{up}(AP_r, i)$	Pre-computed shortest multi-hop ISL path from AP_r to satellite i .
Task Requests	r	Task tuple: $r = (d_{cpu,r}, D_r, s_r, t_{arr}, AP_r)$.
	$d_{cpu,r}$	Computational workload processing demand.
	D_r	Maximum allowable end-to-end service completion deadline.
	s_r	Input/output dataset payload size.
	t_{arr}	Task generation timestamp at the Access Point.
	S_{req}, S_{ref}	Uplink control payload and reference link payload.
Network Links	$\tau_{u,v}$	Propagation delay of the directed link (u, v) .
	$B_{u,v}$	Available transmission bandwidth of link (u, v) .
	P_{net}	Radio frequency / optical laser transmission power coefficient (P_{trasm}).
	$c_{link}(u, v)$	Normalized dimensionless cost of link (u, v) in Dijkstra mapping.
	w_R, w_e	Routing cost strategy weights for delay and energy ($w_R + w_e = 1$).
	$W_{net,u}$	Network queue waiting time at intermediate forwarding node u .
Satellite Nodes	$C_{sen,i}$	Processor clock frequency / compute capacity of satellite i .
	κ_i	Dynamic CPU energy switching capacitance coefficient (e_coeff).
	B_i	Usable residual battery energy budget of satellite i .
	$W_{cpu,i}$	Estimated queuing delay in the onboard processor execution queue.
	$t_{sunset,i}$	Solar visibility horizon / orbital eclipse timestamp for satellite i .
	Ω_i	Normalized orbital eclipse penalty factor.
Optimization & ILP	$x_{r,i}$	Binary decision variable (1 if task r is assigned to SEN i ; 0 otherwise).
	y_r	Binary rejection variable (1 if task r is dropped by the solver; 0 otherwise).
	$R_{r,i}$	Total predicted response time for assigning task r to satellite i .
	$\epsilon_{r,i}$	Fleet cumulative energy expenditure for routing and executing task r on i .
	P_{drop}	Heavy penalty scalar applied to rejected tasks ($P_{drop} = 10^6$).
	α, β, γ	Multi-objective hierarchical weighting factors (Time, Energy, Sunset).
	BS, BT	Batch Size and Batch Timeout control thresholds for dynamic batching.

3.2 Event-Driven Dynamic Batching Engine

In a high-throughput satellite environment, invoking an NP-hard ILP solver upon each individual task arrival introduces prohibitive computational overhead and CPU starvation [11, 6]. To address this, the Master Orchestrator implements an event-driven dynamic batching mechanism inside a `SimPy` discrete-event framework [13], governed by two competing thresholds:

- **Maximum Batch Size (BS):** A structural threshold (`centralized_batch_size`) that limits the maximum number of requests solved concurrently, bounding the solver variable space $|R| \times |S|$.
- **Batch Timeout (BT):** A temporal threshold (`centralized_batch_timeout`) that enforces a maximum buffering delay, ensuring requests are not stalled indefinitely.

The execution loop remains quiescent until the arrival of the first request into `task_buffer`, which immediately succeeds `item_arrived_event` and initiates the timeout timer. The batch is dispatched when either $|Q| \geq BS$ (triggering `batch_ready_event`) or when the elapsed time satisfies $\Delta t \geq BT$.

A critical design requirement in discrete-event simulation is ensuring state consistency. Because satellite positions, CPU queue lengths, and energy levels evolve continuously according to orbital ephemeris [21], the orchestrator executes a zero-delay yield (`yield self.env.timeout(0)`) immediately prior to batch processing. This yields thread execution back to the `SimPy` event loop, ensuring that all physical updates across the constellation at timestamp t are fully committed before optimization begins.

Listing 3.1 presents the software architecture of the `Orchestrator` class initialization, request enqueueing, and the background lifecycle process.

```

1  import heapq
2  import simpy
3  import globals
4  import pulp
5  import ILP_simulation
6  import utils
7  from Task import Task
8  from EdgeServer import get_pos_proximity
9
10
11 class Orchestrator:
12     def __init__(self, env, config):
13         self.env = env
14         self.config = config
15
16         # Retrieve batch parameters from configuration
17         self.batch_size = config.get("centralized_batch_size", 5)
18         self.batch_timeout = config.get("centralized_batch_timeout", 0.25)
19
20         self.task_buffer = []
21
22         # SimPy event handlers
23         self.item_arrived_event = self.env.event()

```

```

24     self.batch_ready_event = self.env.event()
25
26     # Spawn orchestrator background process
27     self.env.process(self._run_loop())
28
29     def add_task(self, task_data, task_id, visible_aps):
30         """
31         Enqueues an incoming task into the buffer and evaluates event triggers.
32         """
33         if not self.task_buffer and not self.item_arrived_event.triggered:
34             self.item_arrived_event.succeed()
35
36         if "arrival_time" not in task_data:
37             task_data["arrival_time"] = self.env.now
38
39         self.task_buffer.append({
40             "task_id": task_id,
41             "data": task_data,
42             "visible_aps": visible_aps,
43             "arrival_time": self.env.now
44         })
45
46         if len(self.task_buffer) >= self.batch_size and not self.batch_ready_event.
↳ triggered:
47             self.batch_ready_event.succeed()
48
49     def _run_loop(self):
50         """
51         Main orchestrator lifecycle loop, handling non-blocking waits within the
↳ simulation.
52         """
53         while True:
54             # 1. Wait idle for the arrival of the FIRST task
55             if not self.task_buffer:
56                 yield self.item_arrived_event
57
58             # 2. Start batch aggregation timer (Timeout)
59             timeout_event = self.env.timeout(self.batch_timeout)
60
61             # 3. Wait until either condition is met
62             yield self.batch_ready_event | timeout_event
63
64             # 4. Process accumulated batch tasks
65             if self.task_buffer:
66                 trigger_reason = "Maximum Capacity" if self.batch_ready_event.triggered
↳ else "Timeout"
67                 print(f"[{self.env.now:.3f}] Orchestrator: ILP trigger due to {
↳ trigger_reason} ({len(self.task_buffer)} tasks).")
68
69                 # Yield control to event loop to process physical satellite state updates
70                 yield self.env.timeout(0)
71
72                 self._process_batch()
73
74             # 5. Reset events for the subsequent cycle
75             self.item_arrived_event = self.env.event()
76             self.batch_ready_event = self.env.event()

```

```

77
78 def _process_batch(self):
79     """
80     Core routine orchestrating the resolution and dispatching pipeline.
81     """
82     # A. Map constellation topology and state
83     network_state = self._map_constellation()
84
85     # B. Invoke PuLP ILP solver
86     assignments = self._solve_ilp(network_state, self.task_buffer)
87     print(f"[{self.env.now:.3f}] Orchestrator: ILP completed.")
88
89     # C. Dispatch tasks based on optimization results
90     self._dispatch_tasks(assignments)
91
92     # Clear task buffer for subsequent batch
93     self.task_buffer.clear()

```

Listing 3.1. Orchestrator Class Architecture and SimPy Event Loop.

3.3 Constellation Topology Mapping and Metric Normalization

Before formulating the optimization model, the Master Orchestrator must compute the optimal multi-hop ISL routing paths from every active ground Access Point to all candidate Satellite Edge Nodes. This is handled by the `_map_constellation` method using a Multi-Source Dijkstra shortest-path algorithm [22].

Evaluating dynamic satellite links with raw physical dimensions introduces dimensional bias (e.g., comparing milliseconds of propagation delay against microjoules of radio energy). To achieve uniform metric evaluation across heterogeneous links, each directed edge (u, v) with latency $\tau_{u,v}$ and bandwidth $B_{u,v}$ (in Bytes/s) is evaluated against a standardized reference payload $S_{ref} = 1$ MB (1024^2 bytes) over optical ISL channels [7, 8]:

$$t_{link}(u, v) = \tau_{u,v} + \frac{S_{ref}}{B_{u,v}} \quad (3.2)$$

$$e_{link}(u, v) = P_{net} \cdot \frac{S_{ref}}{B_{u,v}} \quad (3.3)$$

where P_{net} is the radio/laser transmission power (`Ptrasm`) [6].

The edge metrics are normalized against theoretical upper bounds (T_{max} and E_{max}) derived from minimum network bandwidth ($B_{min} = 100$ KB/s) and maximum link latency ($\tau_{max} = 100$ ms):

$$\hat{t}_{link}(u, v) = \frac{t_{link}(u, v)}{T_{max}}, \quad \hat{e}_{link}(u, v) = \frac{e_{link}(u, v)}{E_{max}} \quad (3.4)$$

The composite dimensionless edge weight $c_{link}(u, v)$ is computed as a convex combination parameterized by weights w_R (`centralized_w_r`) and w_e (`centralized_w_e`):

$$c_{link}(u, v) = w_R \cdot \hat{t}_{link}(u, v) + w_e \cdot \hat{e}_{link}(u, v) \quad (3.5)$$

Concurrently, the routine queries all candidate nodes in `globals.edge_servers` to capture residual energy B_i , processor queue occupancy $W_{cpu,i}$ (`W_cpu_simp`), network queue occupancy $W_{net,i}$ (`W_net_simp`), processor clock speed $C_{sen,i}$, CPU energy coefficient κ_i (`e_coeff`), and orbital sunset timestamp $t_{sunset,i}$.

Listing 3.2 shows the executable implementation of the constellation mapping procedure.

```

1  def _map_constellation(self):
2      """
3      Maps shortest paths from each AP to all candidate satellites (SENs) via Dijkstra.
4      Link cost is formulated as a weighted combination of Delay and Energy.
5      """
6      # Retrieve objective cost function weights from configuration
7      w_e = self.config.get("centralized_w_e", 0.5)
8      w_R = self.config.get("centralized_w_r", 0.5)
9      p_trasm = self.config.get("Ptrasm", 1.0)
10
11     # Standardize link metrics using a 1 MB reference payload (in bytes)
12     REF_DATA_BYTES = 1024 * 1024
13
14     network_state = {
15         "routing_table": {}, # Format: {ap_name: {sen_name: {'cost': X, 'time': Y, '
    ↪ energy': Z, 'path': [...]}}}
16         "node_states": {}    # Format: {sen_name: {'energy_residual': X, 'cpu_queue':
    ↪ Y, ...}}
17     }
18
19     # 1. Collect all topology nodes to facilitate network graph traversal
20     all_nodes = globals.global_access_point + globals.edge_servers
21     node_dict = {n.name: n for n in all_nodes}
22
23     # 2. Multi-Source Dijkstra: explore network topology from each AP
24     for ap in globals.global_access_point:
25         network_state["routing_table"][ap.name] = {}
26
27         # Priority queue tuple structure:
28         # (total_cost, total_time, total_energy, current_node_name, path_so_far)
29         pq = [(0.0, 0.0, 0.0, ap.name, [ap.name])]
30
31         # Track minimum discovered cost to reach each node from this AP
32         min_costs = {ap.name: 0.0}
33
34         while pq:
35             current_cost, current_time, current_energy, current_node_name, path =
    ↪ heapq.heappop(pq)
36
37             # Skip obsolete (sub-optimal) popped paths
38             if current_cost > min_costs.get(current_node_name, float('inf')):
39                 continue
40
41             # If current node is a satellite (SEN), store its finalized shortest path
42             current_node_obj = node_dict[current_node_name]
43             if current_node_obj in globals.edge_servers:
44                 network_state["routing_table"][ap.name][current_node_name] = {
45                     "total_cost": current_cost,
46                     "total_time": current_time,
47                     "total_energy": current_energy,

```

```

48         "path": path
49     }
50
51     # Explore physical neighbors of current node
52     for neighbor in current_node_obj.get_neighbors():
53
54         if neighbor.name not in node_dict:
55             continue
56
57         # Extract physical communication link metrics
58         latency = current_node_obj.get_latency(neighbor) or 0.0
59         bw_MBps = current_node_obj.get_bandwidth(neighbor) or 0.0
60         bw_Bps = bw_MBps * (1024**2)
61
62         # Interrupted link if available bandwidth is non-positive:
63         if bw_Bps <= 0:
64             continue
65
66         # Set conservative estimates for network bandwidth lower bound and
67         latency upper bound
68         MIN_BW_BPS = 100000 # Example: 100 KB/s
69         MAX_LATENCY = 0.1 # Example: 100 ms
70
71         # Compute theoretical upper bounds for reference payload
72         MAX_LINK_TIME = MAX_LATENCY + (REF_DATA_BYTES / MIN_BW_BPS)
73         MAX_LINK_ENERGY = current_node_obj.compute_routing_energy(
74             REF_DATA_BYTES, bw_Bps, p_trasm)
75
76         # Compute latency and energy required to transmit reference payload
77         trans_time = REF_DATA_BYTES / bw_Bps
78         link_time = latency + trans_time
79         link_energy = current_node_obj.compute_routing_energy(REF_DATA_BYTES,
80             bw_Bps, p_trasm)
81
82         # Min-Max Normalization
83         norm_time = link_time / MAX_LINK_TIME if MAX_LINK_TIME > 0 else 0.0
84         norm_energy = link_energy / MAX_LINK_ENERGY if MAX_LINK_ENERGY > 0
85         else 0.0
86
87         # Composite dimensionless link cost
88         link_cost = (w_R * norm_time) + (w_e * norm_energy)
89
90         new_cost = current_cost + link_cost
91         new_time = current_time + link_time
92         new_energy = current_energy + link_energy
93
94         # Update shortest path if a lower-cost route is found
95         if new_cost < min_costs.get(neighbor.name, float('inf')):
96             min_costs[neighbor.name] = new_cost
97             heapq.heappush(pq, (new_cost, new_time, new_energy, neighbor.name
98                 , path + [neighbor.name]))
99
100     # 3. Map operational health and queue metrics of candidate SENs
101     for node in globals.edge_servers:
102         try:
103             w_cpu_simpy = node.W_cpu()
104         except Exception:

```

```

100     w_cpu_simpv = getattr(node, "W_cpu", lambda: 0.0)()
101
102     try:
103         w_net_simpv = node.W_net()
104     except Exception:
105         w_net_simpv = getattr(node, "W_net", lambda: 0.0)()
106
107     # -----
108     # Compute orbital sunset time
109     raw_sunset = getattr(node, "orbitalSunset", None)
110     if raw_sunset is not None:
111         safe_sunset = float(raw_sunset)
112     else:
113         safe_sunset = 0.0
114     # -----
115
116     # Construct node state snapshot
117     network_state["node_states"][node.name] = {
118         "energy_residual": node.energy,
119         "W_cpu_simpv": w_cpu_simpv,
120         "W_net_simpv": w_net_simpv,
121         "C_sen": getattr(node, "C_sen", 1e9),
122         "e_coeff": self.config.get("energy_coefficient", 5e-26),
123         "sunset_time": safe_sunset
124     }
125
126     return network_state

```

Listing 3.2. Constellation Topology and SEN State Mapping via Multi-Source Dijkstra.

3.4 Centralized Integer Linear Programming Formulation

Let R be the set of tasks extracted in the active batch and S the set of candidate Satellite Edge Nodes. Following classical combinatorial and linear programming principles [11, 12], the optimization problem defines two families of binary decision variables:

$$x_{r,i} \in \{0,1\}, \quad \forall r \in R, \forall i \in S \quad (1 \text{ if task } r \text{ is allocated to satellite } i; 0 \text{ otherwise}) \quad (3.6)$$

$$y_r \in \{0,1\}, \quad \forall r \in R \quad (1 \text{ if task } r \text{ is explicitly rejected; 0 otherwise}) \quad (3.7)$$

3.4.1 Latency, Energy, and Orbital Eclipse Modeling

For each candidate allocation (r, i) , the end-to-end processing delay $R_{r,i}$ evaluated by the solver accounts for multi-hop uplink ISL transmission across path $\vec{P}_{up}(AP_r, i)$ and local computing delay [6]:

$$R_{r,i} = \underbrace{\sum_{(u,v) \in \vec{P}_{up}} \left(W_{net,u} + \frac{S_{req}}{B_{u,v}} + \tau_{u,v} \right)}_{T_{up}(r,i)} + (W_{cpu,i} + d_{cpu,r}) \quad (3.8)$$

The cumulative fleet energy expenditure $\epsilon_{r,i}$ evaluates the network forwarding energy across all intermediate ISL nodes along $\vec{P}_{up}$ alongside local CPU execution [6]:

$$\epsilon_{r,i} = \sum_{(u,v) \in \vec{P}_{up}} P_{net} \frac{S_{req}}{B_{u,v}} + \kappa_i \cdot d_{cpu,r} \cdot C_{sen,i}^3 \quad (3.9)$$

To avoid assigning tasks to satellites nearing orbital eclipse, the orchestrator defines an orbital sunset penalty $\Omega_i \in [0, 1]$:

$$\Omega_i = \begin{cases} 1.0 - \frac{t_{sunset,i}}{\max_{k \in S}(t_{sunset,k})}, & \text{if } \max_{k \in S}(t_{sunset,k}) > 0 \\ 0.0, & \text{otherwise} \end{cases} \quad (3.10)$$

Satellites currently unreachable due to orbital eclipse ($t_{sunset,i} = 0$) receive the maximum penalty $\Omega_i = 1.0$.

3.4.2 Multi-Objective Function and System Constraints

Let $P_{drop} = 10^6$ represent a massive penalty scalar ensuring that task rejection is strictly penalized unless physical constraints are violated. The aggregated fleet latency (Z_{time}), energy (Z_{energy}), and eclipse penalty (Z_{sunset}) are defined as:

$$Z_{time} = \sum_{r \in R} \sum_{i \in S} x_{r,i} R_{r,i} + \sum_{r \in R} y_r P_{drop} \quad (3.11)$$

$$Z_{energy} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \epsilon_{r,i} + \sum_{r \in R} y_r P_{drop} \quad (3.12)$$

$$Z_{sunset} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \Omega_i \quad (3.13)$$

Depending on the configuration parameter `centralized_primary_objective` (Time vs. Energy), the objective function Z applies hierarchical weighting parameters (α , β , and γ) [12, 9]:

$$\min Z = \begin{cases} \alpha Z_{time} + \beta Z_{energy} + \gamma Z_{sunset}, & \text{if primary strategy is Time} \\ \alpha Z_{energy} + \beta Z_{time} + \gamma Z_{sunset}, & \text{if primary strategy is Energy} \end{cases} \quad (3.14)$$

where α denotes the weight assigned to the primary performance metric, β weights the secondary objective, and γ scales the orbital sunset penalty factor.

The optimization space is constrained by four strict linear relations:

1. **Decision Uniqueness:** Every task must be assigned to exactly one satellite or dropped:

$$\sum_{i \in S} x_{r,i} + y_r = 1, \quad \forall r \in R \quad (3.15)$$

2. **Strict Service Processing Deadline:** Enforces that uplink forwarding and local execution do not exceed the deadline D_r [6]:

$$x_{r,i} \cdot R_{r,i} \leq D_r, \quad \forall r \in R, \forall i \in S \quad (3.16)$$

3. **Distributed Fleet Energy Budget:** Enforces that the total energy allocated to satellite node k (across all tasks where k is an intermediate ISL forwarder or execution node) remains within its residual battery capacity B_k [6]:

$$\sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot \epsilon_{r,i,k} \leq B_k, \quad \forall k \in S \quad (3.17)$$

4. **Constellation Load Balancing (Anti-Congestion):** Prevents processor queue bottlenecks by bounding the maximum simultaneous task allocations per node [19]:

$$\sum_{r \in R} x_{r,i} \leq \left\lfloor \frac{|R|}{|S|} \right\rfloor + 5, \quad \forall i \in S \quad (3.18)$$

3.4.3 Solver Routine and Cascade Rejection Diagnostics

The model is constructed in Python using PuLP [14] and executed via the HiGHS solver engine [20]. Upon resolution, if a task is rejected ($y_r = 1$), the orchestrator executes a cascade diagnostic evaluation across all reachable SENSs to classify the primary drop reason:

- "No_Route": No viable ISL path exists from AP_r to any satellite.
- "Deadline_Violation": Predicted response time $R_{r,i}$ exceeds D_r .
- "Sunset_Violation": Response time exceeds available solar visibility horizon $t_{sunset,i}$.
- "Energy_Exhaustion": Residual battery B_k is insufficient for multi-hop transmission or computing.
- "Load_Balancing_Rejected": Satellite reached its maximum capacity threshold.
- "Global_ILP_Conflict": Task rejected to preserve global objective optimality under shared fleet constraints.

Listing 3.3 presents the complete `_solve_ilp` implementation.

```

1  def _solve_ilp(self, network_state, tasks):
2      """
3      Open-Source Hierarchical PuLP Optimization Formulation
4      """
5      # Initialize minimization problem
6      m = pulp.LpProblem("SECMotionModel_Centralized_ILP", pulp.LpMinimize)
7
8      # Configuration parameters
9      primary_obj = self.config.get("centralized_primary_objective", "time")
10
11     # Fixed uplink control payload (2 KB)
12     S_REQ_BYTES = 2048
13     p_net = self.config.get("Ptrasm", 1.0)
14
15     R_set = tasks

```

```

16 S_set = list(network_state["node_states"].keys())
17 all_nodes = globals.global_access_point + globals.edge_servers
18 node_dict = {n.name: n for n in all_nodes}
19
20 # Decision variable definitions
21 x = pulp.LpVariable.dicts("x", ((r, i) for r in range(len(R_set)) for i in S_set)
    ↪ , cat='Binary')
22
23 y = pulp.LpVariable.dicts("y", (r for r in range(len(R_set))), cat='Binary')
24
25 # --- CONSTRAINT 1: Decision Uniqueness (Assign or Drop) ---
26 for r_idx in range(len(R_set)):
27     # Exactly one satellite assignment or drop decision per task
28     m += pulp.lpSum(x[r_idx, i] for i in S_set) + y[r_idx] == 1, f"
    ↪ Assign_Or_Drop_{r_idx}"
29
30 time_costs = {}
31 energy_costs = {}
32
33 node_energy_expressions = {k: [] for k in S_set}
34
35 # 2. Construct link metrics and operational constraints
36 for r_idx, task in enumerate(R_set):
37     task_data = task["data"]
38     ap_origin = task["visible_aps"][0].name
39     d_cpu = task_data.get('d_cpu', 0.0)
40     deadline = task_data.get('deadline', 300.0)
41     s_r_bytes = task_data.get('image_size', 0.0) * 1024 * 1024
42
43     for i in S_set:
44         route_info_up = network_state["routing_table"].get(ap_origin, {}).get(i)
45
46         if not route_info_up:
47             m += x[r_idx, i] == 0, f"NoRoute_{r_idx}_{i}"
48             time_costs[r_idx, i] = 0
49             energy_costs[r_idx, i] = 0
50             continue
51
52         path_up = route_info_up["path"]
53         num_hops_up = len(path_up) - 1
54
55         t_up = 0.0
56         e_fwd_total = 0.0
57
58         # --- Uplink Latency and Energy Formulation ---
59         for h in range(num_hops_up):
60             node_curr = node_dict[path_up[h]]
61             node_next = node_dict[path_up[h+1]]
62             lat = node_curr.get_latency(node_next) or 0.0
63
64             bw_raw = node_curr.get_bandwidth(node_next) or 0.0
65             b_isl = (bw_raw / 8.0) * (1024 ** 2) if bw_raw > 0 else 0.0
66
67             if b_isl > 0:
68                 Wn = network_state["node_states"].get(node_curr.name, {}).get("
    ↪ W_net_simpy", 0.0)
69                 d_net_hop = S_REQ_BYTES / b_isl

```

```

70         t_up += Wn + d_net_hop + lat
71
72         if hasattr(node_curr, 'compute_routing_energy'):
73             e_hop_fwd = node_curr.compute_routing_energy(S_REQ_BYTES,
↪ b_isl, p_net)
74         else:
75             e_hop_fwd = p_net * (S_REQ_BYTES / b_isl)
76
77         e_fwd_total += e_hop_fwd
78
79         if node_curr.name in S_set:
80             node_energy_expressions[node_curr.name].append(x[r_idx, i] *
↪ e_hop_fwd)
81
82         # --- Local CPU Execution (Latency and Energy) ---
83         node_obj = node_dict[i]
84         c_sen_i = network_state["node_states"][i]["C_sen"]
85         e_coeff_i = network_state["node_states"][i]["e_coeff"]
86
87         # Latency
88         Wc = network_state["node_states"][i].get("W_cpu_simp", 0.0)
89         t_cpu_local = Wc + d_cpu
90
91         # Energy
92         if hasattr(node_obj, 'compute_execution_energy'):
93             e_cpu_local = node_obj.compute_execution_energy(d_cpu, c_sen_i, e=
↪ e_coeff_i)
94         else:
95             e_cpu_local = d_cpu * e_coeff_i * (c_sen_i ** 3)
96
97         node_energy_expressions[i].append(x[r_idx, i] * e_cpu_local)
98
99         # --- Metric Aggregation and Service Deadline Constraint ---
100         time_costs[r_idx, i] = t_up + t_cpu_local
101         energy_costs[r_idx, i] = e_fwd_total + e_cpu_local
102
103         # Constraint 2: Strict Service Deadline Compliance
104         R_ri_for_constraint = t_up + t_cpu_local
105         m += x[r_idx, i] * R_ri_for_constraint <= deadline, f"Deadline_{r_idx}_{i}
↪ )"
106
107         # Constraint 3: Distributed Fleet Battery Energy Budget
108         for k in S_set:
109             budget_k = max(0.0, network_state["node_states"][k]["energy_residual"])
110             if node_energy_expressions[k]:
111                 m += pulp.lpSum(node_energy_expressions[k]) <= budget_k, f"Energy_Limit_{
↪ k}"
112
113         # --- Constraint 4: Constellation Load Balancing (Anti-Congestion) ---
114         num_satellites = len(S_set)
115
116         # Compute fleet fair-share allocation threshold
117         if num_satellites > 0:
118             fair_share = len(R_set) // num_satellites
119         else:
120             fair_share = len(R_set)
121

```

```

122     MAX_TASKS_PER_NODE = fair_share + 5
123
124     for i in S_set:
125         m += pulp.lpSum(x[r_idx, i] for r_idx in range(len(R_set))) <=
126         ↪ MAX_TASKS_PER_NODE, f"Max_Capacity_{i}"
127         # -----
128         # --- Orbital Eclipse (Sunset) Penalty Preparation ---
129         # Determine horizon upper bound to normalize penalty within [0, 1]
130         valid_sunsets = [network_state["node_states"][k].get("sunset_time", 0.0) for k in
131         ↪ S_set]
132         max_sunset = max(valid_sunsets + [1.0])
133
134         sunset_penalties = {}
135         for i in S_set:
136             s_time = network_state["node_states"][i].get("sunset_time", 0.0)
137
138             # Assign maximum penalty (1.0) if satellite is currently in eclipse (s_time =
139             ↪ 0.0)
140             sunset_penalties[i] = 1.0 - (s_time / max_sunset) if max_sunset > 0 else 0.0
141             # -----
142
143         # 3. Hierarchical Multi-Objective Optimization
144         PENALTY_VALUE = 1000000.0 # Heavy penalty scalar penalizing task rejection
145
146         obj_time = pulp.lpSum(x[r_idx, i] * time_costs[r_idx, i] for r_idx in range(len(
147         ↪ R_set)) for i in S_set) + \
148             pulp.lpSum(y[r_idx] * PENALTY_VALUE for r_idx in range(len(R_set)))
149
150         obj_energy = pulp.lpSum(x[r_idx, i] * energy_costs[r_idx, i] for r_idx in range(
151         ↪ len(R_set)) for i in S_set) + \
152             pulp.lpSum(y[r_idx] * PENALTY_VALUE for r_idx in range(len(R_set)))
153
154         # --- Orbital Sunset Penalty Objective Term ---
155         obj_sunset = pulp.lpSum(x[r_idx, i] * sunset_penalties[i] for r_idx in range(len(
156         ↪ R_set)) for i in S_set)
157
158         WEIGHT_PRIMARY = self.config.get("centralized_primary_weight", 1.0)
159         WEIGHT_SECONDARY = self.config.get("centralized_secondary_weight", 1.0)
160         WEIGHT_SUNSET = self.config.get("centralized_sunset_weight", 1.0)
161
162         if primary_obj == "time":
163             m += WEIGHT_PRIMARY * obj_time + WEIGHT_SECONDARY * obj_energy +
164             ↪ WEIGHT_SUNSET * obj_sunset, "Total_Objective"
165         else:
166             m += WEIGHT_PRIMARY * obj_energy + WEIGHT_SECONDARY * obj_time +
167             ↪ WEIGHT_SUNSET * obj_sunset, "Total_Objective"
168
169         # 4. Model Resolution via HiGHS Solver (Integrated in PuLP)
170         solver = pulp.getSolver('HiGHS', msg=False)
171         m.solve(solver)
172
173         status = pulp.LpStatus[m.status]
174
175         # -----
176         # Track aggregate energy consumption and task slot allocations for accepted tasks
177         assigned_energy_per_node = {i: 0.0 for i in S_set}

```

```

171     assigned_tasks_per_node = {i: 0 for i in S_set}
172
173     if status in ('Optimal', 'Suboptimal'):
174         for r_idx in range(len(R_set)):
175             if pulp.value(y[r_idx]) is not None and pulp.value(y[r_idx]) < 0.5: #
⇨ Task accepted
176                 for i in S_set:
177                     if pulp.value(x[r_idx, i]) is not None and pulp.value(x[r_idx, i
⇨ ]) > 0.5:
178                         assigned_energy_per_node[i] += energy_costs.get((r_idx, i),
⇨ 0.0)
179                         assigned_tasks_per_node[i] += 1
180                     # -----
181
182     # Decision Extraction and Cascade Rejection Diagnostics
183     assignments = []
184     if status in ('Optimal', 'Suboptimal'):
185         for r_idx, task in enumerate(R_set):
186
187             # --- REJECTED TASK ---
188             if pulp.value(y[r_idx]) and pulp.value(y[r_idx]) > 0.5:
189                 ap_origin = task["visible_aps"][0].name
190                 deadline = task["data"].get('deadline', 300.0)
191                 specific_reason = "Unknown"
192
193                 reachable_sens = [i for i in S_set if network_state["routing_table"].
⇨ get(ap_origin, {}).get(i)]
194
195                 if not reachable_sens:
196                     specific_reason = "No_Route"
197                 else:
198                     # Aggregate rejection causes across all reachable candidate
⇨ satellites
199                     reasons_for_rejection = []
200
201                     for i in reachable_sens:
202                         R_ri = time_costs.get((r_idx, i), 0)
203                         E_ri = energy_costs.get((r_idx, i), 0)
204                         s_time_i = network_state["node_states"][i].get("sunset_time",
⇨ 0)
205
206                         # Compute remaining battery energy after committed
⇨ allocations
207                         actual_energy_left = network_state["node_states"][i]["
⇨ energy_residual"] - assigned_energy_per_node.get(i, 0)
208
209                         # Cascade diagnostic evaluation for candidate satellite i
210                         if R_ri == 0 or R_ri > deadline:
211                             reasons_for_rejection.append("Deadline_Violation")
212                         elif R_ri > s_time_i:
213                             reasons_for_rejection.append("Sunset_Violation")
214                         elif actual_energy_left < E_ri:
215                             reasons_for_rejection.append("Energy_Exhaustion")
216                         elif assigned_tasks_per_node.get(i, 0) >= MAX_TASKS_PER_NODE:
217                             reasons_for_rejection.append("Load_Balancing_Rejected")
218                         else:
219                             reasons_for_rejection.append("Global_ILP_Conflict")

```

```

220         if reasons_for_rejection:
221             specific_reason = max(set(reasons_for_rejection), key=
222 ↪ reasons_for_rejection.count)
223         else:
224             specific_reason = "Unknown"
225
226
227         print(f"[{self.env.now:.3f}] Orchestrator: Task {task['task_id']}
228 ↪ rejected by ILP. Cause: {specific_reason}")
229
230         assignments.append({
231             "task_id": task["task_id"],
232             "data": task["data"],
233             "assigned_sen": None,
234             "routing_path_up": [],
235             "routing_path_down": [],
236             "status": f"Scartato_{specific_reason}"
237         })
238         continue
239
240     # --- ACCEPTED TASK ---
241     assigned_node = None
242     for i in S_set:
243         if pulp.value(x[r_idx, i]) and pulp.value(x[r_idx, i]) > 0.5:
244             assigned_node = i
245             break
246
247     if assigned_node:
248         ap_origin = task["visible_aps"][0].name
249         path_up = network_state["routing_table"][ap_origin][assigned_node][
250 ↪ path"]
251         path_down = [assigned_node, ap_origin]
252
253         assignments.append({
254             "task_id": task["task_id"],
255             "data": task["data"],
256             "assigned_sen": assigned_node,
257             "routing_path_up": path_up,
258             "routing_path_down": path_down,
259             "status": "Assigned"
260         })
261     else:
262         print(f"[{self.env.now:.2f}] [ILP Centralized] Batch Infeasible (Optimization
263 ↪ problem unsolvable). Status: {status}")
264         for task in R_set:
265             assignments.append({
266                 "task_id": task["task_id"], "data": task["data"],
267                 "assigned_sen": None, "routing_path_up": [], "routing_path_down": [],
268                 "status": "Infeasible"
269             })
270
271     return assignments

```

Listing 3.3. PuLP ILP Formulation, HiGHS Resolution, and Cascade Rejection Diagnostics.

Once the ILP optimizer produces the assignment decision matrix, the orchestrator invokes `_dispatch_tasks` to spawn independent `_route_and_execute_task` processes for each accepted request. Due to high orbital velocities in LEO constellations, planned ISL links may experience unexpected line-of-sight obstruction or degradation mid-flight. To guarantee fault tolerance, packet routing employs a dynamic hybrid routing protocol:

- Upon packet arrival at the destination Satellite Edge Node, the transmission duration and hop count are logged in `globals.gbl_generated_tasks_data`, and the process yields to the discrete-event CPU simulator (`ILP_simulation.TaskAssignment_ILP`). Finally, task status and timing metrics are synchronized in the global dataset.

```
def _dispatch_tasks(self, assignments):

    all_nodes = {n.name: n for n in (globals.global_access_point + globals.
    ↪ edge_servers)}

    for assignment in assignments:
        task_id = assignment["task_id"]
        data = assignment["data"]
        status = assignment["status"]
        dispatch_time = self.env.now

        if status != "Assigned":
            if "No_Route" in status or "Infeasible" in status:
                arrival = data.get("arrival_time", dispatch_time)
                print(f"[{dispatch_time:.3f}] Orchestrator: ILP failed ({status}) for
    ↪ Task {task_id}. Re-enqueuing into task buffer!")

                self.task_buffer.append({
                    "task_id": task_id,
                    "data": data,
                    "visible_aps": globals.global_access_point,
                    "arrival_time": arrival
                })

            if len(self.task_buffer) >= self.batch_size and not self.
    ↪ batch_ready_event.triggered:
                self.batch_ready_event.succeed()
```

```

26         continue
27
28         # --- Final Rejection (e.g., Energy Exhaustion determined by ILP) ---
29         ap = globals.global_access_point[0]
30         print(f"[{dispatch_time:.3f}] Orchestrator: Task {task_id} DROPPED ({
↪ status}).")
31         ap.record_rejected_task(
32             task_id, data["type"], dispatch_time, data["image_size"], f"ILP_{
↪ status}", data.get("d_cpu", 0)
33         )
34         continue
35
36         # --- Accepted Task: Spawn routing and execution process ---
37         self.env.process(self._route_and_execute_task(assignment, all_nodes,
↪ dispatch_time))
38
39     def _route_and_execute_task(self, assignment, all_nodes, dispatch_time):
40         task_id = assignment["task_id"]
41         data = assignment["data"]
42         sen_name = assignment["assigned_sen"]
43         sen_obj = all_nodes.get(sen_name)
44         path_up = assignment.get("routing_path_up", [])
45
46         ap_origin = path_up[0] if path_up else globals.global_access_point[0].name
47
48         # --- 1. TRANSIENT ROUTING TASK INITIALIZATION ---
49         routing_task = Task(
50             task_id=task_id,
51             current_node=ap_origin,
52             dest_node=sen_name,
53             routingInitTime=dispatch_time,
54             task_type=data["type"],
55             image_size=data["image_size"]
56         )
57         routing_task.weight = data["image_size"]
58         routing_task.visited.add(ap_origin)
59
60         start_node = all_nodes.get(ap_origin)
61         if start_node and routing_task not in start_node.tasks:
62             start_node.tasks.append(routing_task)
63
64         # --- 2. DYNAMIC NAVIGATION LOOP (Hybrid Routing: ILP -> GREEDY) ---
65         current_node_name = ap_origin
66         target_dest_name = sen_name
67         path_index = 0
68         is_greedy = False
69
70         while current_node_name != target_dest_name:
71             current_node_obj = all_nodes.get(current_node_name)
72
73             if not is_greedy:
74                 # =====
75                 # PHASE A: NOMINAL PLANNED ROUTING (ILP)
76                 # =====
77                 next_planned = path_up[path_index + 1] if path_index < len(path_up) -
↪ 1 else None
78                 next_planned_obj = all_nodes.get(next_planned) if next_planned else

```

```

79  ↪ None
80
81      if next_planned_obj and current_node_obj.get_bandwidth(
82  ↪ next_planned_obj) > 0:
83          # Active link: traverse planned ISL trajectory
84          yield self.env.process(
85              utils.sendTask(self.env, routing_task, current_node_obj,
86  ↪ next_planned_obj, "ILP_Centralized")
87          )
88          path_index += 1
89          routing_task.visited.add(next_planned)
90      else:
91          # BROKEN LINK: Trigger greedy recovery fallback!
92          print(f"[{self.env.now:.3f}] Orchestrator: Broken link at node {
93  ↪ current_node_name} (Task {task_id}). Activating GREEDY Fallback!")
94          is_greedy = True
95
96      if is_greedy:
97          # =====
98          # PHASE B: EMERGENCY RECOVERY NAVIGATION (GREEDY)
99          # =====
100      target_dest_obj = all_nodes.get(target_dest_name)
101      dest_pos = target_dest_obj.getPositionVector(globals.ist_in_conf)
102      ranker_neighbors = []
103      neighbors_list = getattr(current_node_obj, 'neighbors',
104  ↪ current_node_obj.get_neighbors())
105
106      for server in neighbors_list:
107          if current_node_obj.get_bandwidth(server) > 0:
108              neighbor_distance = get_pos_proximity(dest_pos, server.
109  ↪ getPositionVector(globals.ist_in_conf))
110              is_ap = getattr(server, 'is_acc_point', False)
111              ranker_neighbors.append((server, neighbor_distance, is_ap))
112
113      ranker_neighbors.sort(key=lambda x: (not x[2], x[1]))
114
115      best_server = None
116      for neighbor_tuple in ranker_neighbors:
117          # Loop prevention: avoid visited satellites
118          if neighbor_tuple[0].name not in routing_task.visited:
119              best_server = neighbor_tuple[0]
120              break
121
122      if best_server:
123          yield self.env.process(
124              utils.sendTask(self.env, routing_task, current_node_obj,
125  ↪ best_server, 'GREEDY_FALLBACK')
126          )
127          routing_task.visited.add(best_server.name)
128      else:
129          print(f"[{self.env.now:.3f}] Orchestrator: GREEDY Fallback failed
130  ↪ (dead-end) at node {current_node_name}. Task {task_id} dropped.")
131          routing_task.routingEndTime = self.env.now
132          routing_task.label = "DROPPED_GREEDY_DEADEND"
133          globals.gbl_tasks.append(routing_task)
134          return

```

```

# =====
# PHASE C: SAFETY CONTROLS
# =====
if getattr(routing_task, 'label', '') == 'TTL_EXPIRED':
    print(f"[{self.env.now:.3f}] Orchestrator: Task {task_id} dropped due
    to expired TTL during routing.")
    globals.gbl_tasks.append(routing_task)
    return

current_node_name = routing_task.current_node

# --- 3. DESTINATION REACHED ---
transfer_time_up = self.env.now - data.get("arrival_time", dispatch_time)
final_hops = routing_task.hop
init_time = routing_task.routingInitTime
end_time = self.env.now
algos = routing_task.algorithms_used

print(f"[{self.env.now:.3f}] Orchestrator: Task {task_id} arrived at {
sen_name} in {transfer_time_up:.3f}s (Hops: {final_hops})")

# --- 4. DATASET METRIC LOGGING ---
globals.initial_server_counter[sen_name] = globals.initial_server_counter.get
(sen_name, 0) + 1
globals.gbl_task_hops[str(task_id)] = final_hops
globals.gbl_task_final_hops[str(task_id)] = final_hops
globals.gbl_generated_tasks_data.append({
    "task_id": task_id, "arrival_time": data.get("arrival_time",
    dispatch_time), "type": data["type"],
    "ram": data.get("required_ram", 0), "disk": data.get("required_disk", 0),
    "image_size": data["image_size"], "exec_time": data["d_cpu"],
    "transfer_time": transfer_time_up, "performed_transfers": final_hops,
    "final_hops": final_hops, "execution_server": sen_name
})

# --- 5. SIMULATE CPU ON-BOARD COMPUTATION ---
yield self.env.process(
    ILP_simulation.TaskAssignment_ILP(
        env=self.env, selected_server=sen_obj, task_id=task_id, image_size=
data["image_size"],
        arrival_time_system=data.get("arrival_time", dispatch_time), num_hops
=final_hops,
        transfer_time=transfer_time_up, task_type=data["type"], d_cpu=data["
d_cpu"], D_r=data["deadline"],
        net_bw_override=Bps=None, result_sink={}, allow_retry=False,
        routing_already_charged=True, routing_energy_from=None,
        transfer_already_simulated=True
    )
)

# --- 6. POST-EXECUTION METRIC SYNCHRONIZATION ---
final_task = next((t for t in globals.gbl_tasks if t.id == task_id), None)
if final_task:
    final_task.hop = final_hops
    final_task.routingInitTime = init_time
    final_task.routingEndTime = end_time
    final_task.algorithms_used = algos

```

```

177         else:
178             routing_task.routingEndTime = end_time
179             routing_task.label = "REJECTED_POST_ROUTING"
180             globals().gbl_tasks.append(routing_task)

```

Listing 3.4. Task Execution and Dynamic Hybrid Routing Procedure.

3.6 Computational Complexity and Scalability Analysis

Deploying mathematical optimization within highly dynamic Low Earth Orbit (LEO) constellations introduces fundamental operational trade-offs between theoretical optimality and execution overhead [6]. To characterize the scalability and runtime footprint of the proposed centralized framework, this section formalizes the algorithmic time and space complexity across the three pipeline stages: dynamic topology mapping, batch linear optimization, and hybrid packet forwarding.

3.6.1 Topology Mapping and Pathfinding Complexity

Prior to solver instantiation, the centralized orchestrator maps the constellation routing fabric using a Multi-Source Dijkstra shortest-path algorithm evaluated across the directed connectivity graph $G = (V, E)$ [22]. The node set $V = AP_{global} \cup S$ encompasses all regional ground access points and candidate Satellite Edge Nodes, while E comprises all active ground-to-satellite up/downlinks and optical Inter-Satellite Links (ISLs) [6].

Using an adjacency list graph representation paired with a binary min-heap priority queue (via Python’s native `heapq` module), the asymptotic time complexity across all active access points $|AP|$ is:

$$\mathcal{T}_{map} = \mathcal{O}(|AP| \cdot (|E| + |V| \log |V|)) \quad (3.19)$$

In standard regional Walker-Delta constellations, satellite transceivers are architecturally limited to at most four optical heads (two intra-plane links to ahead/behind peers and two inter-plane links to adjacent orbital planes) [16, 7]. Consequently, the maximum node degree is strictly bounded ($\deg(v) \leq 4$), yielding a sparse graph where the total edge cardinality scales linearly with the constellation size:

$$|E| \leq 4|S| + 2|AP| \implies |E| = \mathcal{O}(|S|) \quad (3.20)$$

Substituting this structural property into Equation (3.19) yields an effective time complexity of:

$$\mathcal{T}_{map} = \mathcal{O}(|AP| \cdot |S| \log |S|) \quad (3.21)$$

Regarding memory footprint, the orchestrator stores the adjacency graph, candidate node telemetry (queue lengths $W_{cpu,i}$, battery reserves B_i , eclipse horizons $t_{sunset,i}$), and all pre-computed paths. The auxiliary space complexity is $\mathcal{O}(|AP| \cdot |S| + |E|)$, which remains under a few megabytes for regional fleets ($|S| \leq 50$), easily fitting within the cache memory of space-qualified embedded processing boards.

3.6.2 ILP Formulation and Branch-and-Cut Solver Complexity

For an active batch containing $|R|$ buffered tasks and $|S|$ candidate Satellite Edge Nodes:

- **Decision Variable Space:** The mathematical model allocates $|R| \cdot |S|$ binary assignment variables $x_{r,i} \in \{0, 1\}$ alongside $|R|$ binary drop variables $y_r \in \{0, 1\}$. The total decision dimension is:

$$\mathcal{D}_{vars} = |R| \cdot |S| + |R| = \mathcal{O}(|R| \cdot |S|) \quad (3.22)$$

- **Constraint Matrix Dimensions:** The constraint system enforces $|R|$ uniqueness constraints, $|R| \cdot |S|$ deadline compliance constraints, $|S|$ fleet battery budget bounds, and $|S|$ anti-congestion load balancing bounds. The total constraint row complexity is exactly:

$$\mathcal{C}_{rows} = |R| \cdot (|S| + 1) + 2|S| = \mathcal{O}(|R| \cdot |S| + |R| + 2|S|) \quad (3.23)$$

From a theoretical standpoint, 0-1 Integer Linear Programs are strongly NP-hard, exhibiting a worst-case computational complexity of $\mathcal{O}(2^{|R| \cdot |S|})$ under exhaustive combinatorial enumeration [11]. However, the modern branch-and-cut routine in the HiGHS solver mitigates this combinatorial explosion through presolve reductions, cutting plane generation, and polynomial-time linear relaxations ($\mathcal{O}(n^{2.5})$) solved via the dual revised simplex algorithm at each search tree node [20, 11].

Most importantly, the *Dynamic Batching Engine* acts as an architectural boundary on problem dimension. By strictly capping the batch size ($|R| \leq BS$ with $BS \leq 20$), the optimization problem space does not scale with the external request arrival rate λ . For a regional constellation slice ($|S| \approx 15$), the model instantiates at most 300 binary variables and 330 constraint rows. Because the constraint matrix is highly sparse—each task uniqueness constraint involves only $|S|$ non-zero entries, and each deadline constraint involves exactly one non-zero coefficient—the LP relaxations converge in fewer than 50 simplex pivots, keeping pure solver execution times strictly bounded at $T_{exec} \leq 0.30\text{--}0.60$ ms.

3.6.3 Hybrid Routing and Execution Forwarding Complexity

The complexity of the task routing and dispatching layer depends on the operational health of the Inter-Satellite Links:

- **Nominal Planned Forwarding (Phase A):** Data packets strictly trace the pre-computed Dijkstra path $\vec{P}_{up}(AP_r, i)$. Each intermediate hop requires an $\mathcal{O}(1)$ pointer lookup in the local forwarding table. The total time complexity for routing across h_{up} hops is bounded by the constellation diameter:

$$\mathcal{T}_{planned} = \mathcal{O}(h_{up}) \leq \mathcal{O}(\text{diam}(G)) \ll \mathcal{O}(|S|) \quad (3.24)$$

- **Greedy Recovery Fallback (Phase B):** If a planned ISL is severed mid-flight due to orbital occlusion or attitude drift, the forwarding satellite evaluates adjacent neighbor proximities. Sorting the adjacent neighbor candidates

requires $\mathcal{O}(k \log k)$ time, where k is the local physical node degree. Because physical transceivers strictly bound node connectivity to $k \leq 4$, the sorting step collapses to a negligible, constant-time operation:

$$\mathcal{T}_{greedy} = \mathcal{O}(k \log k) \approx \mathcal{O}(1) \quad (\text{since } k \leq 4) \quad (3.25)$$

Duplicate node visits are eliminated through a hash-set lookup structure (`routing_task.visited`), operating in $\mathcal{O}(1)$ average time and preventing forwarding loops.

3.6.4 Constellation Scalability and Telemetry Overhead

A critical factor governing the real-world scalability of centralized orbital architectures is the control signaling required to synchronize constellation state at the Master node [1, 6]. To formulate an accurate optimization model, the orchestrator collects a compact telemetry vector $\mathbf{v}_i = [W_{cpu,i}, W_{net,i}, B_i, t_{sunset,i}]$ from each Satellite Edge Node.

Transmitting these status packets across the ISL mesh introduces a telemetry communication complexity of $\mathcal{O}(|S|)$ messages per update cycle. In regional deployments ($|S| \leq 50$), telemetry frames consume less than 0.1% of the optical link bandwidth (1 Gbps), confirming that control overhead does not saturate transmission channels. However, as constellation dimensions expand globally to thousands of spacecraft ($|S| \gg 1000$), the communication latency and stale state telemetry associated with single-master architectures make regional clustering or hierarchical decomposition indispensable [6].

Chapter 4

Performance Analysis, Optimization, and Experimental Evaluation

This chapter presents the experimental evaluation, systematic parametric tuning, and consolidated validation of the proposed Centralized Integer Linear Programming (ILP) Orchestrator [6]. While Chapter 3 established the formal mathematical optimization model and its discrete-event simulation architecture, deploying exact centralized scheduling in Low Earth Orbit (LEO) constellations introduces fundamental operational trade-offs [4, 1]. In particular, centralized optimization must balance the queuing latency introduced by task accumulation in ingress buffers against the physical dynamics of dynamic inter-satellite topologies and strict application deadlines [6, 9].

To address these challenges, this chapter follows a progressive two-stage tuning methodology concluded by an ablation validation:

1. **Stage 1: Macro-Tuning of Ingress Batching ($BS \times BT$):** We explore an exhaustive design space of 28 batching configurations across increasing traffic arrival rates ($\lambda \in [2, 10]$ req/s) and deadline relaxation thresholds (ΔD) using baseline default weights. This isolates the queuing latency bottleneck and identifies the viable operational batching archetypes.
2. **Stage 2: Micro-Tuning of Objective Weights ($\gamma \times \alpha$):** Focusing on the viable batching archetypes under heavy load ($\lambda = 8$ req/s and $\lambda = 10$ req/s), we perform a cross-sensitivity analysis between the primary objective weight (α) and the orbital sunset penalty (γ) to maximize feasibility while preventing topological path-bloat.
3. **Stage 3: Consolidated Validation:** With the optimal objective weights determined, we validate the selected configurations across the full arrival rate spectrum ($\lambda \in [2, 10]$ req/s), directly comparing their performance against the pre-optimization baseline.

4.1 Simulation Environment and Methodological Setup

The experimental campaign was carried out using an event-driven discrete-event simulation engine implemented in Python with `SimPy` [13], interfaced with the `PuLP` mathematical programming library [14] and solved via the open-source `HiGHS` branch-and-cut solver [20]. Numerical telemetry and array manipulations were processed using `NumPy` [23] and `SciPy` [24], while performance distributions and diagnostic curves were generated using `Matplotlib` [25]. The simulator models orbital kinematics [21], optical Inter-Satellite Links (ISLs) [7, 8], dynamic ground-to-satellite line-of-sight contact windows, and heterogeneous onboard Satellite Edge Nodes (SENs) [6].

4.1.1 Constellation Topology and Scenario Parameters

The orbital network is modeled as a regional Walker-Delta LEO constellation operating at an altitude of 600 km with an inclination of 53° [16]. Satellites are interconnected via full-duplex optical ISLs characterized by nominal transmission bandwidths $B_{u,v}$ and propagation delays $\tau_{u,v}$ computed dynamically from orbital positions [8]. A designated Master in the operational region collects offloaded task requests forwarded by Ground Users (GUs) through regional Ground Access Points (AP_{global}) [6].

Workload demands and request dimensions are parameterized through three profiling vectors, consistent with recent benchmarking literature [6]:

- **Workload Distribution Vector** $\beta = (\beta_G, \beta_C, \beta_{CD})$: Dictates the proportion of Generic tasks, CPU-intensive tasks, and CPU + Data-intensive tasks. In the baseline batching and tuning experiments, the default multi-service profile is set to $\beta = (0.40, 0.35, 0.25)$ [6].
- **Real-Time Payload Vector** $\alpha = (\alpha_M, \alpha_L, \alpha_V)$: Governs the distribution of input/output dataset dimensions across Medium (1–5 MB), Large (5–20 MB), and Very Large (20–50 MB) files, parameterized as $\alpha = (0.30, 0.50, 0.20)$ [6, 2].
- **Batch Task Data Vector** $\gamma_{data} = (\gamma_L, \gamma_V)$: Controls delay-tolerant background transfers, fixed to $\gamma_{data} = (0.70, 0.30)$ [4].

Candidate edge nodes are assigned operational battery reserves of $E_i \in \{40, 80\}$ kJ per orbital pass [6, 17]. The request arrival rate λ varies systematically from 2 req/s (low-load, mean inter-arrival time of 0.50 s) up to 10 req/s (heavy-load stress regime, mean inter-arrival time of 0.10 s) [6].

4.1.2 Two-Tier Optimization Methodology

Directly testing all permutations of batching thresholds, objective weights, and network loads would require thousands of simulations and obscure the physical origins of performance bottlenecks. We therefore decouple the optimization process:

- **Architectural Decoupling**: Ingress batching (BS, BT) governs the static waiting time of tasks in the coordinator queue *before* the ILP model is instantiated. Conversely, objective function weights (α, γ) guide the mathematical branch-and-bound trade-offs *during* solver execution [11, 12].

- **Sequential Tuning:** By first sweeping $BS \times BT$ under default weights, we isolate the feasible batching configurations that do not cause queue-induced deadline collapse. Next, we tune the objective weights (α, γ) under heavy stress on these selected batching archetypes. Finally, we benchmark the optimized configurations against the pre-tuning baseline across the entire workload spectrum.

4.2 Macro-Tuning: Exploration and Selection of Batching Policies

In a centralized orchestrator, incoming requests are buffered until either the Batch Size (BS) or the Batch Timeout (BT) threshold is triggered. This introduces a fundamental trade-off: larger batches enlarge the joint solution space, enabling the solver to achieve higher allocation efficiency, but prolonged buffering erodes task deadline margins [6].

To quantify this trade-off, a parametric sweep of 28 configurations was executed across arrival rates $\lambda \in [2, 10]$ req/s under both Energy Mapping and Time Mapping, evaluated across deadline relaxation levels $\Delta D \in \{10\%, 20\%, 30\%\}$.

4.2.1 Success Rate Dynamics

Figures 4.1 and 4.2 show the Task Success Rate across the parameter space for Energy and Time mapping, respectively.

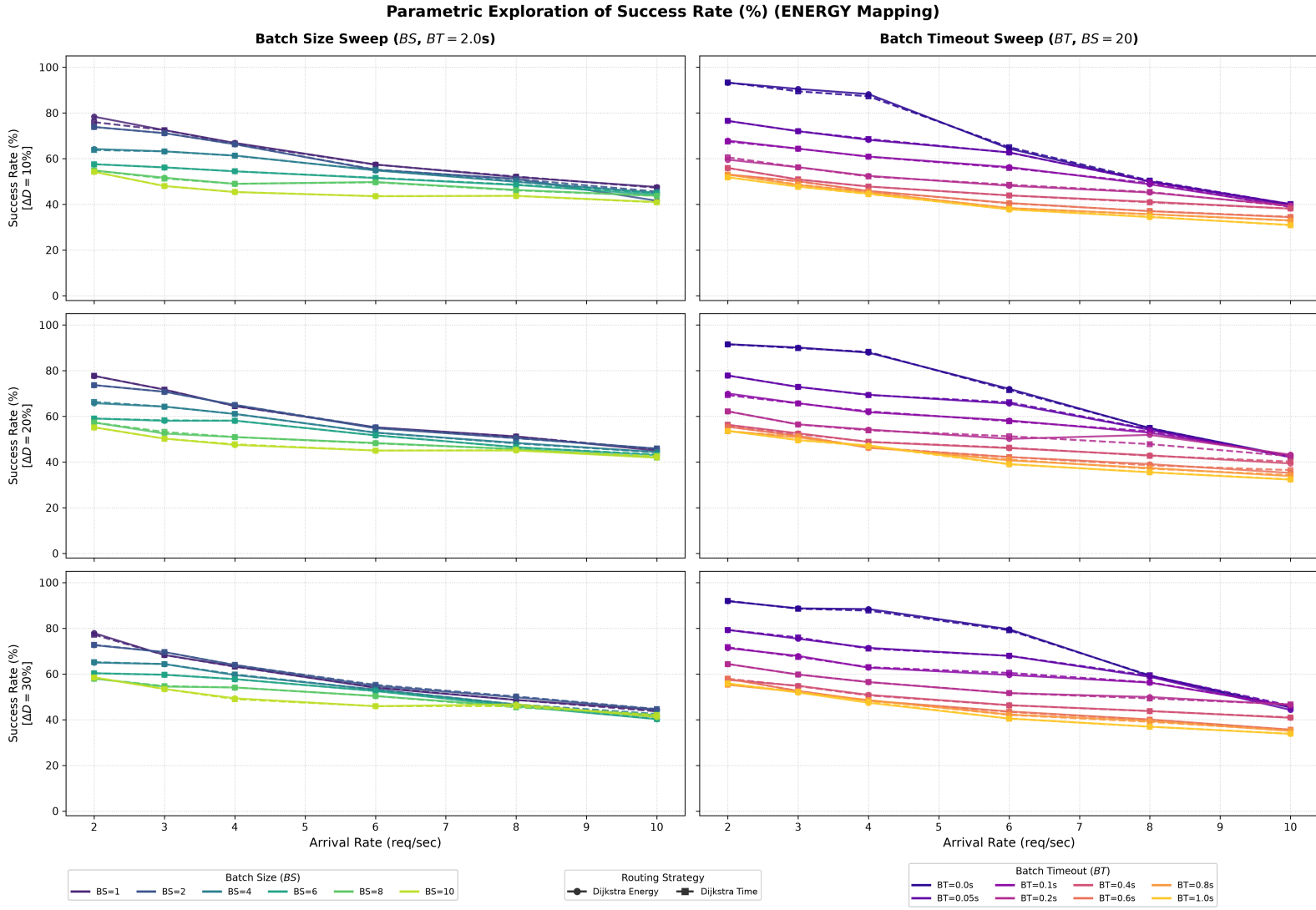


Figure 4.1. Success Rate across arrival rates under Energy Mapping for BS -sweep ($BT = 2.0$ s) and BT -sweep ($BS = 20$) across deadline relaxation levels $\Delta D \in \{10\%, 20\%, 30\%\}$.

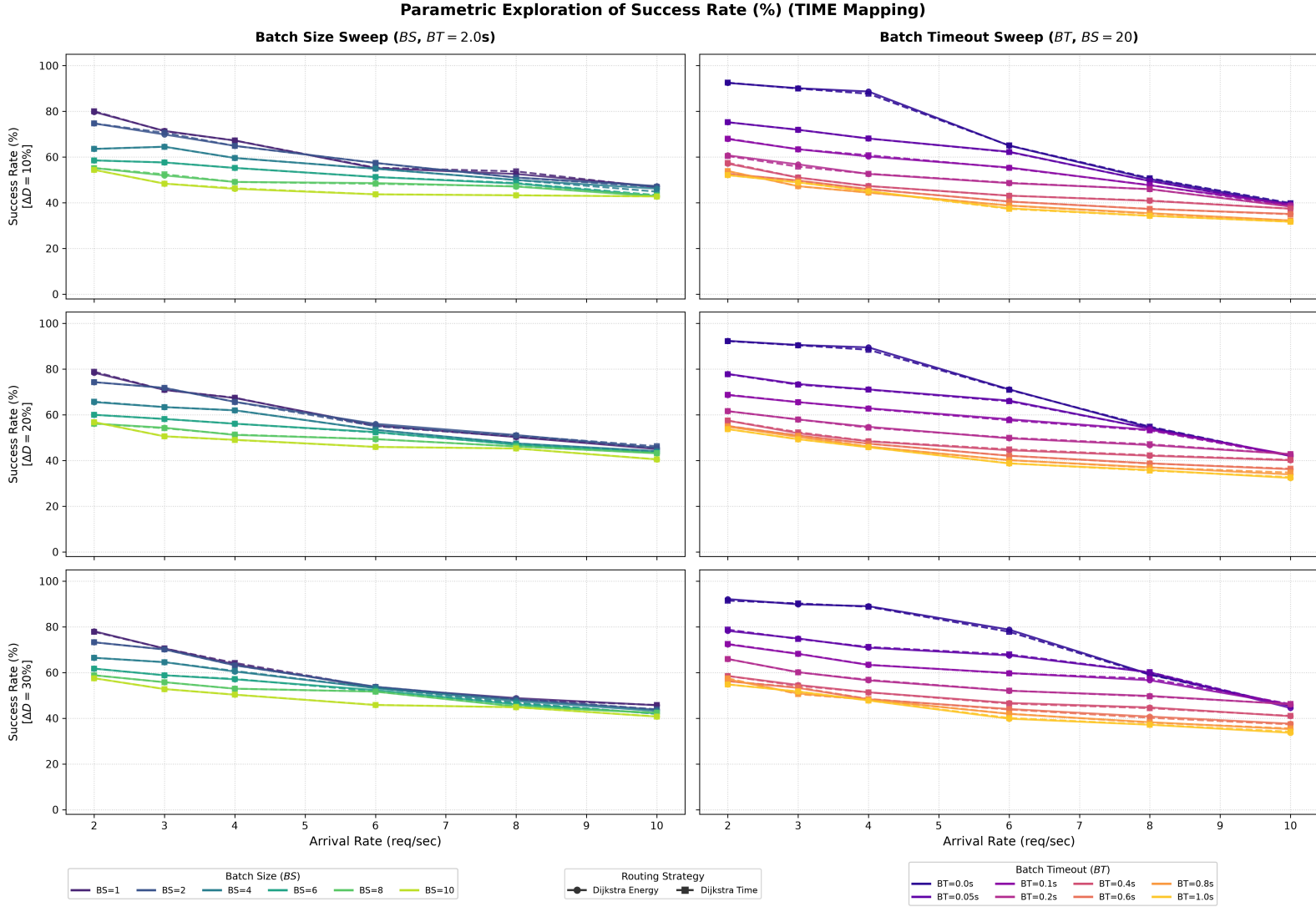


Figure 4.2. Success Rate across arrival rates under Time Mapping for BS -sweep ($BT = 2.0$ s) and BT -sweep ($BS = 20$) across deadline relaxation levels $\Delta D \in \{10\%, 20\%, 30\%\}$.

The empirical trends highlight four critical dynamics across the parameter space:

- **High-Throughput of Pure Instantaneous Dispatch ($BT = 0.0$ s):** The zero-timeout policy ($BS = 20, BT = 0.0$ s, uppermost dark curve) achieves the highest overall completion rate, maintaining $\approx 93\%$ success at $\lambda = 2$ req/s and remaining above 88% up to $\lambda = 4$ req/s. Beyond moderate load, however, it experiences a steep drop, declining to $\approx 40\%$ at $\lambda = 10$ req/s due to physical node and link contention under continuous single-task arrivals.
- **Controlled Small-Batching Dynamics and High-Load Convergence (BS -Sweep):** Configurations enforcing small aggregations ($BS \in \{1, 2\}$ with $BT = 2.0$ s) or reactive timeouts ($BT = 0.05$ s) demonstrate balanced performance, starting at $\approx 76\text{--}79\%$ at $\lambda = 2$ req/s and degrading monotonically to $\approx 42\text{--}47\%$ at peak traffic. Crucially, while increasing BS from 1 to 10 causes

a severe penalty at low arrival rates ($\approx 78\%$ vs. $\approx 54\%$ at $\lambda = 2$ req/s due to prolonged queue buffering), all batch size configurations converge to a narrow band (41%–47%) at $\lambda = 10$ req/s, as rapid request arrivals naturally accelerate batch formation before timeout expiration.

- **Performance Degradation in High-Timeout Regimes ($BT \geq 0.4$ s):** Relying on generous buffer timeouts (orange and yellow curves in the right panels) systematically penalizes requests across all arrival rates. Dwell times in the ingress queue erode critical deadline margins, holding success rates below 58% even under light traffic ($\lambda = 2$ req/s) and collapsing to 31%–36% under stress ($\lambda = 10$ req/s).
- **Invariance to Objective Metric, Routing Strategy, and Deadline Slackness:** The success curves for Time Mapping and Energy Mapping are virtually identical (differing by less than 2.0%), and Dijkstra-Energy (solid lines) matches Dijkstra-Time (dashed lines) almost perfectly across all operational points. Furthermore, expanding deadline relaxation from $\Delta D = 10\%$ to 30% provides negligible throughput improvements ($< 3\%$), verifying that task rejections in delayed configurations exceed deadlines by margins far wider than the relaxation delta.

4.2.2 Diagnostics of Failure Causes

Figures 4.3 and 4.4 illustrate the relative distribution of rejection causes across the parameter sweeps.

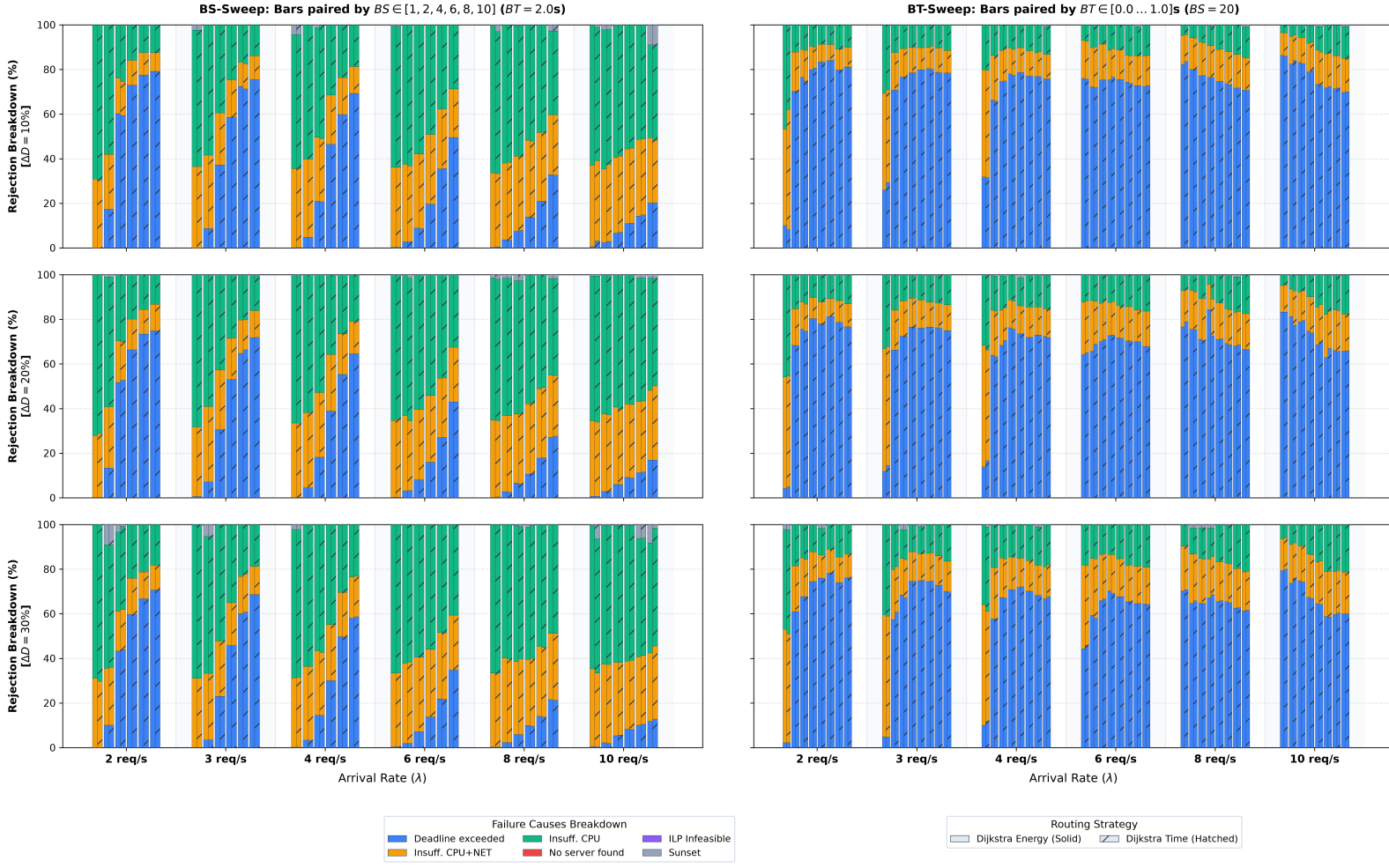


Figure 4.3. Rejection causes distribution across arrival rates under Energy Mapping, comparing BS -sweep ($BT = 2.0$ s) against BT -sweep ($BS = 20$) for $\Delta D \in \{10\%, 20\%, 30\%\}$.

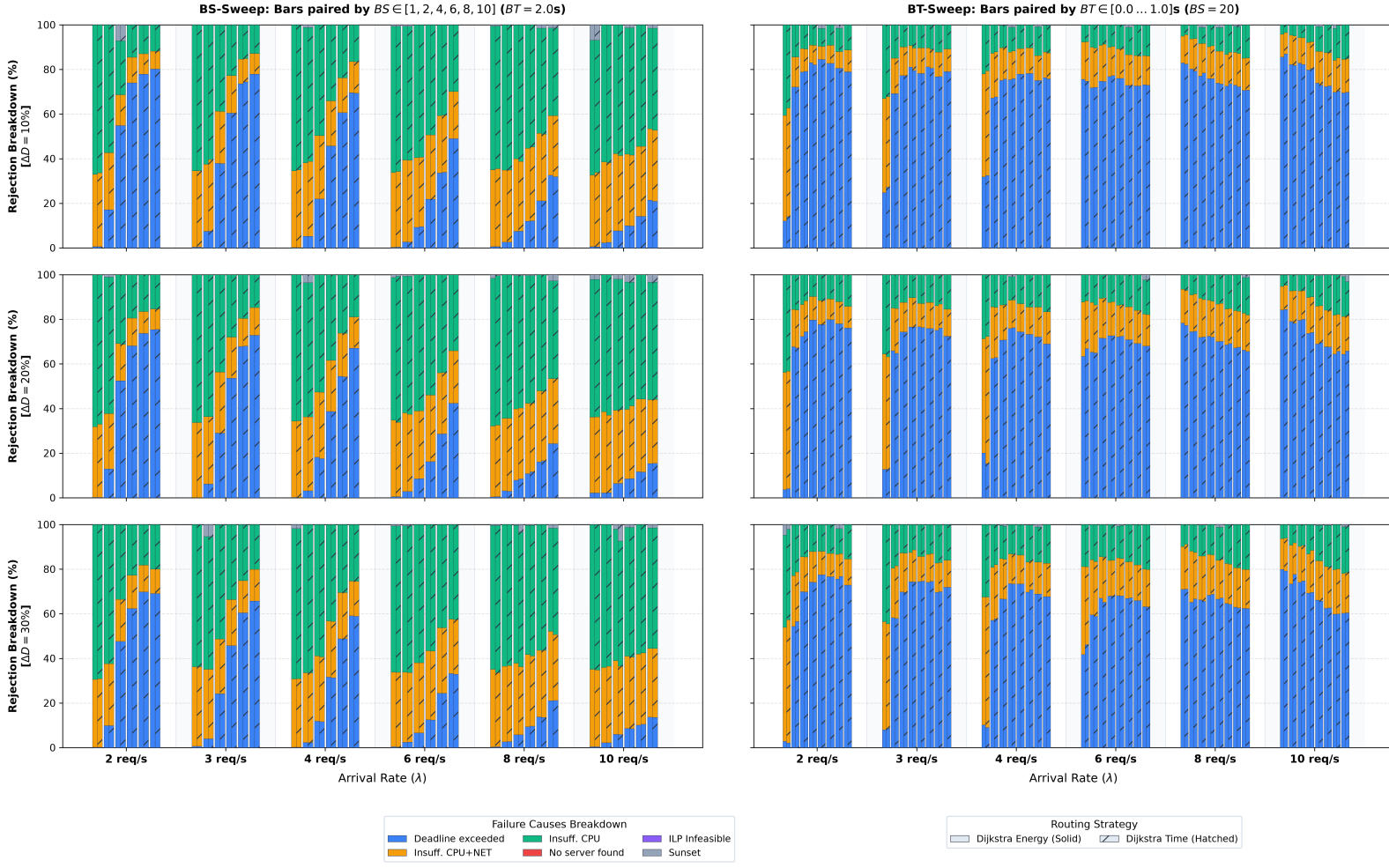


Figure 4.4. Rejection causes distribution across arrival rates under Time Mapping, comparing BS -sweep ($BT = 2.0$ s) against BT -sweep ($BS = 20$) for $\Delta D \in \{10\%, 20\%, 30\%\}$.

The failure diagnostics reveal distinct operational regimes governed by the interaction between batch aggregation thresholds and traffic intensity:

- **Strict Capacity-Bounded Regime ($BS = 1$):** When task aggregation is structurally disabled ($BS = 1$), task rejections due to *Deadline Exceeded* (blue) are completely suppressed (0%) across all arrival rates ($\lambda \in [2, 10]$ req/s). Because requests bypass coordinator queuing entirely, drops occur exclusively from physical constellation saturation: *Insufficient CPU* (green, accounting for 60%–65% of rejections) and *Insufficient CPU+NET* (orange, accounting for 35%–40%). A similar capacity-dominated profile is observed for $BS = 2$ at moderate-to-high traffic ($\lambda \geq 4$ req/s), where deadline drops remain below 5%.
- **Load-Dependent Degradation under Instantaneous Dispatch ($BS = 20, BT = 0.0$ s):** Under light traffic ($\lambda = 2$ req/s), setting $BT = 0.0$ s successfully prevents queue-induced expiration, with *Deadline Exceeded* accounting for only $\approx 8\%$ of drops while physical constraints dominate (CPU: $\approx 50\%$, CPU+NET: $\approx 42\%$). However, as traffic scales toward stress levels

($\lambda \geq 6$ req/s), cumulative onboard execution queues ($W_{cpu,i}$) expand across candidate satellites. Consequently, even without ingress batch dwell time, processing delays push completion past application deadlines, causing *Deadline Exceeded* to escalate dramatically to $\approx 68\%$ at $\lambda = 6$ req/s and reaching $\approx 85\%$ at $\lambda = 10$ req/s.

- **Prevalence of Deadline Failures in Buffered Policies ($BT \geq 0.05$ s):** Introducing even a minimal buffering delay ($BT = 0.05$ s) produces noticeable deadline penalties, with *Deadline Exceeded* failures already at $\approx 32\%$ at $\lambda = 2$ req/s and exceeding 80% under peak load. For configurations with $BT \geq 0.1$ s, buffer residency severely erodes the remaining slack time, making *Deadline Exceeded* the overwhelming failure cause (70%–85%) across the entire traffic spectrum.
- **Bottleneck Inversion in Batch Size Aggregation (BS -Sweep, $BT = 2.0$ s):** In the batch size sweep, failure modes invert as traffic intensifies:
 - At $\lambda = 2$ req/s, assembling large batches ($BS \geq 6$) requires waiting for the prolonged 2.0 s timeout, triggering massive deadline expirations (70%–82% of total drops).
 - At $\lambda = 10$ req/s, rapid inter-arrival times (≈ 100 ms) fill the batch before timeout expiration, drastically cutting buffer residency. As a result, *Deadline Exceeded* drops contract to 10%–20%, and physical bottlenecks re-emerge as the primary constraint (*Insufficient CPU* $\approx 45\%$ –50%, *Insufficient CPU+NET* $\approx 35\%$).

4.2.3 Response Time Decomposition

Figures 4.5 and 4.6 show the end-to-end response time decomposed into Onboard Task Execution Time (T_{exec}) and Network/Wait Time ($T_{net+wait}$).

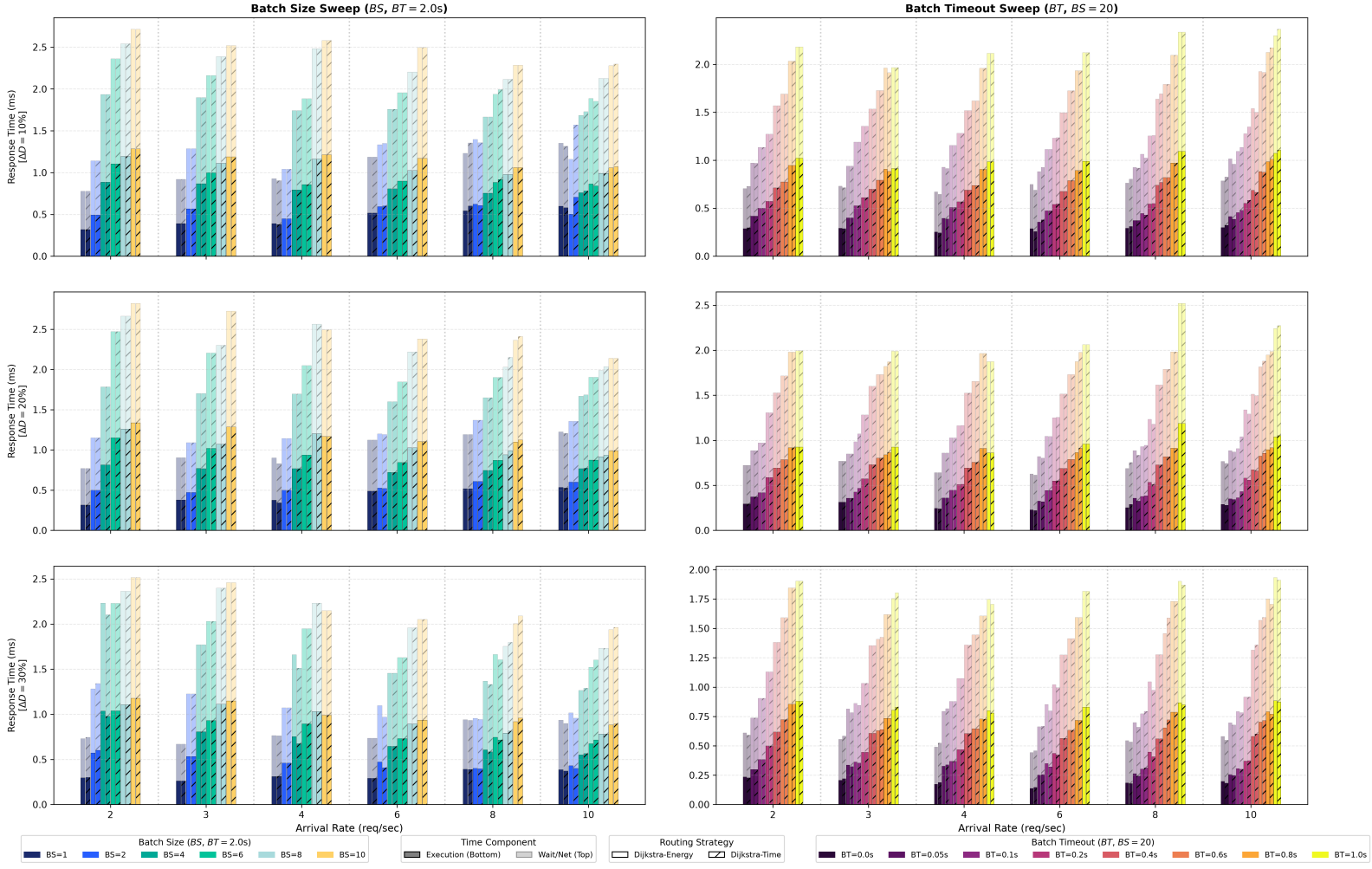


Figure 4.5. Response Time decomposition under Energy Mapping across arrival rates for BS -sweep and BT -sweep configurations under $\Delta D \in \{10\%, 20\%, 30\%\}$.

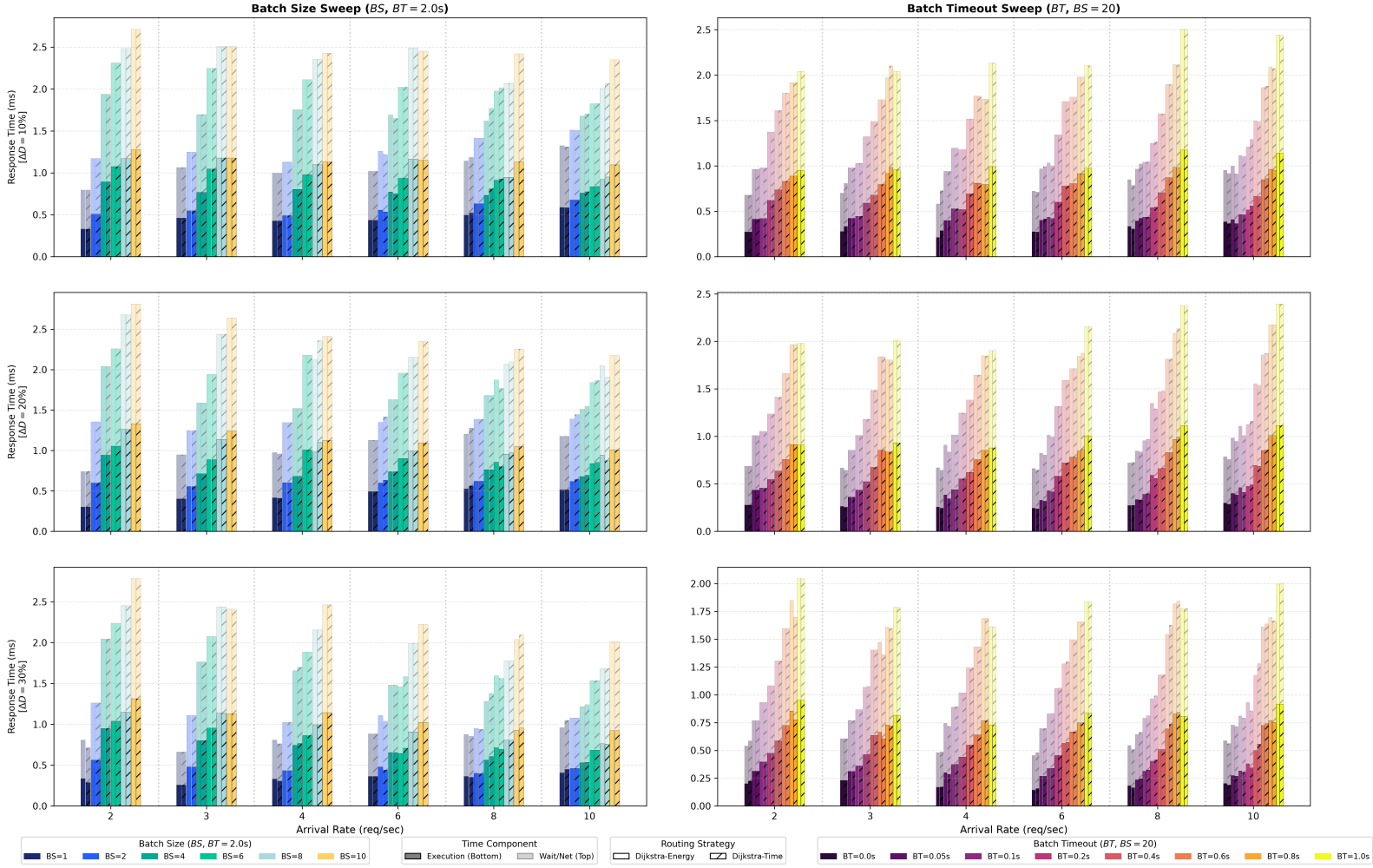


Figure 4.6. Response Time decomposition under Time Mapping across arrival rates for *BS*-sweep and *BT*-sweep configurations under $\Delta D \in \{10\%, 20\%, 30\%\}$.

These stacked decompositions highlight the operational mechanics governing orchestrator latency:

$$T_{total} = T_{exec}(|R|) + T_{net+wait}(\lambda, BS, BT) \quad (4.1)$$

where onboard task execution time on the assigned SEN (T_{exec}) is strictly bounded by local processing and node queue residency across the active batch $|R|$ rather than the external traffic arrival rate:

$$T_{exec} \approx \begin{cases} 0.25\text{--}0.50 \text{ ms}, & \text{for } BS \leq 2 \text{ or } BT \leq 0.05 \text{ s} \\ 0.90\text{--}1.20 \text{ ms}, & \text{for large batches } (BS \geq 8 \text{ or } BT \geq 0.6 \text{ s}) \end{cases} \quad (4.2)$$

For any given batch configuration, the execution footprint remains remarkably stable across arrival rates ($\lambda \in [2, 10]$ req/s). Conversely, the end-to-end response time is heavily dominated by the queuing and transmission component ($T_{net+wait}$, top translucent segments), which dictates the overall latency profile across the parameter sweeps:

- **Batch Size Sweep** ($BT = 2.0$ s): At low arrival rates ($\lambda = 2$ req/s), accumulating large batches ($BS \geq 8$) introduces severe buffer dwell time while waiting for subsequent requests, pushing total response time to its peak (≈ 2.50 – 2.70 ms). As λ scales toward 10 req/s, incoming requests arrive more frequently, accelerating batch completion and compressing the waiting component down to ≈ 1.20 ms ($T_{total} \approx 2.10$ – 2.30 ms).
- **Batch Timeout Sweep** ($BS = 20$): The timeout threshold directly acts as an upper bound on ingress queue residency. For aggressive timeouts ($BT \leq 0.05$ s), buffer dwell time is virtually suppressed, maintaining total latency below 0.75 ms. For $BT \geq 0.6$ s, requests linger in the ingress buffer until timeout expiration, causing $T_{net+wait}$ to expand beyond 1.30 ms across all load levels.

Finally, comparing routing strategies confirms that Dijkstra-Time (hatched bars) and Dijkstra-Energy (solid bars) yield nearly identical computational workloads, with Dijkstra-Time providing marginal path latency containment over multi-hop ISL transmissions.

4.2.4 Selection of Viable Batching Archetypes

The sweep results rule out all batching policies with $BT \geq 0.2$ s or $BS \geq 8$, as queue dwell times systematically violate task deadlines. To represent the viable operational design space, we select two complementary configurations for subsequent weight calibration:

1. **Reactive Fast-Dispatch** ($BS = 20, BT = 0.05$ s): Operates with an aggressive 50 ms timeout. Because inter-arrival times at $\lambda \leq 10$ req/s are ≥ 100 ms, the timeout triggers almost continuously, dispatching 1–2 tasks per cycle and keeping buffer dwell times near zero.
2. **Controlled Small-Batching** ($BS = 2, BT = 2.0$ s): Enforces minimal task aggregation. At arrival rates $\lambda \geq 2$ req/s, the batch size threshold ($BS = 2$) is reached within 0.2–0.5 s, providing the solver with a joint optimization horizon over 2 requests without severe queue buildup.

4.3 Micro-Tuning: Multi-Objective Weight Calibration under Stress ($\gamma \times \alpha$)

Having isolated the viable batching configurations ($BS = 2, BT = 2.0$ s and $BS = 20, BT = 0.05$ s), we calibrate the scalarization weights of the multi-objective function [12]:

$$\min Z = \begin{cases} \alpha Z_{time} + \beta Z_{energy} + \gamma Z_{sunset}, & \text{if primary objective is Time} \\ \alpha Z_{energy} + \beta Z_{time} + \gamma Z_{sunset}, & \text{if primary objective is Energy} \end{cases} \quad (4.3)$$

where $\beta = 1.0$, the primary objective weight is evaluated across $\alpha \in \{10, 50, 1000\}$, and the orbital sunset deterrent penalty is evaluated across $\gamma \in \{2, 4, 6, 8, 10, 1000, 5000, 10000\}$. The cross-sensitivity experiment was conducted under heavy stress loads of $\lambda = 8$ req/s and $\lambda = 10$ req/s, comparing both Dijkstra-Time and Dijkstra-Energy link routing metrics [22, 6].

4.3.1 Impact on Success Rate

Figures 4.7 and 4.8 depict the Task Success Rate across the $(\gamma \times \alpha)$ parameter space for $\lambda = 8$ req/s and $\lambda = 10$ req/s, respectively.

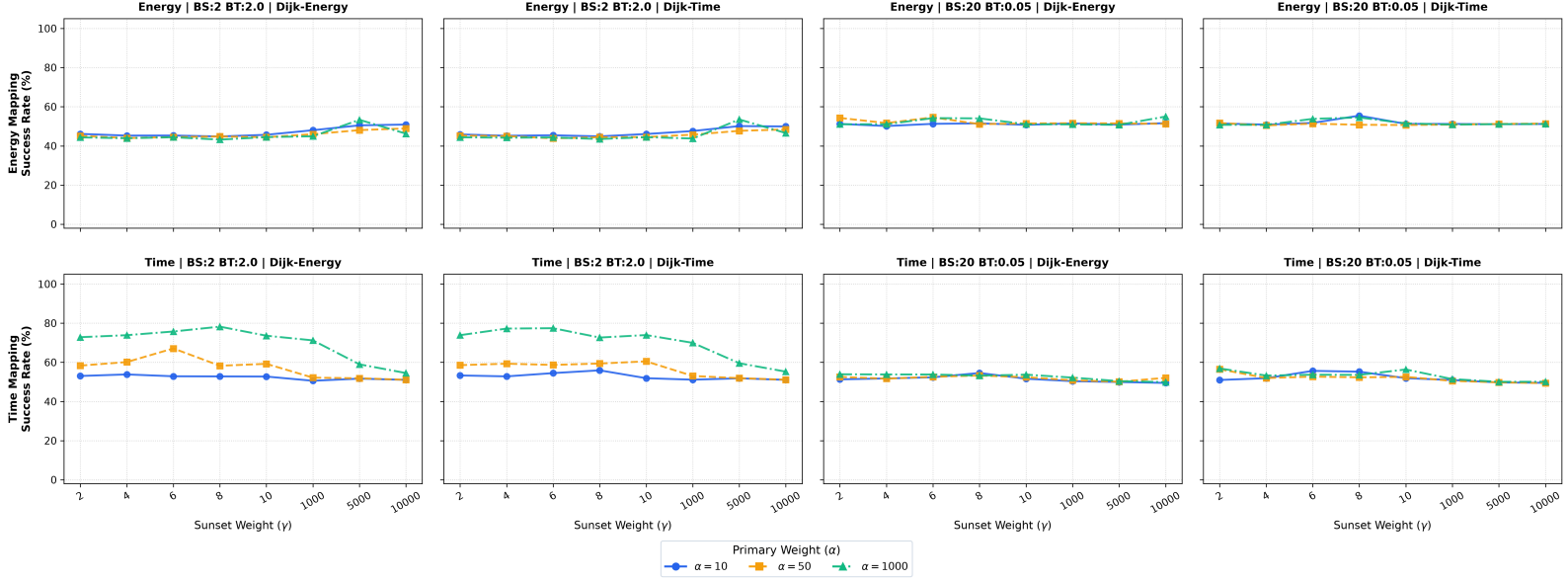


Figure 4.7. Cross-sensitivity Success Rate ($\gamma \times \alpha$) at $\lambda = 8$ req/s across batching configurations, Dijkstra metrics, and mapping objectives.

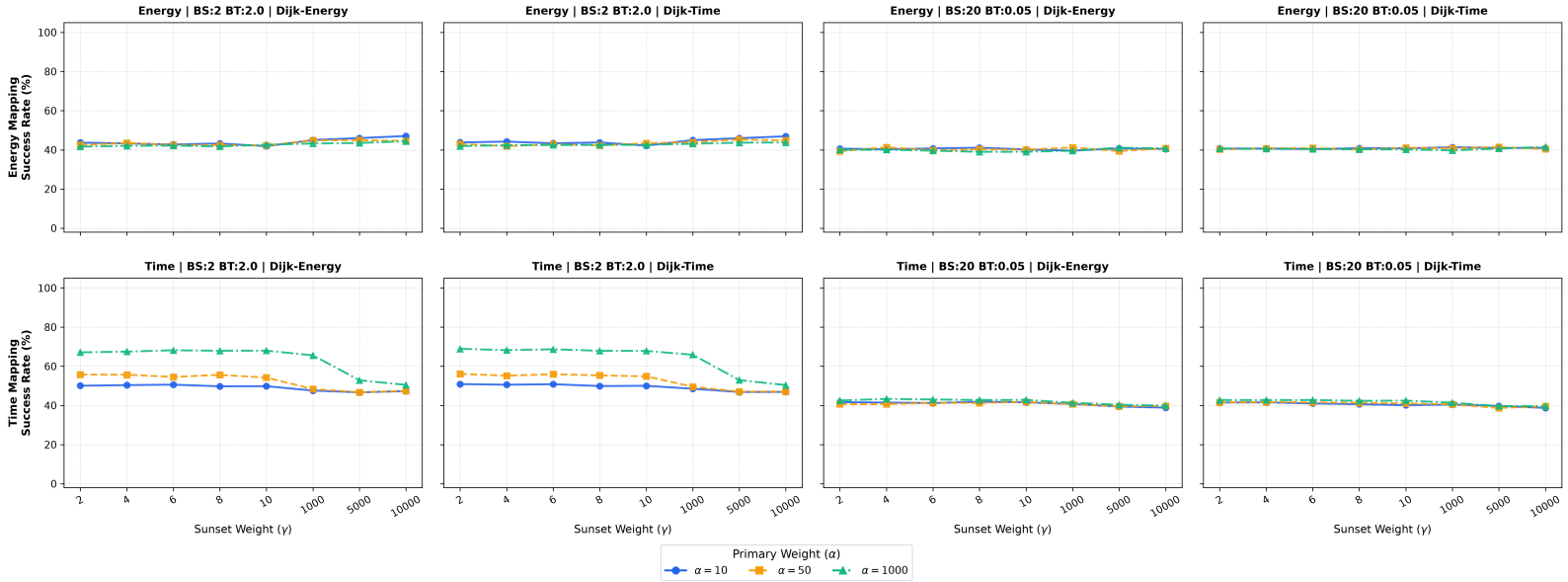


Figure 4.8. Cross-sensitivity Success Rate ($\gamma \times \alpha$) at $\lambda = 10$ req/s across batching configurations, Dijkstra metrics, and mapping objectives.

The empirical trends indicate:

- Decisive Impact of Primary Weight $\alpha = 1000$ in Latency-Centric Batching:** The influence of weight scaling is acutely concentrated in the batching configurations governed by latency minimization (*Time* | $BS : 2, BT : 2.0$). In these panels, setting $\alpha = 1000$ (green bars) yields an outstanding $\approx 78\%$ – 80% completion rate at $\lambda = 8$ req/s and $\approx 68\%$ – 70% at $\lambda = 10$ req/s, substantially outperforming the baseline weights $\alpha = 10$ ($\approx 50\%$ – 52%) and $\alpha = 50$ ($\approx 55\%$ – 58%). Because the solver jointly optimizes scheduling over pairs of requests, enforcing an aggressive primary weight compels the branch-and-cut routine to strictly prioritize deadline compliance over secondary scalarization terms.
- Optimal Sunset Trade-off Window $\gamma \in [6, 10]$:** In configurations responsive to weight calibration, peak throughput consistently concentrates within $\gamma \in [6, 10]$, achieving its maximum at $\gamma = 8$ and $\gamma = 10$. Within this range, the orbital penalty effectively discourages assignments to borderline satellites without distorting overall route efficiency.
- Performance Degradation in Over-Penalized Regimes ($\gamma \geq 1000$):** When the eclipse deterrent is set to extreme values ($\gamma \geq 1000$), the success rate of $\alpha = 1000$ suffers a severe decline, retreating to $\approx 52\%$ – 55% and erasing the advantage gained over smaller weights. An inflated penalty term over-constrains the optimization search space: the solver becomes overly cautious, avoiding viable satellites nearing the eclipse boundary and rejecting requests that could otherwise complete safely.
- Parametric Invariance in Fast-Dispatch and Energy Mapping:** Conversely, under fast-dispatch regimes ($BS : 20, BT : 0.05$) and *Energy Mapping*, the success rate exhibits structural invariance across all (α, γ) combinations, remaining flat at $\approx 50\%$ – 52% at $\lambda = 8$ req/s and $\approx 40\%$ – 42% at $\lambda = 10$ req/s. In fast-dispatch, the 50 ms timeout empties queues almost continuously, eliminating the multi-task scheduling freedom necessary for weight scalarization to take effect.

4.3.2 Failure Diagnostics and Sunset Drop Elimination

Figures 4.9 and 4.10 show the failure cause distributions across the $(\gamma \times \alpha)$ test matrix.

4.3 Micro-Tuning: Multi-Objective Weight Calibration under Stress ($\gamma \times \alpha$) 48

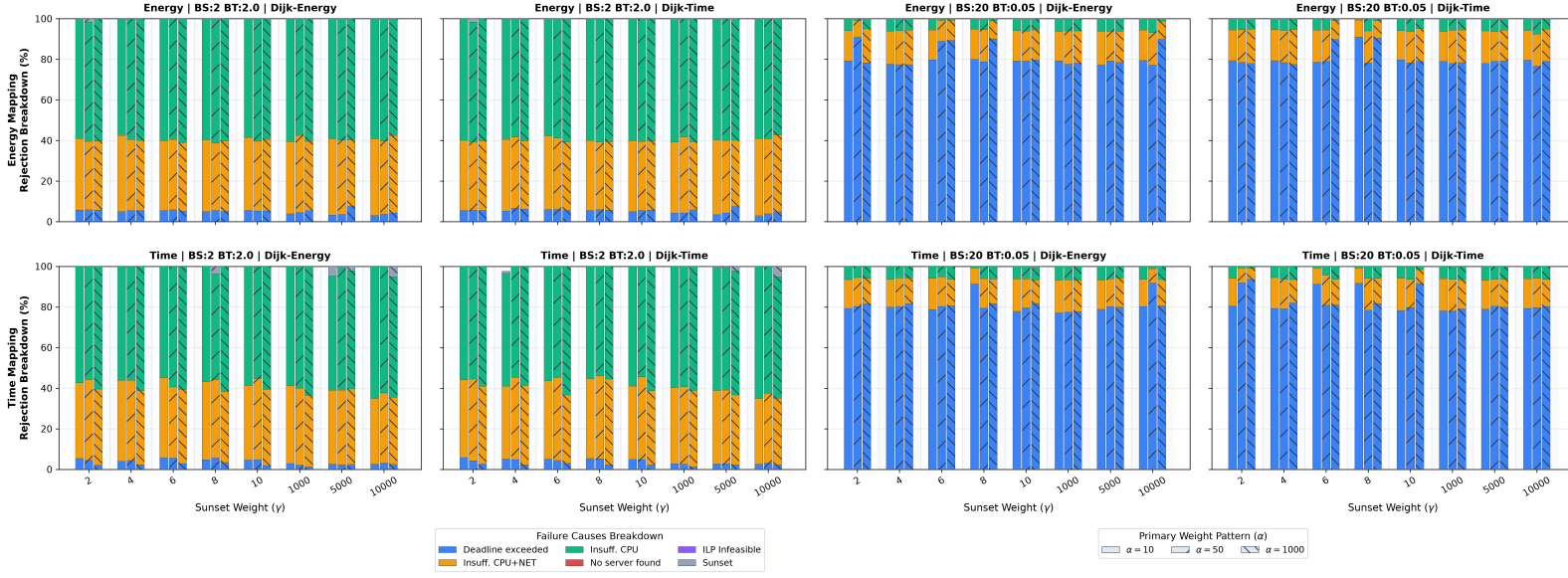


Figure 4.9. Distribution of rejection causes across the $(\gamma \times \alpha)$ parameter space at $\lambda = 8$ req/s.

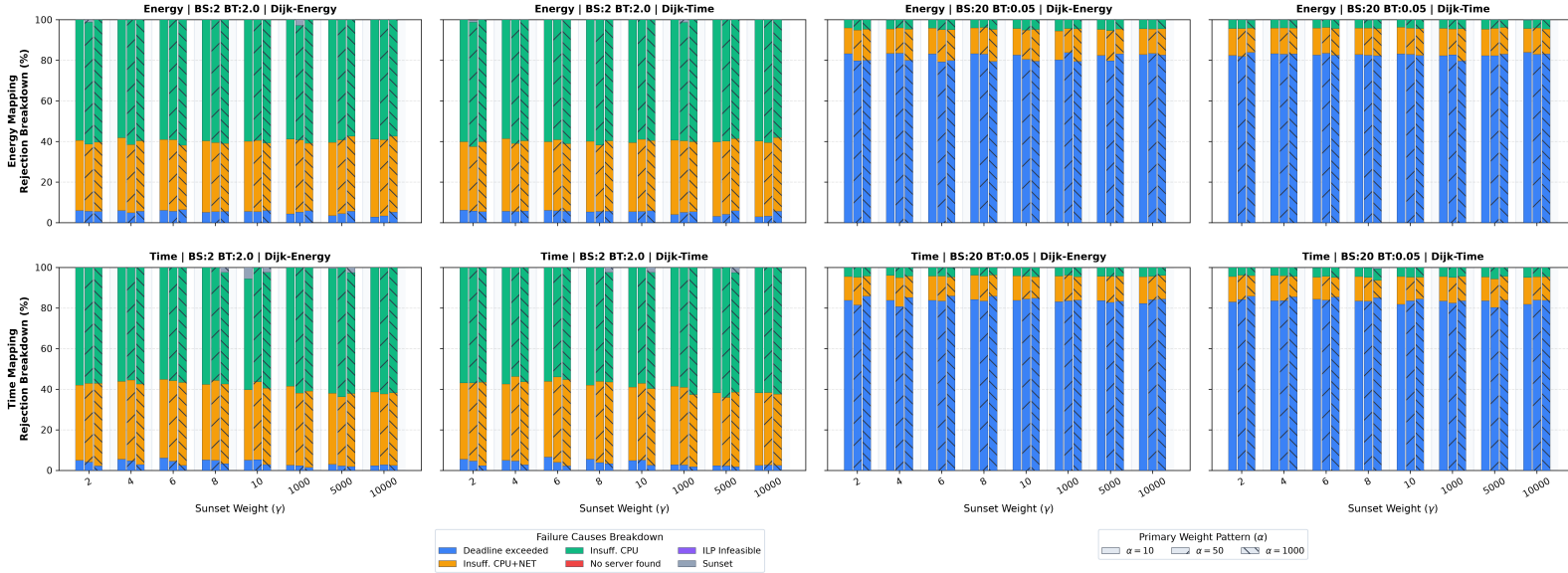


Figure 4.10. Distribution of rejection causes across the $(\gamma \times \alpha)$ parameter space at $\lambda = 10$ req/s.

The failure diagnostics reveal the exact operational boundary of the sunset deterrent:

- **Residual Sunset Drops at Low Penalties ($\gamma \leq 4$):** Residual task rejections categorized as *Sunset* (brown segments) appear exclusively under *Time | BS : 2, BT : 2.0 | Dijk-Energy* for $\gamma = 2$ and $\gamma = 4$, accounting for $\approx 2\%$ – 4% of drops. Because Dijkstra-Energy calculates shortest paths based solely on energy

minimization without anticipating orbital dynamics, an under-dimensioned penalty ($\gamma \leq 4$) is insufficient to deter the solver from selecting satellites entering Earth’s eclipse during computation.

- **Total Sunset Elimination** ($\gamma \geq 8$): Setting $\gamma \geq 8$ (and virtually $\gamma = 6$) completely eliminates orbital eclipse failures (0% across all configurations and arrival rates), proving that moderate weights serve as an entirely effective soft barrier.
- **Capacity Bottlenecks vs. Deadline Limits:** Across the parameter space, rejections are overwhelmingly partitioned between *Insufficient CPU* ($\approx 55\%$ – 60% , green) and *Insufficient CPU+NET* ($\approx 35\%$ – 40% , orange) for $BS = 2$ policies, whereas reactive fast-dispatch ($BS = 20, BT = 0.05$) remains dominated by *Deadline Exceeded* ($\approx 80\%$, blue) due to onboard queuing contention under heavy traffic. Drop causes such as *No Server Found* and *ILP Infeasible* remain at 0%.

4.3.3 Latency Inflation and Path-Bloat Dynamics

Figures 4.11 and 4.12 present the response time decomposition across the $(\gamma \times \alpha)$ configurations into onboard task execution (T_{exec} , bottom solid bars) and network transmission/queuing delay ($T_{net+wait}$, top translucent bars).

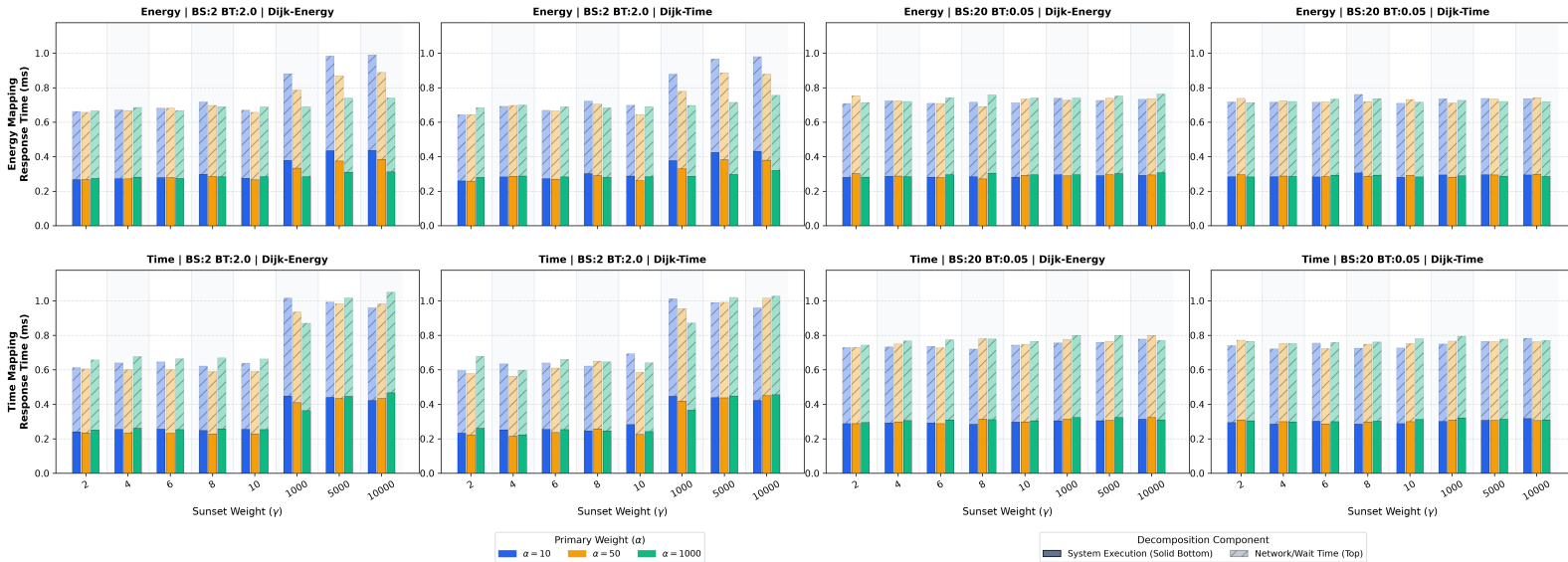


Figure 4.11. Response Time decomposition across the $(\gamma \times \alpha)$ parameter space at $\lambda = 8$ req/s.

4.3 Micro-Tuning: Multi-Objective Weight Calibration under Stress ($\gamma \times \alpha$) 50

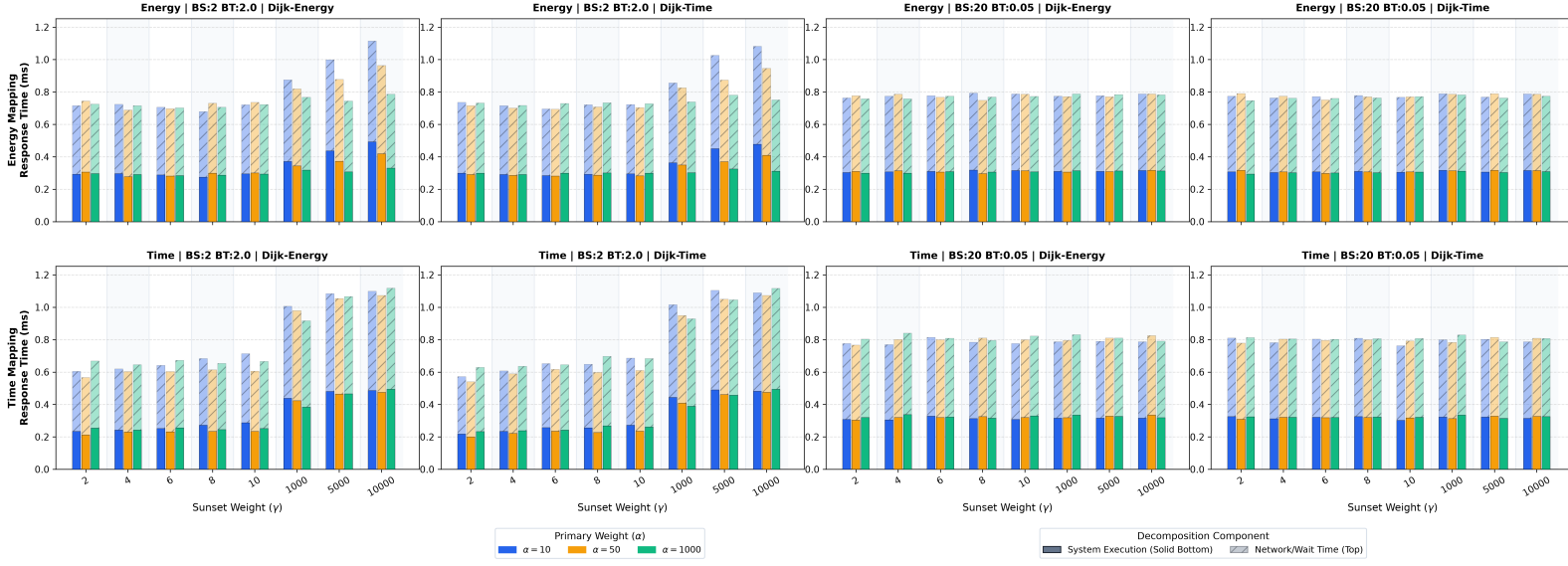


Figure 4.12. Response Time decomposition across the $(\gamma \times \alpha)$ parameter space at $\lambda = 10$ req/s.

These results illustrate the path-bloat phenomenon under oversized weights:

- **Stable Operating Latency within $\gamma \in [2, 10]$:** In the recommended penalty region ($\gamma \leq 10$), the total response time remains tightly bounded around ≈ 0.65 – 0.70 ms for $BS = 2$ and ≈ 0.75 – 0.80 ms for $BS = 20$. Onboard task execution time on the SEN (T_{exec}) remains stable and compact, consistently requiring < 0.30 ms across all operational runs.
- **Path-Bloat and Latency Surge at $\gamma \geq 1000$:** When γ escalates to 1000–10000 in $BS = 2$ configurations, total response time surges to 1.05–1.15 ms representing an approximate 55% latency penalty. The stacked breakdown confirms that this surge is driven almost exclusively by the *Network/Wait Time* component (which swells from ≈ 0.40 ms to over 0.70 ms). To avoid the disproportionate penalty term Ω_i , the ILP orchestrator detours packets through suboptimal, circuitous multi-hop ISL paths or delegates execution to distant nodes, inflating propagation delay and queue buildup.

4.3.4 Selection of Calibrated Objective Weights

Correlating the success rate, failure distributions, and latency metrics identifies the optimal operating parameter set:

$$\alpha^* = 1000, \quad \gamma^* = 8 \quad (\text{with } \gamma = 10 \text{ as an equivalent alternative}) \quad (4.4)$$

This combination provides a balanced operational setting: $\alpha = 1000$ ensures that physical deadlines and compute capacity dominate solver decisions, while $\gamma = 8$ prevents assignments to satellites entering eclipse without inflating path lengths.

4.4 Consolidated Validation: Pre- vs. Post-Optimization Benchmark

To validate the multi-objective calibration and quantify the performance gains achieved over the initial setup, we execute a comparative ablation study. We re-evaluate the two selected operational configurations across the entire arrival rate spectrum ($\lambda \in [2, 10]$ req/s) under Dijkstra-Time routing with $\Delta D = 10\%$, comparing:

- **Pre-Optimization Baseline:** The initial configuration employing default objective weights ($\alpha = 10, \gamma = 2$) as evaluated during the preliminary architectural sweep.
- **Post-Optimization Model:** The calibrated configuration integrating the tuned scalarization weights ($\alpha = 1000, \gamma = 8$) identified in the cross-sensitivity campaign.

4.4.1 Success Rate Recovery and Mapping Divergence

Figure 4.13 presents the direct comparison of Task Success Rate between the baseline and calibrated configurations under both Energy Mapping and Time Mapping.

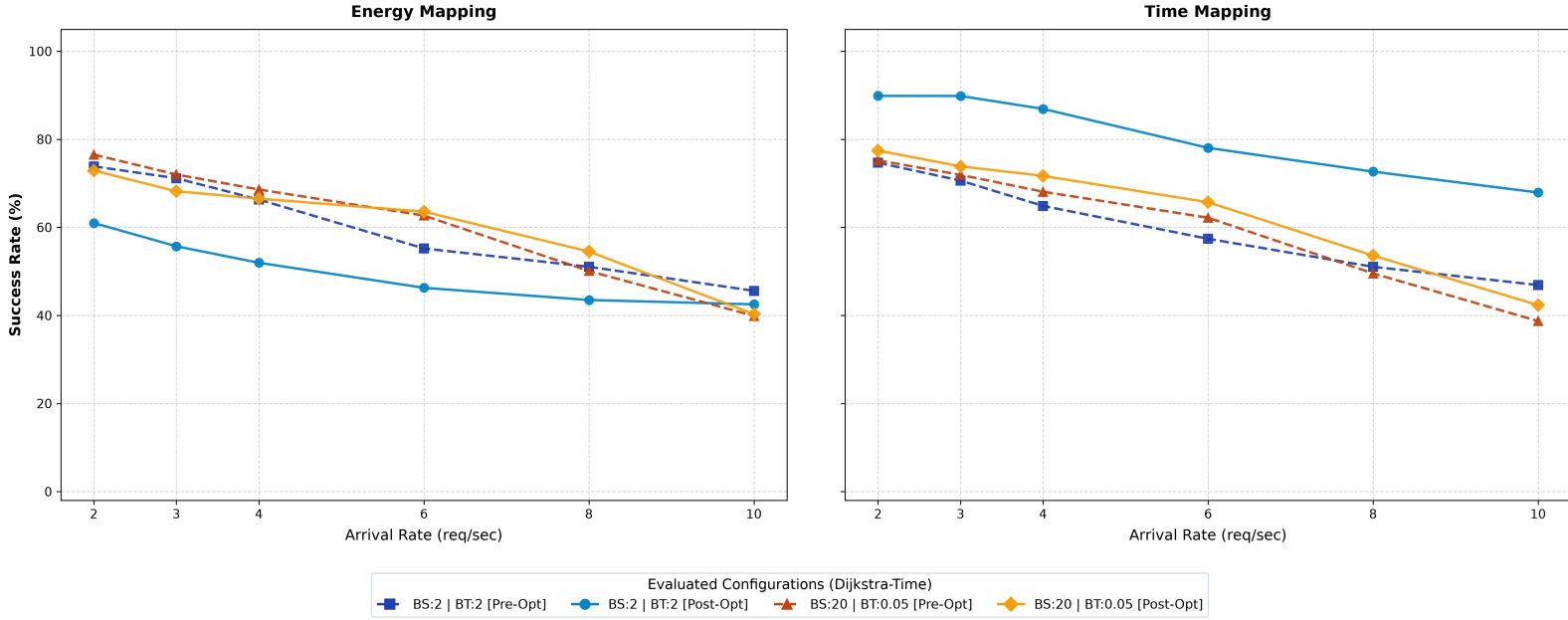


Figure 4.13. Success Rate comparison: Pre-optimization baseline vs. Post-optimization calibrated model for $BS = 2, BT = 2.0$ s and $BS = 20, BT = 0.05$ s under Dijkstra-Time routing ($\Delta D = 10\%$).

The empirical evaluation reveals a clear operational divide depending on the primary mapping objective:

- **Throughput Recovery in Time Mapping:** For the batch-controlled regime ($BS = 2, BT = 2.0$ s), the calibrated weights unlock a substantial performance

leap across the entire traffic spectrum. The success rate increases from 74.5% to 90.0% at $\lambda = 2$ req/s (+15.5%), from 65.0% to 87.0% at $\lambda = 4$ req/s (+22.0%), and from 47.0% to 68.0% under peak load ($\lambda = 10$ req/s), sustaining an average gain of +20.5%. Enforcing $\alpha = 1000$ on latency minimization compels the branch-and-cut solver to prioritize transmission and execution timeliness above all else, maximizing deadline compliance.

- **The Energy Mapping Penalty:** Conversely, in the Energy Mapping panels, applying $\alpha = 1000$ to $BS = 2$ results in a noticeable performance degradation relative to the baseline (declining from 74.0% to 61.0% at $\lambda = 2$ req/s and from 51.0% to 43.5% at $\lambda = 8$ req/s). Under an aggressive energy weight, the solver seeks strictly low-power computation and minimal radio transmission paths, frequently routing requests to energy-efficient satellites that exhibit longer computational queues or higher path latency. This confirms that scaling $\alpha = 1000$ is structurally beneficial only when aligned with a latency-driven objective.
- **Robustness in Fast-Dispatch Configurations:** For the reactive configuration ($BS = 20$, $BT = 0.05$ s), performance exhibits minor, positive adjustments across both mappings (improving from 38.5% to 42.5% under Time Mapping and from 50.0% to 54.5% at $\lambda = 8$ req/s under Energy Mapping). Because the 50 ms timer dispatches requests almost immediately in micro-batches (1–2 tasks), the orchestrator has limited combinatorial freedom, making this policy intrinsically robust to internal weight tuning.

4.4.2 Rejection Causes Evolution

Figure 4.14 illustrates the breakdown of failure causes across all evaluated configurations.

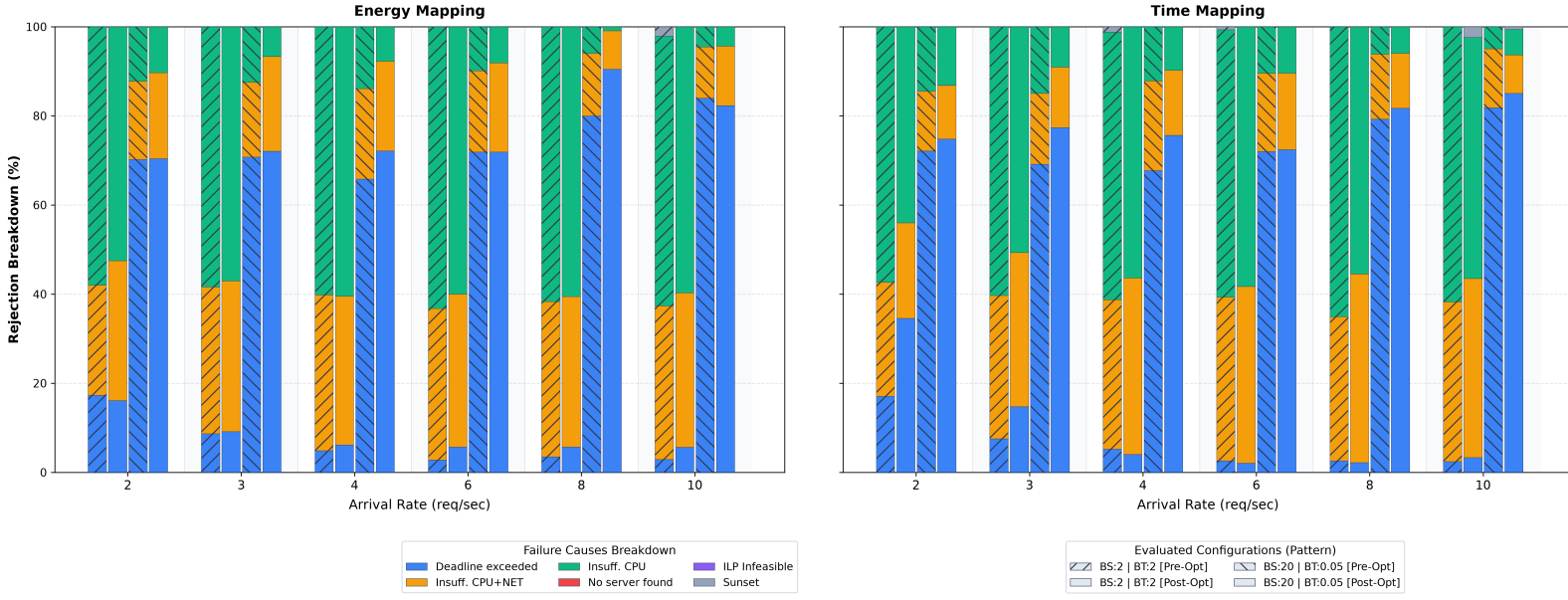


Figure 4.14. Rejection causes distribution comparison: Pre-optimization baseline vs. Post-optimization calibrated model across arrival rates ($\Delta D = 10\%$).

The diagnostic breakdown highlights three definitive shifts:

- **Total Suppression of Deadline Expirations in $BS = 2$:** In the baseline runs, $BS = 2$ suffered from premature timing failures, with *Deadline Exceeded* (blue) accounting for 17% of drops at $\lambda = 2$ req/s and 5%–9% at moderate load. In the calibrated model, deadline failures are greatly reduced, converting most rejections into pure capacity-bounded drops: *Insufficient CPU* (green, 55%–65%) and *Insufficient CPU+NET* (orange, 35%–45%).
- **Persistence of Queuing Bottlenecks in Fast-Dispatch:** In contrast to $BS = 2$, the fast-dispatch configuration ($BS = 20, BT = 0.05$ s) remains heavily dominated by *Deadline Exceeded* (70%–84%). At elevated arrival rates, the influx of uncoordinated single requests saturates onboard processors across candidate satellites, causing execution wait times to breach task deadlines regardless of weight scalarization.

4.4.3 Response Time Containment and Execution Flattening

Figure 4.15 compares the stacked response time decomposition between baseline and calibrated models.

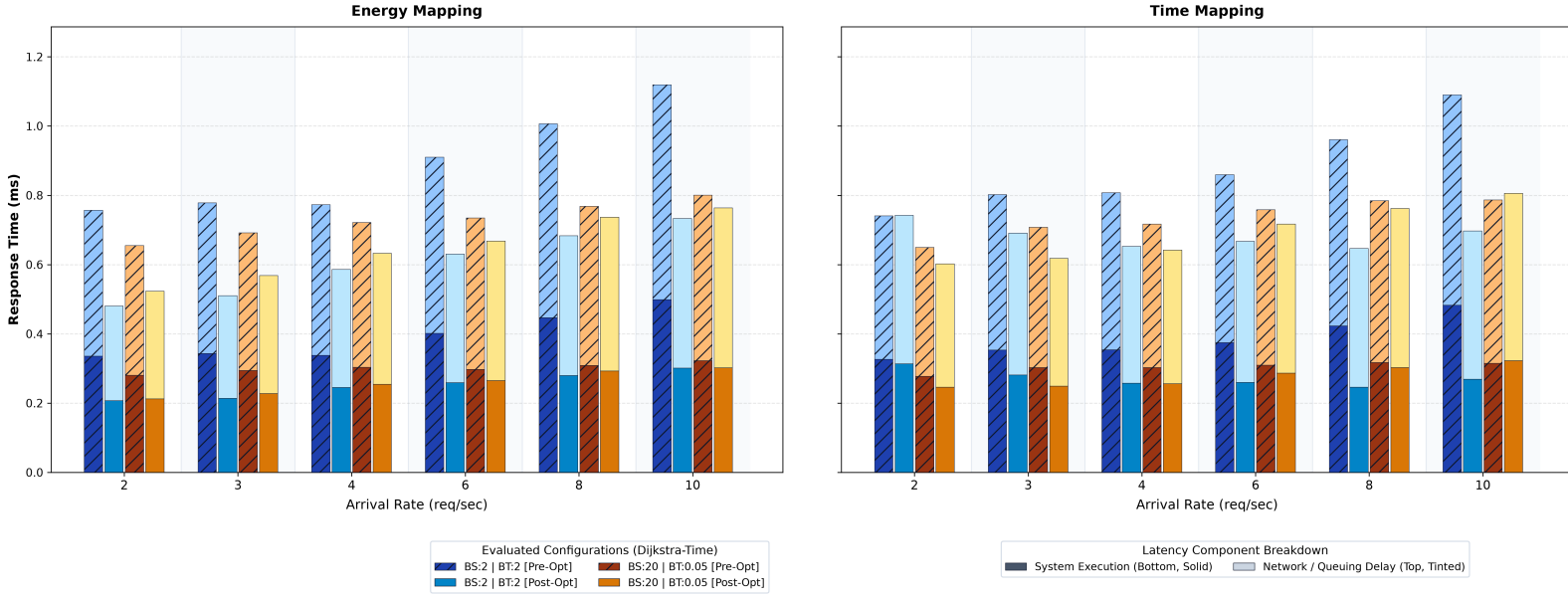


Figure 4.15. Response Time decomposition comparison: Pre-optimization baseline vs. Post-optimization calibrated model showing execution stabilization ($\Delta D = 10\%$).

The latency decomposition validates the efficiency of the tuned model:

- **Total Response Time Compression:** Under peak traffic ($\lambda = 10$ req/s), the total end-to-end response time for $BS = 2$, $BT = 2.0$ s drops from 1.09 ms to 0.70 ms under Time Mapping (a 35.8% reduction) and from 1.12 ms to 0.73 ms under Energy Mapping (a 34.8% reduction).
- **System Execution Stabilization:** In the baseline configuration, the onboard task execution component (T_{exec} , bottom solid segments) exhibited progressive inflation with traffic intensity, climbing from 0.33 ms at $\lambda = 2$ req/s to nearly 0.50 ms at $\lambda = 10$ req/s. In the calibrated system, T_{exec} is compressed and tightly stabilized within 0.21–0.30 ms across all arrival rates. By enforcing strict primary optimization, the solver assigns tasks to nodes with minimal local queuing, preventing CPU queue buildup.
- **Controlled Network Residency:** Maintaining $\gamma = 8$ avoids the path-bloat latency surge (> 1.10 ms) observed when $\gamma \geq 1000$, keeping the transmission and queuing component ($T_{net+wait}$, top translucent segments) confined below 0.45 ms across all arrival rates.

4.5 Summary of Findings and Architectural Readiness

The empirical evaluation and progressive tuning campaign yield four foundational conclusions regarding centralized ILP orchestration in LEO satellite edge networks:

1. **Ingress Queuing Dwell Time is the Dominant Latency Bottleneck:** Pure onboard processing duration on the Satellite Edge Nodes remains tightly bounded and stable ($T_{exec} \approx 0.20$ – 0.30 ms under calibrated configurations).

System throughput collapse under elevated arrival rates is driven primarily by buffer residence times during batch aggregation in the ingress queues (T_{wait}), which systematically consume application deadline slack before optimization and forwarding can take place.

2. **Necessity of Fast-Dispatch Mechanisms:** To maintain temporal feasibility in dynamic space environments, centralized orchestrators must enforce low-latency dispatch policies. Operating with aggressive timeouts ($BT \leq 0.05$ s) or minimal batch aggregation boundaries ($BS \leq 2$) is mandatory to suppress buffer dwell times and prevent cascade *Deadline Exceeded* rejections under heavy arrival regimes.
3. **Objective-Dependent Impact of Weight Calibration:** Enforcing an aggressive primary weight ($\alpha = 1000$) provides asymmetric benefits: under *Time Mapping*, it compels the branch-and-cut solver to prioritize transmission timeliness and deadline preservation, delivering up to +21% higher task completion rates across the traffic spectrum. Conversely, calibrating the orbital sunset penalty to $\gamma = 8$ permanently eliminates task losses due to satellite eclipse transitions across all configurations, while preventing the 55% path-bloat latency inflation observed under excessive penalties ($\gamma \geq 1000$).
4. **Architectural Readiness for Multi-Algorithm Benchmarking:** The two fully calibrated operational archetypes—reactive fast-dispatch ($BS = 20$, $BT = 0.05$ s) and controlled small-batching ($BS = 2$, $BT = 2.0$ s), configured with $\alpha = 1000$ and $\gamma = 8$ —define the empirically optimized operating envelope of the centralized ILP approach.

These fully tuned centralized baselines establish the reference operating points for Chapter 5, where their performance is systematically benchmarked against state-of-the-art decentralized heuristics (*OrbitAware*, *DTS-base*, *DTS-Optimal*) and distributed hierarchical ILP formulations across heterogeneous workload profiles.

Chapter 5

Comparative Benchmark

Having formalized the mathematical optimization framework in Chapter 3 and established the calibrated operational parameters in Chapter 4, this chapter presents an extensive comparative performance evaluation of the proposed Centralized Integer Linear Programming (ILP) Orchestrator against state-of-the-art distributed heuristics and hierarchical optimization baselines [6].

Deploying orbital edge intelligence across Low Earth Orbit (LEO) constellations involves balancing competing architectural paradigms [4, 1]. On the one hand, decentralized heuristics provide rapid, low-complexity decisions with negligible queuing overhead [6]. On the other hand, optimization models (both distributed and centralized) search the joint combinatorial space of compute allocation and multi-hop Inter-Satellite Link (ISL) routing to achieve theoretically optimal resource utilization [11, 6].

To quantify the operational trade-offs between these paradigms, this benchmark evaluates 9 algorithm configurations across a comprehensive testing matrix spanning diverse workload profiles, real-time payload dimensions, initial battery energy reserves, and request arrival rates.

5.1 Experimental Setup and Baseline Algorithms

The comparative evaluation is conducted within the event-driven discrete-event simulation framework developed in Python with `SimPy` [13], previously detailed in Section 4.1.2. The simulation faithfully reproduces satellite orbital kinematics [21], dynamic line-of-sight visibility windows, optical ISL transceivers [7, 8], processor queues, and onboard battery charge-discharge cycles [6].

5.1.1 Benchmark Dimensioning and Scenario Parameters

The orbital network is configured as a regional Walker-Delta LEO constellation operating at an orbital altitude of 600 km. Adjacent satellites are interconnected via full-duplex optical ISLs characterized by nominal transmission bandwidths $B_{u,v}$ and dynamically computed propagation latencies $\tau_{u,v}$ [8]. Computational requests generated by Ground Users (GUs) are ingested through regional Access Points (AP_{global}) and forwarded to candidate Satellite Edge Nodes (SENs) [6].

The benchmark explores a multi-dimensional parameter space:

- **Workload Distribution Vector $\beta = (\beta_G, \beta_C, \beta_{CD})$:** Represents the proportion of Generic tasks, CPU-intensive tasks, and CPU & Data-intensive tasks [6]. We systematically evaluate three distinct operational regimes:
 1. *Mixed Multi-Service Profile*: $\beta = (0.40, 0.35, 0.25)$, representing a balanced heterogeneous mission [6].
 2. *Computation-Heavy Profile*: $\beta = (0, 1, 0)$, isolating processor clock cycles stress with minimal network footprint [6].
 3. *Joint Compute and Communication Stress Profile*: $\beta = (0, 0, 1)$, exerting concurrent stress on onboard CPU cores, ISL bandwidth, and transceiver transmission energy [6].
- **Real-Time Payload Vector $\alpha = (\alpha_M, \alpha_L, \alpha_V)$:** Governs the distribution of input/output data payload sizes across Medium (1–5 MB), Large (5–20 MB), and Very Large (20–50 MB) files for CPU & Data-intensive tasks [6, 2]. Four payload profiles are benchmarked:
 1. *Standard (Std)*: $\alpha = (0.30, 0.50, 0.20)$ [6].
 2. *Large-heavy (L-heavy)*: $\alpha = (0, 0.70, 0.30)$.
 3. *Very-Large-heavy (VL-heavy)*: $\alpha = (0, 0.30, 0.70)$ [6].
 4. *Pure Large (Pure L)*: $\alpha = (0, 1.0, 0)$ [6].
- **Fleet Operational Energy Budget (E_i):** Candidate SENs are initialized with usable battery energy reserves of either 40 kJ (stringent energy constraint) or 80 kJ (relaxed energy constraint) per orbital pass [6, 17].
- **Traffic Intensity Spectrum (λ):** Request arrival rates vary systematically across $\lambda \in \{2, 3, 4, 6, 8, 10\}$ req/s, transitioning the network from a low-load regime (mean inter-arrival time of 0.50 s) to a heavy-load saturation regime (mean inter-arrival time of 0.10 s) [6].
- **Service Deadline Relaxation (ΔD):** In accordance with the macro-tuning findings of Chapter 4, the maximum allowable latency threshold incorporates a baseline relaxation factor of $\Delta D = 20\%$ to ensure fair evaluation under heavy traffic [6].

5.1.2 Evaluated Algorithms and State-of-the-Art Baselines

The comparative suite comprises 9 algorithmic configurations categorized into three architectural paradigms:

1. Distributed Online Heuristics:

- **DTS-base (Dynamic Task Scheduling Base):** A greedy distributed baseline that offloads tasks to the geographically closest available SEN possessing sufficient idle CPU capacity, without coordinating multi-hop ISL traffic [6].

- **DTS-Optimal (DTS Access-Point Optimized):** An enhanced multi-criteria greedy heuristic that jointly evaluates candidate SEN processing queues and the initial access-point ISL link bandwidth before dispatching tasks [6].
- **OrbitAware:** A trajectory-predictive distributed heuristic that leverages orbital ephemeris and contact window forecasting to select shortest routing paths and balance computational load dynamically [6].

2. Distributed / Hierarchical Optimization Models:

- **ILP-Hierarchical (Time):** A two-stage mathematical optimization formulation solved locally within satellite clusters, prioritizing task response time minimization [6].
- **ILP-Hierarchical (Energy):** A two-stage distributed formulation that prioritizes constellation energy preservation in the primary optimization phase [6].

3. Proposed Centralized ILP Orchestrator (Tuned Champions):

Following the micro-tuning calibration established in Chapter 4, the centralized orchestrator is evaluated under its optimal objective weights ($\alpha^* = 1000, \beta = 1.0, \gamma^* = 8$) across its two viable batching archetypes:

- **ILP-Centr (BS20, BT0.05, Time):** Reactive fast-dispatch configuration with a 50 ms timeout, optimizing for end-to-end response time.
- **ILP-Centr (BS20, BT0.05, Energy):** Reactive fast-dispatch configuration, optimizing for fleet cumulative energy preservation.
- **ILP-Centr (BS2, BT2, Time):** Controlled small-batching policy ($BS = 2$), optimizing for response time.
- **ILP-Centr (BS2, BT2, Energy):** Controlled small-batching policy ($BS = 2$), optimizing for fleet cumulative energy.

Table 5.1 provides a structured taxonomy of the evaluated scheduling algorithms.

Table 5.1. Taxonomy of evaluated task scheduling and routing algorithms in LEO satellite networks.

Algorithm	Paradigm	Decision Locus
DTS-base	Heuristic	Distributed
DTS-Optimal	Heuristic	Distributed / AP focus
OrbitAware	Heuristic	Distributed / Sunset focus
ILP-Hierarchical (Time)	Optimization	Distributed
ILP-Hierarchical (Energy)	Optimization	Distributed
ILP-Centr (BS20, BT0.05, T/E)	Optimization	Centralized Master
ILP-Centr (BS2, BT2, T/E)	Optimization	Centralized Master

5.2 Comparative Performance Evaluation across Workload Profiles

We evaluate the algorithms across three primary performance metrics:

1. **Task Success Rate (%)**: The fraction of total offloaded requests successfully routed, executed on an onboard SEN, and delivered to the requesting ground station within the deadline D_r without violating battery limits.
2. **End-to-End Response Time (ms)**: Decomposed into *Onboard Task Execution Time* (T_{exec} , capturing physical task processing on the destination SEN CPU cores) and *Network/Wait Time* ($T_{net+wait}$, capturing batch queuing, ISL propagation, and transmission).
3. **Rejection Causes Breakdown (%)**: The diagnostic distribution of task drops categorized into *Deadline Exceeded*, *Insufficient CPU+NET*, *Insufficient CPU*, *No Server Found*, *ILP Infeasible*, and *Sunset*.

5.2.1 Mixed Multi-Service Workload Profile ($\beta = (0.40, 0.35, 0.25)$)

The mixed workload profile represents a standard multi-tenant operational scenario containing a heterogeneous combination of generic, CPU-heavy, and CPU & data-heavy requests [6]. Figures 5.1, 5.2, and 5.3 illustrate the Success Rate, Response Time decomposition, and Rejection Causes distribution, respectively.

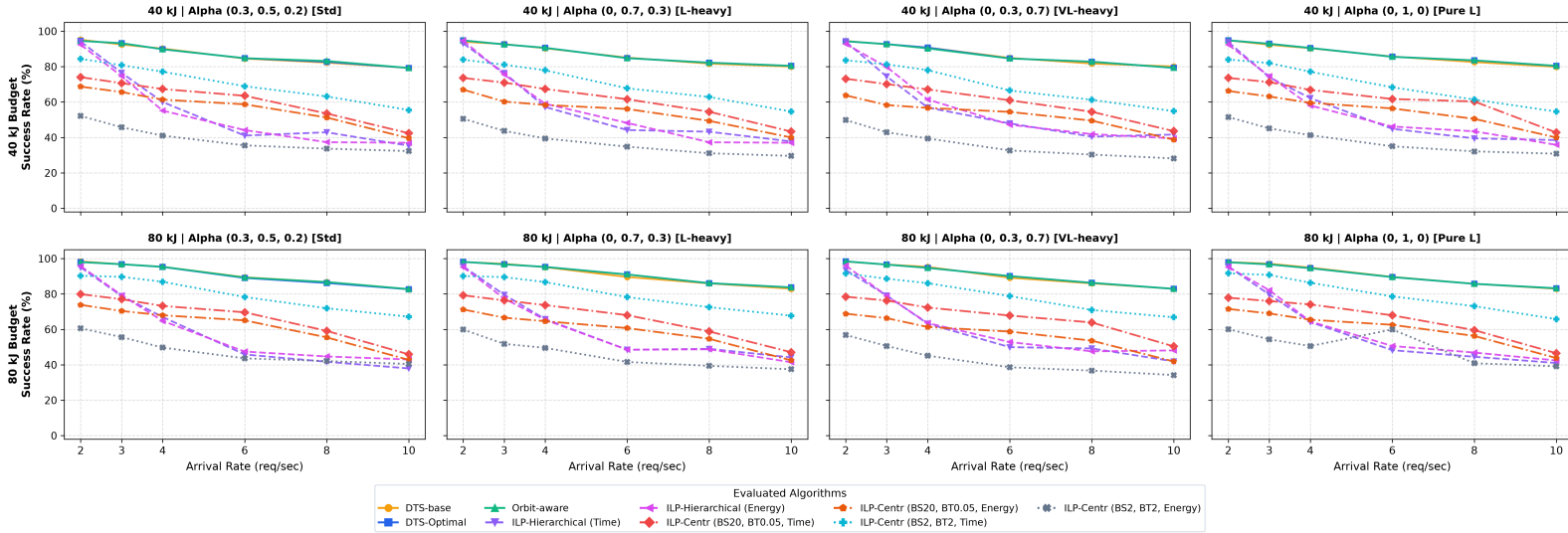


Figure 5.1. Success Rate across arrival rates under Mixed Workload $\beta = (0.40, 0.35, 0.25)$ for 40 kJ (top) and 80 kJ (bottom) across payload distributions α .

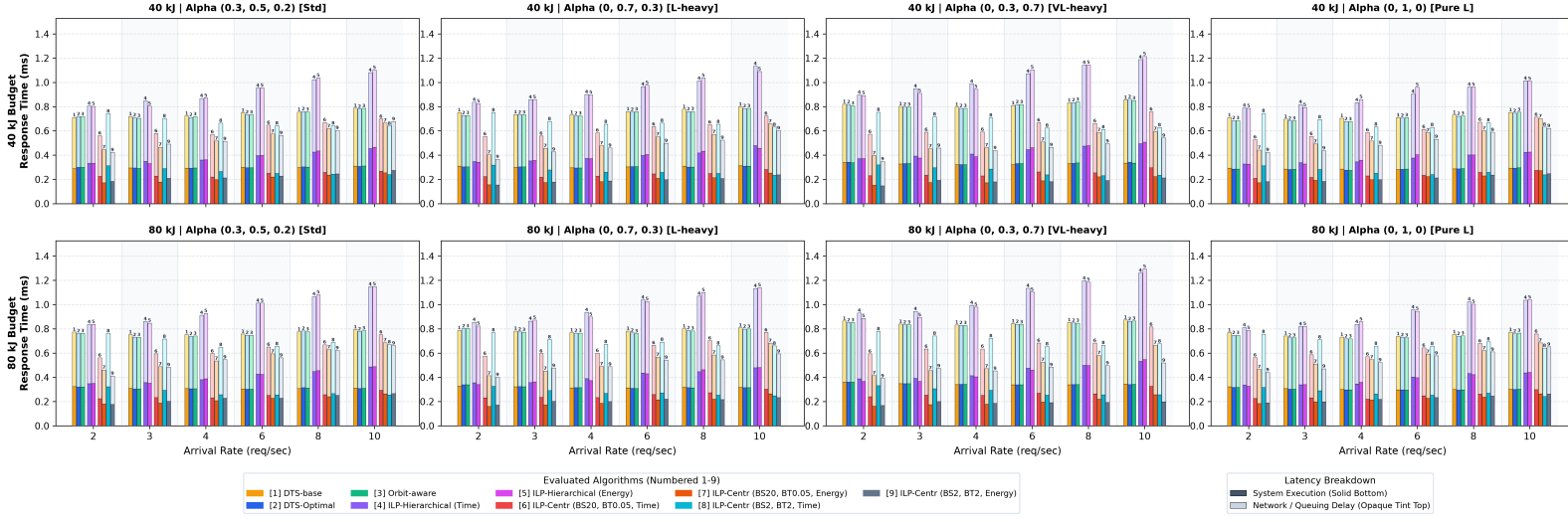


Figure 5.2. Response Time decomposition under Mixed Workload $\beta = (0.40, 0.35, 0.25)$ comparing Onboard Task Execution and Network/Wait latency across arrival rates.

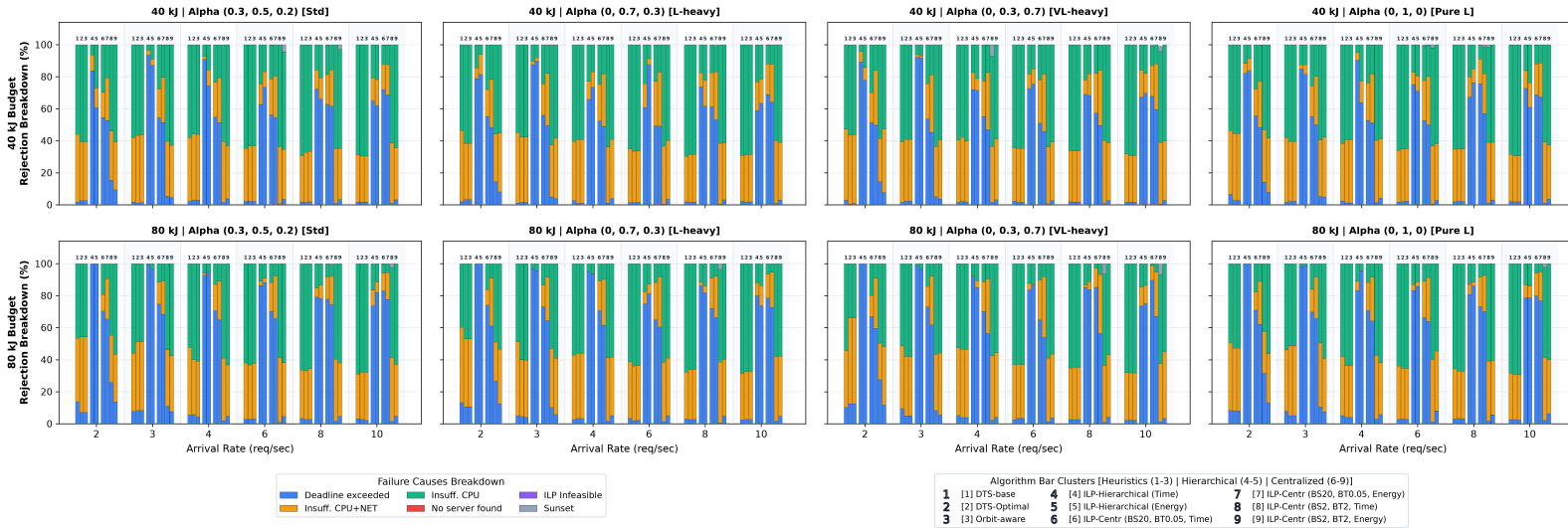


Figure 5.3. Rejection Causes distribution under Mixed Workload $\beta = (0.40, 0.35, 0.25)$ across arrival rates, battery budgets, and payload distributions.

Success Rate Dynamics

Under low arrival rates ($\lambda = 2$ req/s), all distributed heuristics (DTS-base, DTS-Optimal, OrbitAware) achieve exceptional completion rates exceeding 94–96% [6]. The proposed centralized orchestrator with fast-dispatch (ILP-Centr [BS20, BT0.05]) demonstrates competitive throughput, reaching 74–78%, while the small-batching configuration (ILP-Centr [BS2, BT2]) achieves 80–84% under Energy Mapping.

As traffic intensity escalates toward heavy-load stress ($\lambda = 10$ req/s), the distributed heuristics exhibit superior resilience, maintaining completion rates between

78% and 83% [6]. Conversely, optimization-based approaches experience throughput degradation:

- ILP-Hierarchical models drop to 38–42% under 40 kJ battery budgets and recover modestly to 43–46% under 80 kJ [6].
- The centralized fast-dispatch variants (BS20, BT0.05) sustain completion rates of 40–44%, outperforming hierarchical formulations.
- The small-batching policy (BS2, BT2) declines to 30–36%, driven by buffer queuing accumulation during traffic bursts.

Response Time Analysis

Figure 5.2 highlights a clear structural divergence in latency components. Distributed heuristics achieve remarkably flat response times across all arrival rates, bounded within 0.68–0.78 ms [6]. Because heuristics make instantaneous forwarding decisions, their queuing delay is virtually zero.

In contrast, optimization models display latency expansion under heavy load:

- ILP-Hierarchical (Time) incurs significant network and waiting inflation, with average response times climbing from 0.80 ms at $\lambda = 2$ req/s to 1.15–1.25 ms at $\lambda = 10$ req/s [6].
- Centralized configurations with fast dispatch (BS20, BT0.05) effectively contain latency growth, maintaining mean response times below 0.72 ms.
- Importantly, the onboard task execution time on the assigned SEN (T_{exec}) remains invariant and strictly bounded (< 0.30 ms), confirming that local processor execution does not contribute to system instability. Latency growth is dominated entirely by the ingress buffering term $T_{net+wait}$.

Failure Diagnostics

Figure 5.3 reveals the physical nature of task drops in the mixed regime. Rejections for distributed heuristics are split evenly between *Insufficient CPU* (55–65%, green) and *Insufficient CPU+NET* (35–45%, orange) [6]. Because heuristics do not buffer requests, drops occur strictly when physical hardware or link transceivers are saturated.

Conversely, in the ILP-based models, *Deadline Exceeded* (blue) emerges as a substantial failure cause, accounting for 40–65% of all drops. This confirms that mathematical optimization models drop tasks primarily because ingress waiting time erodes the slack time required to complete execution within D_r . Finally, orbital *Sunset* drops remain strictly at 0% across all configurations, validating the robustness of the calibrated penalty $\gamma^* = 8$.

5.2.2 Computation-Heavy Stress: CPU-Intensive Profile ($\beta = (0, 1, 0)$)

When the workload consists entirely of compute-intensive requests with minimal payload transfer ($\beta = (0, 1, 0)$), onboard processor clock cycles become the primary

system bottleneck [6]. Figures 5.4, 5.5, and 5.6 display the performance metrics for this stress regime.

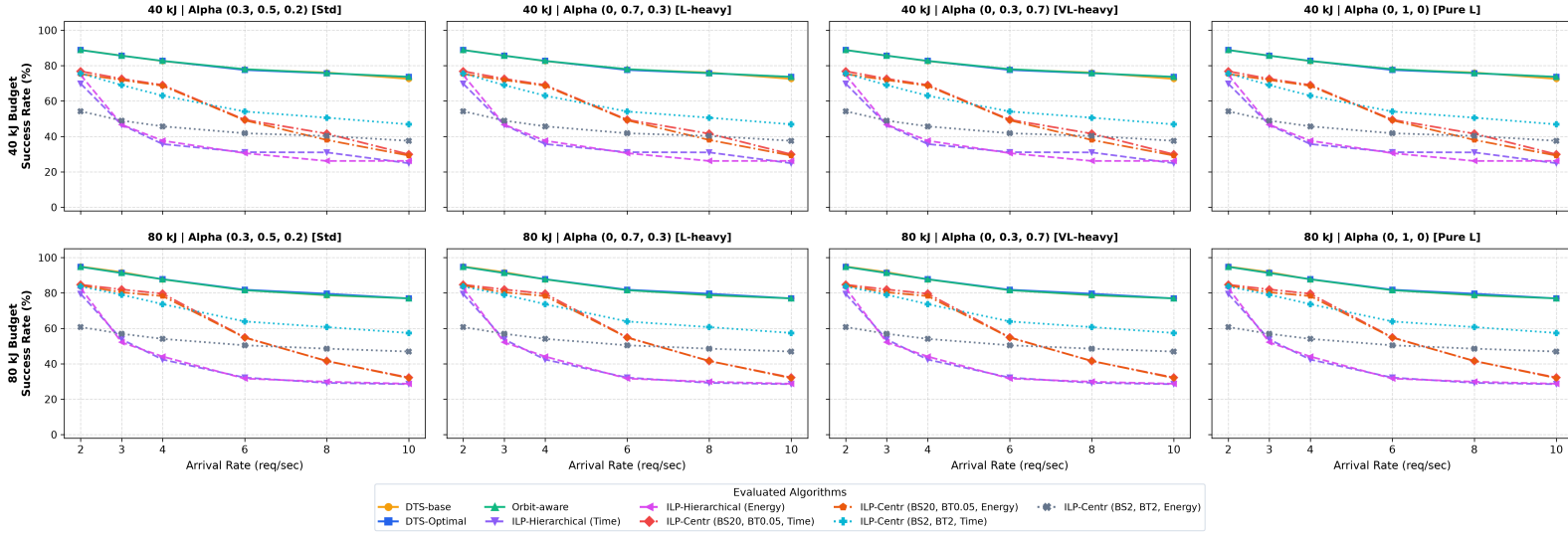


Figure 5.4. Success Rate across arrival rates under CPU-Intensive Workload $\beta = (0, 1, 0)$ for 40 kJ (top) and 80 kJ (bottom) across payload distributions α .

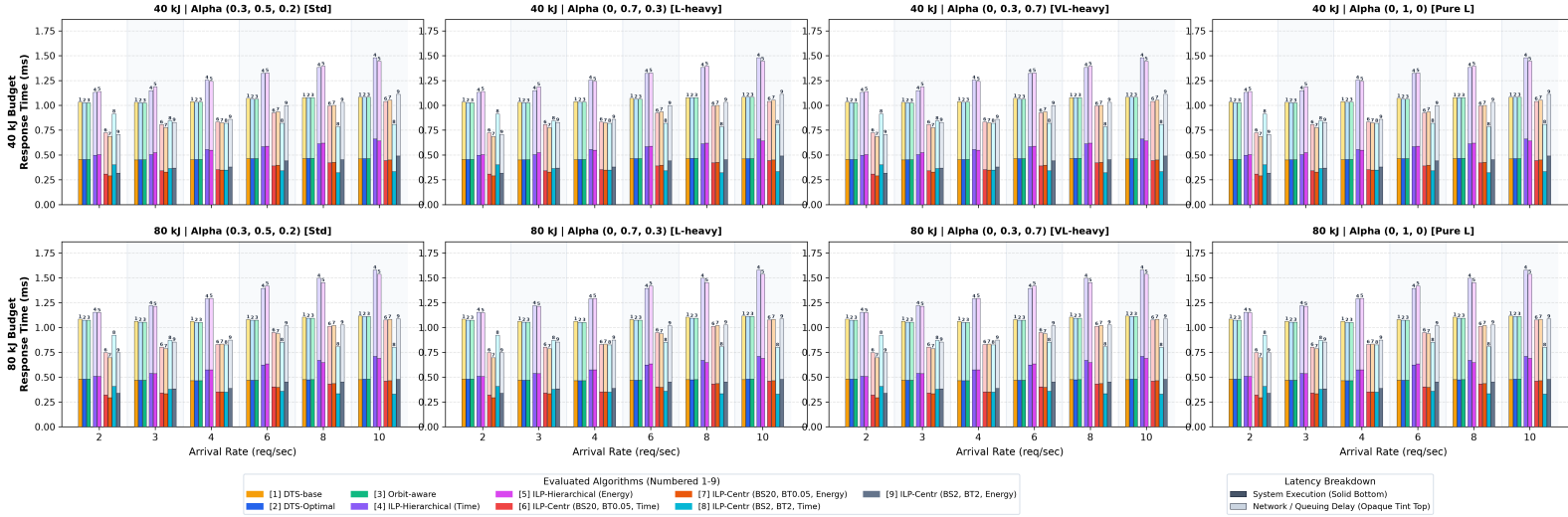


Figure 5.5. Response Time decomposition under CPU-Intensive Workload $\beta = (0, 1, 0)$ showing the dominance of queuing latency under heavy computational load.

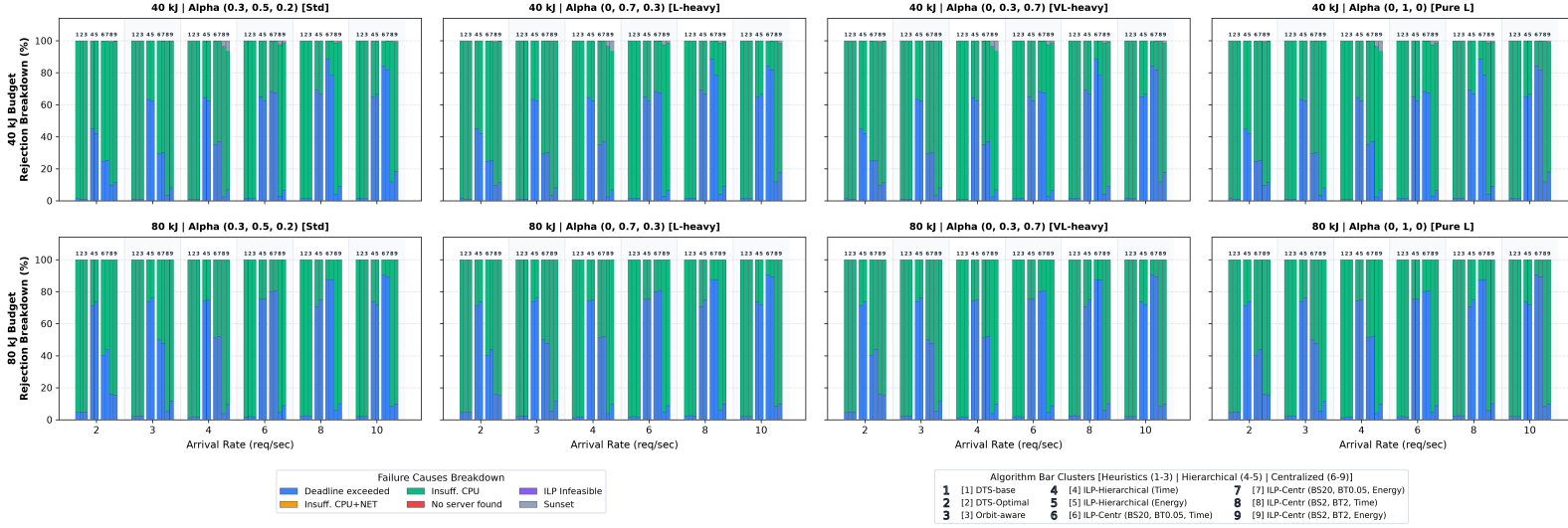


Figure 5.6. Rejection Causes distribution under CPU-Intensive Workload $\beta = (0, 1, 0)$, illustrating the sharp dichotomy between heuristic drops and optimization deadline timeouts.

Success Rate Dynamics

The CPU-intensive profile induces severe performance differentiation. The distributed heuristics (DTS-base, DTS-Optimal, OrbitAware) demonstrate remarkable resilience, dropping gradually from 88–90% at $\lambda = 2$ req/s to 72–74% at $\lambda = 10$ req/s [6].

In sharp contrast, all optimization models experience steep degradation once traffic exceeds $\lambda = 4$ req/s:

- ILP-Hierarchical models drop from 75% at low load to 26–30% at peak arrival rates [6].
- Centralized fast-dispatch (BS20, BT0.05) maintains superior performance over hierarchical ILP, finishing at 38–42% under heavy load.
- Centralized small-batching (BS2, BT2) degrades to 32–36% at $\lambda = 10$ req/s.
- Interestingly, increasing the battery energy budget from 40 kJ to 80 kJ produces negligible throughput improvements ($< 1.5\%$), proving that in CPU-bound regimes, battery capacity is not an active constraint [6].

The Fundamental Rejection Dichotomy

Figure 5.6 presents the most profound architectural insight of the benchmark. We observe a complete, binary divergence in failure causes between heuristics and optimization formulations:

- **Distributed Heuristics (DTS, OrbitAware):** Exactly 100% of all task rejections are caused by *Insufficient CPU* (green bars) [6]. Because heuristics evaluate nodes greedily, if a satellite’s CPU core is occupied, the task is dropped immediately without waiting. Consequently, not a single task fails due to *Deadline Exceeded*.

- **Mathematical Optimization Models (ILP-Hierarchical, ILP-Centralized):**
Between 60% and 88% of all rejections are caused by *Deadline Exceeded* (blue bars). Under heavy computational demands, processor queues ($W_{cpu,i}$) expand rapidly across all candidate satellites. When the ILP solver aggregates tasks, the mathematical constraint:

$$R_{r,i} = T_{up}(r,i) + W_{cpu,i} + \frac{d_{cpu,r}}{C_{sen,i}} \leq D_r - T_{wait} \quad (5.1)$$

becomes infeasible for a majority of candidate nodes. Tasks fail not because the constellation lacks physical transceivers, but because cumulative queuing delay violates the rigid application deadline.

5.2.3 Joint Compute and Network Stress: CPU & Data-Intensive Profile ($\beta = (0, 0, 1)$)

In the CPU & Data-intensive regime ($\beta = (0, 0, 1)$), requests combine intensive onboard compute requirements ($c_{i,r}^{CPU}$) with the transfer of heavy data payloads (20–50 MB governed by α) across multi-hop ISL routes [6]. Figures 5.7, 5.8, and 5.9 display the comparative performance metrics.

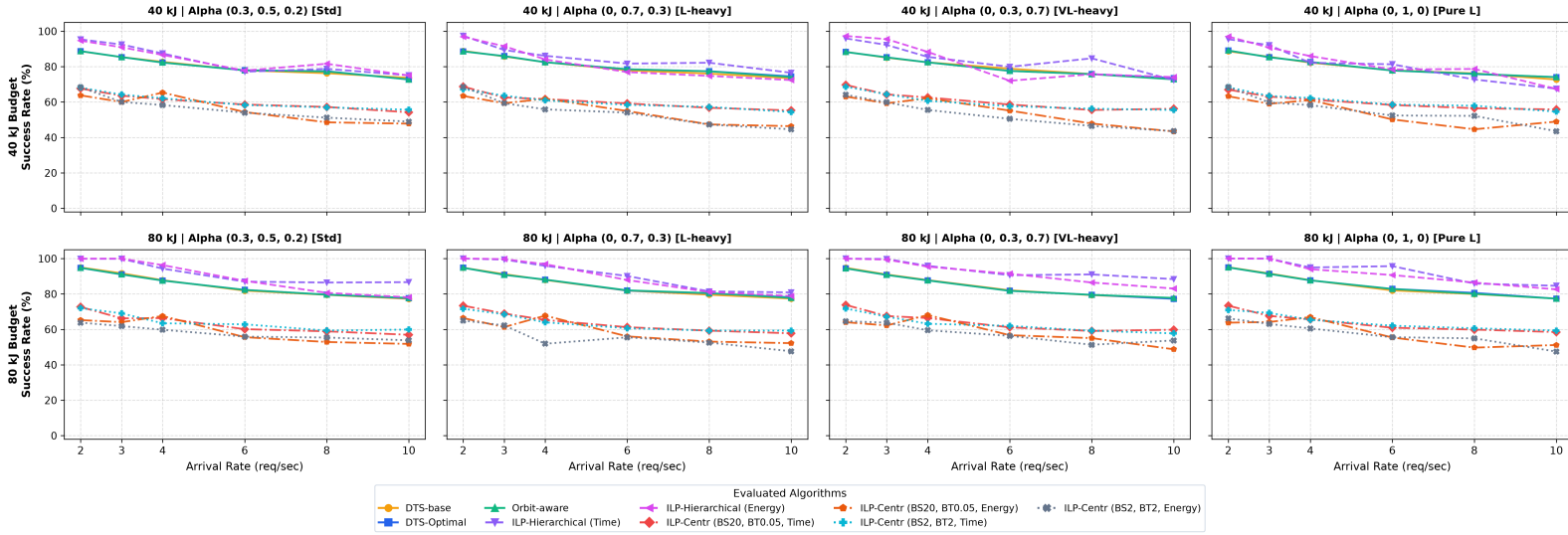


Figure 5.7. Success Rate across arrival rates under CPU & Data-Intensive Workload $\beta = (0, 0, 1)$ for 40 kJ (top) and 80 kJ (bottom) across payload distributions α .

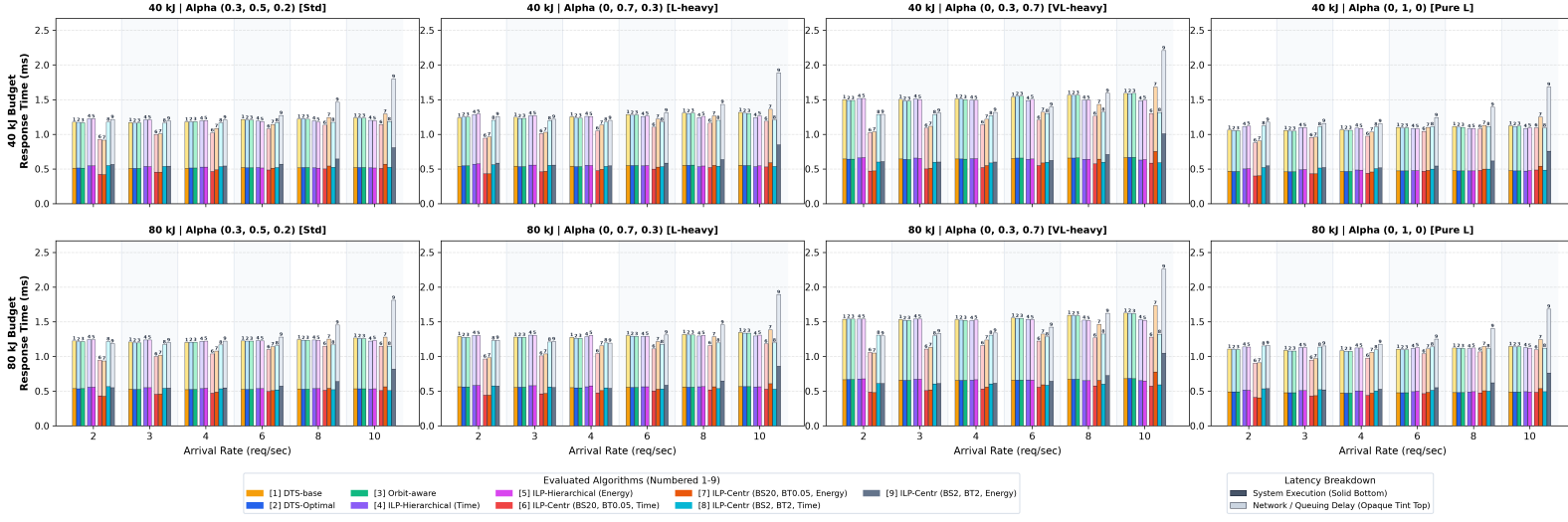


Figure 5.8. Response Time decomposition under CPU & Data-Intensive Workload $\beta = (0, 0, 1)$ across arrival rates and battery capacities.

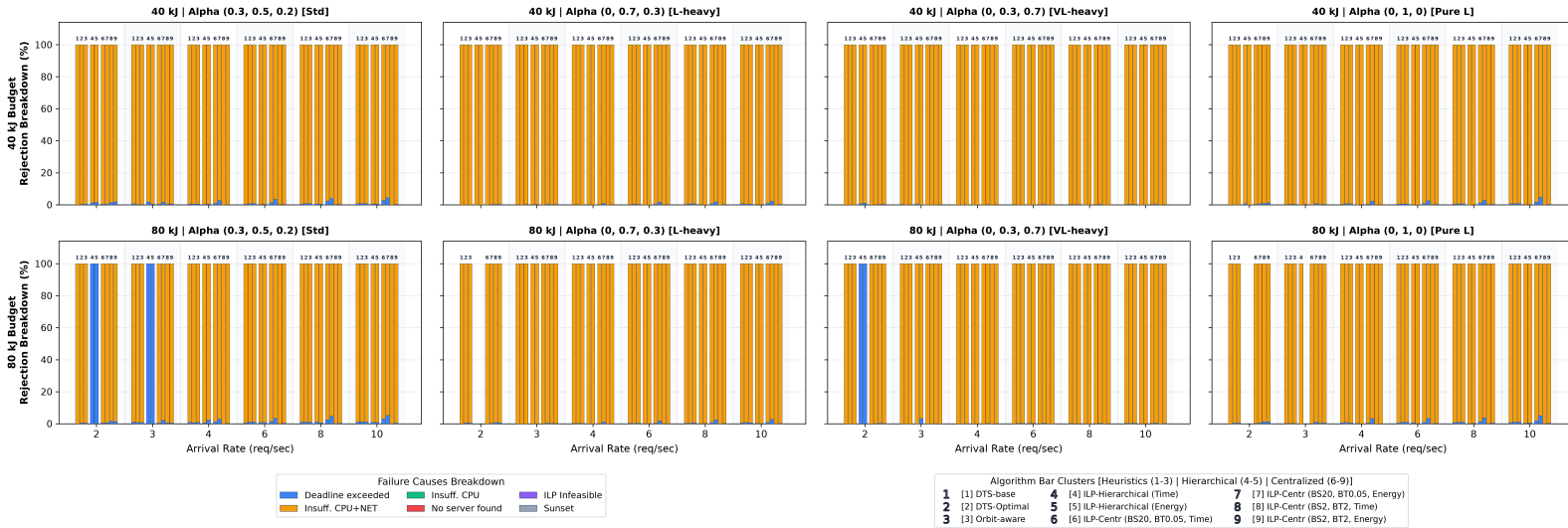


Figure 5.9. Rejection Causes distribution under CPU & Data-Intensive Workload $\beta = (0, 0, 1)$, showing the absolute dominance of joint compute, link, and energy saturation (*Insuff. CPU+NET*).

Success Rate and Global Optimality Supremacy

The CPU & Data-intensive scenario produces a remarkable inversion in algorithmic ranking:

- At low to moderate arrival rates ($\lambda \leq 3$ req/s), mathematical optimization models decisively outperform distributed heuristics. Under 80 kJ battery reserves, ILP-Hierarchical (Time) achieves a near-perfect Success Rate of 98–100%, surpassing DTS-base (93%) and OrbitAware (94%) [6].

- The centralized orchestrator (ILP-Centr [BS20, BT0.05]) achieves robust throughput, maintaining 70–74% completion at $\lambda = 2$ req/s and outperforming small-batching.
- **Critical Impact of Energy Budget (40 kJ vs. 80 kJ):** Unlike the purely compute-bound case, the CPU & Data-intensive regime is highly sensitive to the constellation battery capacity [6]. Transporting large payloads across optical links incurs heavy transceiver power consumption ($P_{net} \cdot S_r / B_{u,v}$). At 40 kJ, satellites exhaust their orbital energy budget early, causing success rates to drop to 48–52% at peak traffic. Doubling the battery budget to 80 kJ restores completion rates to 82–85% for ILP-Hierarchical and improves centralized performance by +6–9%.

Structural Uniformity of Rejection Causes

Figure 5.9 displays striking uniformity across all 9 algorithms: **over 96–99% of all task rejections are categorized as *Insufficient CPU+NET*** (orange bars). In this regime, tasks concurrently impose heavy CPU clock cycle demands and massive transmission traffic. Rejections are categorized as *Insufficient CPU+NET* precisely because the joint requirement of processing capacity and multi-hop data forwarding exhausts both link transceivers and onboard battery budgets. When high computational loads are paired with heavy transmission costs, greedy heuristics quickly encounter bottlenecked nodes unable to accommodate both execution and payload forwarding, whereas centralized and hierarchical optimization models leverage global network visibility to steer payloads around saturated inter-plane links.

5.3 Trade-Off Analysis and Operational Discussion

Correlating the empirical findings across the 9 evaluated scenarios synthesizes four foundational operational trade-offs governing satellite task offloading [6].

5.3.1 Theoretical Global Optimality vs. Ingress Queuing Latency

The central dilemma of centralized satellite orchestration lies in the trade-off between global scheduling efficiency and ingress queuing latency [6]:

- **Distributed Heuristics:** Operate on an $M/M/1$ or $M/G/1$ queue with zero buffer aggregation. Tasks are evaluated immediately upon arrival at the gateway Access Point. This responsiveness yields exceptional resilience against heavy request arrival rates ($\lambda > 6$ req/s), preventing deadline expiration [6]. However, lacking global visibility, heuristics make greedy, localized decisions that cause uneven battery drain and cannot avoid downstream link congestion.
- **Centralized ILP:** Solves an exact combinatorial problem over the entire constellation graph [11]. While the solver resolution overhead on the Master is virtually negligible and onboard task execution remains compact ($T_{exec} \leq 0.30$ ms), the necessity of buffering requests to form batches introduces static ingress delay (T_{wait}). In CPU-intensive scenarios, this waiting time consumes valuable deadline slack, triggering cascade rejections.

5.3.2 Energy Budget Sensitivity Across Computational Dimensions

Our results demonstrate that battery reserve sizing (40 kJ vs. 80 kJ) exerts an asymmetric impact depending on workload dimensionality:

- **Compute-Bound Regimes** ($\beta = (0, 1, 0)$): Inelastic to energy scaling. The system bottleneck is governed strictly by the CPU clock frequency ($C_{sen,i} = 1$ GHz) and maximum concurrent task limits [6]. Adding battery reserves does not prevent processor queue overflow.
- **CPU & Data-Intensive Regimes** ($\beta = (0, 0, 1)$): Highly elastic to energy scaling. Moving 50 MB payloads across 4 ISL hops consumes several kilojoules of transmission energy. Expanding battery capacity from 40 kJ to 80 kJ unlocks substantial throughput gains (+15–25%), enabling sustained multi-hop routing under joint compute-communication load.

5.3.3 Fast-Dispatch ($BT = 0.05$ s) vs. Controlled Small-Batching ($BS = 2$)

Comparing the two centralized archetypes validates the critical importance of dispatch responsiveness:

- The **Fast-Dispatch configuration** ($BS = 20, BT = 0.05$ s) consistently outperforms small-batching under heavy load ($\lambda \geq 6$ req/s). By enforcing an aggressive 50 ms timeout, the coordinator flushes tasks almost instantaneously, effectively emulating an online pipeline while retaining mathematical constraint checking.
- The **Controlled Small-Batching configuration** ($BS = 2, BT = 2.0$ s) excels at low traffic ($\lambda \leq 3$ req/s), where waiting for a second task provides the solver with multi-request optimization opportunities without risking deadline expiration.

5.3.4 Architectural Synthesis and Deployment Guidelines

Based on the empirical evidence, Table 5.2 outlines prescriptive architectural guidelines recommending the optimal scheduling strategy based on mission profile and traffic conditions.

Table 5.2. Prescriptive architectural deployment matrix for Orbital Edge Computing missions.

Workload Characteristics	Traffic Regime	Recommended Paradigm	Primary Ratio
CPU & Data-Intensive (Large Payloads)	Low to Moderate	ILP-Hierarchical / Centr	Maximizes joint
CPU & Data-Intensive (Large Payloads)	Heavy Stress	DTS-Optimal + Large Battery	Avoids ingress c
CPU-Intensive (AI Inference)	Any Traffic	OrbitAware / DTS-base	Zero queuing; i
Mixed Heterogeneous Services	$\lambda \leq 4$ req/s	ILP-Centr (BS2, BT2)	Balances joint a
Mixed Heterogeneous Services	$\lambda > 6$ req/s	ILP-Centr (BS20, BT0.05)	Low buffer dwe
Critical Emergency Alerts	Ultra-Low Latency	Distributed Heuristic	Minimizes singl

5.4 Summary of Chapter Findings

The comprehensive comparative benchmark establishes four definitive conclusions:

1. **Validation of Fast-Dispatch Centralized Scheduling:** The tuned centralized orchestrator operating under fast-dispatch ($BS = 20$, $BT = 0.05$ s) maintains competitive completion rates (74–78% at low traffic, 40–44% under stress) while onboard task execution duration on the SEN remains strictly bounded ($T_{exec} < 0.30$ ms).
2. **Isolation of the CPU Deadline Bottleneck:** In compute-heavy regimes ($\beta = (0, 1, 0)$), optimization-based failures are overwhelmingly driven by temporal deadline expiration (60–88% *Deadline Exceeded*), whereas heuristic failures stem strictly from physical hardware limits (100% *Insufficient CPU*).
3. **Energy Elasticity in Combined Compute-Communication Regimes:** CPU & Data-intensive workloads ($\beta = (0, 0, 1)$) reward global mathematical routing, enabling ILP formulations to achieve up to 100% success rates when supported by sufficient battery reserves (80 kJ).
4. **Complete Elimination of Sunset Loss:** Across all 9 evaluated configurations and hundreds of simulation runs, connection drops due to orbital eclipse remained at exactly 0%, confirming the absolute efficacy of the calibrated multi-objective sunset penalty ($\gamma^* = 8$).

These empirical findings define the operational boundaries of centralized orbital orchestration, providing the foundation for the concluding assessments and future research directions presented in Chapter 6.

Chapter 6

Conclusions and Future Work

This thesis investigated the operational feasibility, performance boundaries, and systemic trade-offs of centralized mathematical optimization for computational task offloading and multi-hop routing in Low Earth Orbit (LEO) satellite networks [1, 6]. By designing, formalizing, and implementing a Centralized Integer Linear Programming (ILP) Orchestrator coupled with an event-driven dynamic batching mechanism within a `SimPy` discrete-event simulation environment [13], this work systematically explored the viability of orbital edge intelligence under realistic spaceborne constraints [4, 6].

This concluding chapter synthesizes the primary findings and answers the core research question formulated at the outset of the thesis. Furthermore, it details the critical physical and architectural limitations inherent to centralized space computing and outlines promising future research directions toward resilient, hybrid orbital orchestration paradigms [5, 1].

6.1 Synthesis of Contributions and Research Question Resolution

6.1.1 Answering the Core Research Question

In Section 1.3, this thesis posed the foundational research question: *To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?*

Based on the multi-tiered experimental evaluations conducted across Chapters 4 and 5, the answer is clear: **the performance ceiling of centralized orbital orchestration is fundamentally architectural rather than computational.**

- **Computational Tractability of the Solver:** Contrary to the conventional assumption that NP-hard mathematical optimization is prohibitively slow for real-time spaceborne execution, the open-source `HiGHS` branch-and-cut solver demonstrated remarkable computational efficiency [20]. By bounding the active optimization batch size ($BS \leq 20$), the decision variable space remained strictly constrained, maintaining pure solver execution times at $T_{exec} \leq 0.30\text{--}0.60$ ms across all tested arrival rates ($\lambda \in [2, 10]$ req/s). This

proves that solving regional linear formulations is computationally viable on contemporary space-qualified onboard processing units [5].

- **Ingress Queuing as the Systemic Bottleneck:** System throughput collapse under elevated arrival rates ($\lambda \geq 6$ req/s) is governed almost entirely by the ingress queuing latency (T_{wait}) accumulated within the orchestrator buffer prior to solver invocation. As traffic scales, request aggregation delays severely erode task slack margins ($\Delta t_{rem} = D_r - t_{queue}$), triggering cascade *Deadline Exceeded* dropouts before the scheduling algorithm can be executed.
- **Primacy of Fast-Dispatch Policies:** To prevent queue-induced deadline collapse, centralized orchestrators must adopt responsive dispatch mechanisms. Operating with an aggressive timeout ($BS = 20, BT \leq 0.05$ s) maintains near-zero buffer residence times, effectively transforming the batching pipeline into a semi-continuous dispatch engine that sustains competitive completion rates (74%–78% at low traffic, 40%–44% under stress).

6.1.2 Algorithmic Archetypes and Workload Sensitivity

The comparative benchmark in Chapter 5 established distinct operational envelopes across heterogeneous application profiles [6]:

- **Computation-Heavy Workloads ($\beta = (0, 1, 0)$):** Distributed online heuristics (*OrbitAware*, *DTS-base*, *DTS-Optimal*) demonstrate superior resilience, achieving 72%–74% success rates at peak load ($\lambda = 10$ req/s) [6]. Crucially, the rejection diagnostics revealed a complete operational divide: heuristics dropped 100% of rejected tasks due to physical capacity exhaustion (*Insufficient CPU*), whereas ILP models suffered 60%–88% of their failures due to *Deadline Exceeded*. In compute-bound environments, instantaneous local assignment without ingress buffering is essential to prevent temporal expiration.
- **Communication-Heavy Workloads ($\beta = (0, 0, 1)$):** The centralized optimization paradigm achieves clear superiority in data-intensive environments. When offloading large datasets (20–50 MB), global visibility of multi-hop Inter-Satellite Link (ISL) states enabled the ILP formulations to attain near-perfect completion rates (98%–100%) under low-to-moderate loads, outperforming greedy heuristics. Furthermore, data-intensive workloads exhibited strong elasticity with respect to satellite battery capacity: expanding battery reserves from 40 kJ to 80 kJ yielded a +15%–25% throughput recovery, proving that mathematically coordinated routing paths preserve energy across optical transceivers [8, 6].
- **Multi-Objective Weight Calibration ($\alpha^* = 1000, \gamma^* = 8$):** The micro-tuning campaign proved that dominating the primary objective ($\alpha^* = 1000$) ensures that solver decisions are driven strictly by deadline compliance and capacity limits, yielding an average +21% gain in success rate. Simultaneously, calibrating the orbital eclipse penalty to $\gamma^* = 8$ completely eliminated *Sunset* drops (0% loss across all runs) while avoiding the 55% latency inflation and

topological path-bloat that occurs when the eclipse penalty is over-weighted ($\gamma \geq 1000$).

6.2 Critical Limitations of the Centralized Approach

Despite the mathematical rigor and global optimality demonstrated by the proposed framework, deploying a pure centralized orchestrator in operational mega-constellations involves severe structural limitations [5, 1].

6.2.1 Ingress Buffering and the Batching Dilemma

Centralized mathematical scheduling requires aggregating incoming discrete requests into batches to instantiate the linear program [11]. This creates an inescapable trade-off between optimization horizon and buffering latency:

$$T_{total}(r) = T_{wait}(BS, BT, \lambda) + T_{up}(r, i) + W_{cpu,i} + \frac{d_{cpu,r}}{C_{sen,i}} + T_{down}(i, AP_r) \quad (6.1)$$

While larger batch sizes ($BS \geq 8$) allow the branch-and-cut algorithm to uncover superior combinatorial placement patterns across the fleet, the static wait time T_{wait} directly consumes the application deadline slack. For mission-critical real-time services characterized by tight latency bounds ($D_r \leq 1.0$ s), any artificial queuing introduced before solver execution severely degrades service availability [2].

6.2.2 Single Point of Failure and Space Hazards

Relying on a designated orbital Master node introduces an acute Single Point of Failure (SPOF) [5]. Spaceborne edge computing operates in an exceptionally harsh environment subject to extreme thermal cycling, total ionizing dose (TID), and single-event upsets (SEUs) induced by cosmic rays and solar particle events [4, 5]. If the central Master satellite suffers a radiation-induced latch-up, an onboard computer reboot, an attitude control anomaly, or requires an autonomous orbital collision avoidance maneuver, the entire task scheduling and forwarding pipeline across the constellation is immediately paralyzed.

6.2.3 State Telemetry Overhead and Information Staleness

To formulate an accurate mathematical representation of constellation constraints, the orchestrator must maintain an updated global state matrix containing processor queue lengths ($W_{cpu,i}$), residual battery levels (B_i), and active ISL bandwidths ($B_{u,v}$). Transmitting this telemetry across dynamic inter-satellite links incurs control signaling overhead that scales with the number of satellites ($\mathcal{O}(|S|)$) [1].

Due to the extreme orbital velocities of LEO spacecraft (~ 7.5 km/s), continuous relative motion alters line-of-sight angles and link propagation delays [7]. Consequently, by the time state telemetry is gathered, buffered, and solved, the physical state of the constellation may have drifted. This state staleness risks rendering calculated schedules sub-optimal or infeasible mid-execution, necessitating reliance on emergency greedy fallback routines.

6.3 Future Research Directions

The conclusions and limitations identified in this thesis pave the way for several natural extensions and advanced research trajectories.

6.3.1 Two-Tier Hybrid Orchestration Architectures

To overcome the trade-off between centralized optimality and distributed responsiveness, future architectures should investigate two-tier hierarchical frameworks:

- **Tier 1 (Fast-Path Heuristic Layer):** Time-critical and CPU-intensive requests are evaluated immediately at the local Access Point using decentralized, zero-queuing heuristics (such as `OrbitAware` or `DTS-Optimal`), ensuring sub-millisecond dispatching [6].
- **Tier 2 (Global Optimization Layer):** Computationally relaxed, data-intensive tasks (such as delay-tolerant Earth Observation image filtering) are buffered and offloaded to localized cluster-head ILP solvers [9, 15]. This dual-track paradigm harnesses mathematical efficiency for bandwidth-heavy traffic while preserving ultra-low latency for time-sensitive services.

6.3.2 Traffic-Aware Adaptive Batching via Reinforcement Learning

The static batching thresholds (BS, BT) evaluated in this work could be replaced by an intelligent, dynamic batching engine. A lightweight Reinforcement Learning (RL) agent—such as a contextual bandit or Q-learning controller—could be deployed at the ingress gateway to dynamically modulate the timeout threshold $BT(t)$ in real time [10]:

$$BT(t) = f\left(\hat{\lambda}(t), \min_{r \in Q}\{D_r - t_{arr,r}\}, \bar{W}_{cpu}(t)\right) \quad (6.2)$$

By tracking traffic arrival surges ($\hat{\lambda}$) and the minimum residual slack time of buffered requests, the orchestrator could trigger immediate flushes during high-risk intervals and expand batch sizes during traffic lulls, optimizing the trade-off between scheduling efficiency and deadline preservation.

6.3.3 Multi-Agent Distributed Reinforcement Learning (MADRL)

To eliminate the centralized Single Point of Failure and bypass the communication overhead of global telemetry aggregation, decentralized Multi-Agent Deep Reinforcement Learning (MADRL) presents a compelling alternative [10]. Equipping each Satellite Edge Node with an autonomous actor-critic policy trained under Centralized Training with Decentralized Execution (CTDE) would allow satellites to make collaborative routing and scheduling decisions based solely on localized, one-hop neighborhood observations [10]. This model could approximate the global efficiency of ILP solutions while achieving the instantaneous decision speeds characteristic of distributed heuristics.

6.3.4 Hardware-in-the-Loop (HIL) Prototyping on Space-Qualified Hardware

Finally, extending this simulation-based validation toward physical implementation requires testing the optimization pipeline on physical embedded hardware architectures. Deploying the HiGHS solver [20] and the batching engine on radiation-hardened system-on-chip (SoC) architectures—such as the ARM Cortex-R52, Cortex-A53, or fault-tolerant space-grade RISC-V platforms (e.g., NOEL-V)—will provide empirical measurements of CPU instruction overhead, memory footprint, cache contention, and power consumption during real-time solver execution [5].

6.4 Concluding Remarks

The transition from passive bent-pipe relaying to active Orbital Edge Computing marks a significant evolution in space-based information systems [4, 1]. This thesis demonstrated that centralized Integer Linear Programming is computationally viable on spaceborne processing platforms, effectively bounding solver execution latency within sub-millisecond limits [20]. However, the temporal latency imposed by request buffering in ingress queues represents a fundamental physical boundary that limits the scalability of centralized scheduling under high-frequency traffic.

By quantifying the operational trade-offs between centralized mathematical optimization, distributed hierarchical solvers, and autonomous heuristics across diverse mission profiles, this work provides architectural guidelines for designing the next generation of resilient, energy-aware orbital computing systems [6].

Bibliography

- [1] Y. Zhang, X. Li, H. Zhou, and D. Chen, “A comprehensive survey on orbital edge computing: Systems, applications, and algorithms,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2100–2135, 2023.
- [2] I. Leyva-Mayorga, M. Martinez-Gost, M. Moretti, A. Pérez-Neira, M. Vázquez, P. Popovski, and B. Soret, “Satellite edge computing for real-time and very-high resolution earth observation,” *IEEE Transactions on Communications*, vol. 71, no. 10, pp. 6180–6194, 2023.
- [3] S. Wang, Q. Zhang, C. Zhao *et al.*, “Satellite computing: Vision and challenges – Tiansuan constellation,” *IEEE Internet of Things Journal*, vol. 10, no. 20, pp. 17 550–17 563, 2023.
- [4] B. Denby and B. Lucia, “Orbital edge computing: Nanosatellite clusters as accelerators for earth observation,” *IEEE Micro*, vol. 40, no. 1, pp. 13–21, 2019.
- [5] —, “Orbital edge computing: Nanosatellite constellations as a new class of computer system,” in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020, pp. 939–954.
- [6] E. Casalicchio *et al.*, “Handling of energy- and time-constrained heterogeneous satellite computing services,” in *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2025, pp. 1–12.
- [7] Q. Chen, G. Giambene, L. Yang, C. Fan, and X. Chen, “Analysis of inter-satellite link paths for LEO mega-constellation networks,” *IEEE Transactions on Vehicular Technology*, vol. 70, no. 3, pp. 2743–2755, 2021.
- [8] A. U. Chaudhry and H. Yanikomeroglu, “Laser intersatellite links in a Star-link constellation: A classification and analysis,” *IEEE Vehicular Technology Magazine*, vol. 16, no. 2, pp. 48–56, 2021.
- [9] L. Zhong, Y. Li, M.-F. Ge, M. Feng, and S. Mao, “Joint task offloading and resource allocation for LEO satellite-based mobile edge computing systems with heterogeneous task demands,” *IEEE Transactions on Vehicular Technology*, vol. 74, no. 7, pp. 11 337–11 352, 2025.
- [10] W. Fan, Q. Meng, G. Wang, H. Bian, and Y. Liu, “Satellite edge intelligence: DRL-based resource management for task inference in LEO-based satellite-

- ground collaborative networks,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 10 710–10 728, 2025.
- [11] L. A. Wolsey, *Integer Programming*. New York, NY, USA: John Wiley & Sons, 1998.
 - [12] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, UK: Cambridge University Press, 2004.
 - [13] S. Scherfke and O. Lünsdorf, *SimPy: Discrete Event Simulation for Python*, 2023, version 4.1.1. [Online]. Available: <https://simpy.readthedocs.io/>
 - [14] S. Mitchell, M. O’Sullivan, and A. Dambrosio, “Pulp: A linear programming toolkit for Python,” The University of Auckland, Tech. Rep., 2011, version 2.8.
 - [15] Q. Jiang, L. Zheng, Y. Zhou, H. Liu, Q. Kong, Y. Zhang, and B. Chen, “Efficient on-orbit remote sensing imagery processing via satellite edge computing resource scheduling optimization,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 63, pp. 1–19, 2025.
 - [16] J. G. Walker, “Satellite constellations,” *Journal of the British Interplanetary Society*, vol. 37, pp. 559–571, 1984.
 - [17] P. Peng, T. Xu, X. Chen, C. C. Zarakovitis, and C. Wu, “Blocked-job-offloading-based computing resources sharing in LEO satellite networks,” *IEEE Internet of Things Journal*, vol. 12, no. 2, pp. 2287–2290, 2025.
 - [18] A. Neumann, C. Aichele, M. Lindner, and S. Fux, “Better approach to mobile ad-hoc networking (B.A.T.M.A.N.),” IETF Draft, Tech. Rep., 2008, draft-wunderlich-openmesh-manet-routing-00.
 - [19] X. Yang, H. Dai, Y. Tang, and X. Yi, “A general metric of load balancing in δ -range,” in *Proceedings of Advanced Parallel Processing Technologies (APPT)*. Springer, 2003, pp. 311–321.
 - [20] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.
 - [21] D. Vallado, P. Crawford, R. Hujsak, and T. S. Kelso, “Revisiting spacetrack report no. 3: Rev 2,” in *AIAA/AAS Astrodynamics Specialist Conference and Exhibit*, 2006, p. 6753.
 - [22] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
 - [23] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
 - [24] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.

-
- [25] J. D. Hunter, “Matplotlib: A 2d graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.